

Technische Universität Berlin
School IV - Electrical Engineering and Computer Science
Intelligent Multi-Robot Coordination Lab

Master Thesis

ST - Db - RRT: Kinodynamic Motion Planning in Known Dynamic Environments

presented by

Franz Friedrich Queißner

Matr.-Nr.: 391301

M.Sc. Computer Engineering

Date of submission: July 21, 2026

Supervisor:
Dr. Andreas Orthey

Examiners:
Prof. Dr. Wolfgang Hönig
Prof. Dr. Marc Toussaint

Declaration of Authorship

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit eigenständig ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen die den benutzten Quellen und Hilfsmitteln unverändert oder sinngemäß entnommen sind, habe ich als solche kenntlich gemacht.

Sofern generative KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die jeweiligen Einsatzzwecke (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

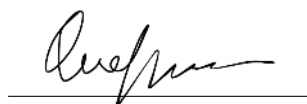
Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 15. Februar 2023. https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMB1/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf habe ich zur Kenntnis genommen. Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Declaration of Authorship (Translated)

I hereby affirm that I have composed the present thesis independently, without the assistance of third parties and solely using the sources and aids listed. All passages that have been taken from the sources and aids used, either verbatim or in substance, have been marked as such.

If generative AI tools were employed, I have named product names, manufacturers, the specific software version used in each case, and the respective purposes of use (e.g., linguistic review and improvement of texts, systematic research). I take full responsibility for the selection, incorporation, and all results of the AI-generated output used by me.

I have taken note of the Statute for the Assurance of Good Scientific Practice at TU Berlin of 15 February 2023. https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMB1/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf. I further declare that I have not submitted this work, in the same or similar form, to any other examination authority.



Berlin,
July 21, 2026

Franz Queißner

Abstract

Robots that leave structured, static environments must plan motions that are kinodynamically feasible and safe among moving obstacles. Both requirements are well studied in isolation. To generate kinodynamically feasible motions, sampling-based kinodynamic planners such as iDb-RRT and GBRRT find solutions fast and without a steering function, but only in static environments. In contrast, to generate motions among moving obstacles, planners like ST-RRT plan optimally through space-time with moving obstacles and unknown arrival time, but require a steering function and are thus limited to systems which can explicitly solve a two-point boundary value problem.

This thesis combines the two lines of work and presents three new planners, ST-Db-RRT, ST-GBRRT and KIST, that plan kinodynamically feasible trajectories in dynamic environments without requiring a steering function or a known arrival time. The three planners cover different kinodynamic search paradigms: ST-Db-RRT expands with randomly selected discontinuity-bounded motion primitives and defers feasibility to a final trajectory optimization at the end, whereas KIST and ST-GBRRT maintain a kinodynamically feasible tree through forward propagation and differ in how much effort they invest into best-input expansion and heuristic guidance. Since all three share the same space-time sampling strategy of ST-RRT*, these paradigms can be studied independently of the sampling strategy, isolating how they behave when lifted into space-time. We analyze which properties of the underlying planners are preserved or removed and show that planners without an explicit time dimension, such as iDb-RRT, are incomplete in dynamic environments.

For the empirical evaluation, we design a suite of dynamic benchmark environments, evaluate the influence of the planners central parameters, and compare the planners across four dynamical systems. The results show that planning in space-time achieves 1) lower costs in dynamic environments 2) is necessary where robots face narrow time passages 3) and that deferring feasibility to a final optimization-based repair step is faster than optimization-free approaches in most cases.

ST-Db-RRT stands out as the most promising of the three space-time planners. It is able to solve every benchmark instance, reaches first solutions up to $39\times$ faster, and stays within 6% of the best cost, while the optimization-free planners KIST and ST-GBRRT remain valuable where the optimization-based repair step is fragile.

Keywords

motion planning, kinodynamic motion planning, dynamic environment, space-time planning, benchmark

Deutsche Kurzzusammenfassung

Roboter, die außerhalb strukturierter, statischer Arbeitszellen eingesetzt werden, müssen Bewegungen planen, die sowohl kinodynamisch ausführbar als auch in dynamischen Umgebungen sicher sind. Beide Anforderungen wurden bislang weitgehend getrennt erforscht: Samplingbasierte kinodynamische Planer wie iDb-RRT und GBRRT finden effizient kinodynamisch gültige Trajektorien, ohne aufwendige Randwertprobleme lösen zu müssen - allerdings nur in statischen Umgebungen. ST-RRT* hingegen plant in Raum und Zeit mit bewegten Hindernissen und ohne vorgegebene Ankunftszeit, ist für komplexere Systeme jedoch auf die Lösung von Randwertproblemen angewiesen. Die vorliegende Arbeit führt beide Forschungsrichtungen zusammen und stellt drei neue Planer vor ST-Db-RRT, ST-GBRRT und KIST, die kinodynamisch ausführbare Trajektorien in dynamischen Umgebungen planen, ohne Randwertprobleme zu lösen oder eine bekannte Ankunftszeit vorauszusetzen. Alle drei übernehmen die Raumzeit-Samplingstrategie von ST-RRT*, wenden sie jedoch auf unterschiedliche kinodynamische Suchparadigmen an, was eine systematische Untersuchung dieser Paradigmen in der Raumzeit ermöglicht. Wir analysieren, welche theoretischen Eigenschaften der zugrunde liegenden Planer erhalten bleiben, verloren gehen oder wiederhergestellt werden können, und zeigen darüber hinaus, dass Planer ohne explizite Zeitdimension in dynamischen Umgebungen unvollständig sind. Für die empirische Evaluation entwerfen wir eine Reihe dynamischer Benchmarkprobleme, untersuchen den Einfluss der zentralen Parameter und vergleichen die Planer über vier dynamische Systeme hinweg. Die Ergebnisse zeigen, dass Planung in der Raumzeit nicht nur notwendig ist, um anspruchsvolle dynamische Probleme zu lösen, sondern auch niedrigere Kosten erzielt. Als Überlegen erweist es sich, die kinodynamische Ausführbarkeit in einen einzelnen optimierungsbasierten Schritt auszulagern: ST-Db-RRT löst jedes Problem im Benchmark, findet erste Lösungen bis zu $39\times$ schneller und erreicht Kosten innerhalb von 6% des jeweils besten Werts, während die optimierungsfreien Planer sich in Problemen wo die Optimierung instabil ist, bewähren.

Acknowledgements

I want to express my sincere gratitude to a few people who have supported and guided me throughout the process that led me to completing this thesis. First, to my supervisor, Dr. Andreas Orthey, for his guidance, encouragement, and availability that kept me motivated and focused until submission, and to Prof. Dr. Wolfgang Hönig for his valuable feedback and suggestions that have helped me improve the quality of my work. I want to thank the people in the Intelligent Multi-Robot Coordination Lab for creating such a friendly and supportive environment that made it a pleasure to work in and to Viktor Lorentz for his excellent latex thesis template. Außerdem möchte ich mich bei meinen Eltern, Großeltern, Schwestern und Freunden bedanken. Vielen Dank für eure Geduld, euer Verständnis und eure Unterstützung während dieser Zeit. Finally, I would like to thank my girlfriend, for her love, patience and support during this time.

Additionally, I would like to acknowledge the work of preceding researchers and highlight the contributions and tools that made this thesis possible:

- **Dynoplan and Dynobench:** The work of Joaquim Ortiz-Haro et al. [1], [2], [3] on Dynoplan and Dynobench has been the practical basis for this thesis. The authors have provided a solid theoretical and practical foundation for the development of the thesis.
- **ST-RRT* and OMPL:** The work of Francesco Grothe et al. [4] on ST-RRT* and the implementation in the Open Motion Planning Library (OMPL) [5] have been instrumental in the development of this thesis.
- **Claude (Opus 4.8):** Anthropic's large language model has been used for drafting and refining text, for completion, debugging, refactoring of code, and as a tool for explorative research, summarization, and brainstorming.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Research Objectives	2
1.3	Contribution	3
2	Related Work	4
2.1	Planning in Dynamic Environments	4
2.2	Kinodynamic Motion Planning	5
2.3	Kinodynamic Planning in Dynamic Environments	5
3	Background	7
3.1	Motion Planning Definition	7
3.2	Search-Based Motion Planning	8
3.3	Sampling-Based Motion Planning	8
3.3.1	RRT	9
3.3.2	RRT* and Asymptotic Optimality	10
3.3.3	Bidirectional Search	10
3.4	Space-Time Motion Planning	11
3.4.1	Dynamic Environments	11
3.4.2	Space-Time RRT*	11
3.5	Kinodynamic Motion Planning	13
3.5.1	Kinodynamic Constraints	13
3.5.2	Tree Expansion	14
3.5.3	The Two-Point Boundary Value Problem	14
3.5.4	Hybrid and Optimization-based Planning	15
3.5.5	Bidirectional Planning Without BVP	15

4	Kinodynamic Space-Time Planning	16
4.1	Notation and Problem Formulation	17
4.2	Shared Components	18
4.2.1	Space-Time Search Strategy	18
4.2.2	Expansion and Connection Functions	19
4.3	Space-Time Db-RRT	21
4.3.1	Naive Db-RRT for Dynamic Environments	22
4.3.2	ST-Db-RRT	23
4.4	KIST	26
4.4.1	Algorithm Description	27
4.4.2	Completion Policy: Heuristic Completion	28
4.4.3	Completion Policy: Replay	29
4.4.4	Theoretical Analysis	31
4.5	ST-GBRRT	31
4.5.1	Algorithm Description	32
5	Experiments and Benchmark	35
5.1	Dynamical Systems	35
5.2	Motion Primitives	37
5.3	Environments	37
5.4	Algorithms	39
5.5	Parameter Study	42
5.5.1	Time-Optimal vs. Random Motion Primitives	42
5.5.2	Primitive Density and Discontinuity Bound	42
5.5.3	Sampling Time Bounds	45
5.5.4	Time Weight	47
5.5.5	Parameter Selection	49
5.6	Planner Comparison	50
5.6.1	Db-RRT vs ST-Db-RRT	50
5.6.2	Space-Time Planner Comparison	52
6	Discussion	57
6.1	Parameter Study	57
6.1.1	Primitive Density and Discontinuity Bound	57
6.1.2	Sampling Time Bounds	58
6.1.3	Time Weight	59
6.2	Planner Comparison	59
6.2.1	Db-RRT vs ST-Db-RRT	60
6.2.2	Space-Time Planner Comparison	60
6.3	Key Findings	62
6.4	Limitations	64
6.5	Future Research Directions	64
7	Conclusion	65
	Bibliography	66
	Appendices	69

Appendix A Planner Comparison: Cost-Success Plots	70
A.1 Success-Plots: Db-RRT vs. ST-Db-RRT	70
A.2 Success-Plots: ST-Db-RRT vs. KIST vs. ST-GBRRT	70
A.3 Trajectory Plots	70

List of Figures

4.1	Time-optimal and time-suboptimal motions	25
4.2	KIST Illustration	28
5.1	Example of motion primitives	37
5.2	Overview of dynamic environments 1	38
5.3	Overview of dynamic environments 2	40
5.4	Overview of dynamic environments 3	40
5.5	Overview of dynamic environments 4	41
5.6	Overview of dynamic environments 5	41
5.7	Benchmark: Random vs Time-Optimal Motion Primitives	43
5.8	Parameter Study Motion Primitives: ST-Db-RRT	43
5.9	Parameter Study Motion Primitives: KIST	44
5.10	Parameter Study Motion Primitives: ST-GBRRT	44
5.11	Parameter Study Time Bounds: ST-Db-RRT	46
5.12	Parameter Study Time Bounds: KIST	46
5.13	Parameter Study Time Bounds: ST-GBRRT	47
5.14	Parameter Study Time Weight: ST-Db-RRT	48
5.15	Parameter Study Time Weight: KIST	48
5.16	Parameter Study Time Weight: ST-GBRRT	49
5.17	Planner Comparison: Db-RRT vs ST-Db-RRT	51
5.18	Planner Comparison: Db-RRT vs ST-Db-RRT in Static Environments . . .	52
5.19	Example ST-Db-RRT solution trajectories	53
5.20	Planner Comparison: ST-Db-RRT vs KIST vs ST-GBRRT	54
5.21	Planner Comparison: Selected Cost-Success Plots	55
A.1	Planner Comparison: Db-RRT vs ST-Db-RRT for Unicycle Systems	71
A.2	Planner Comparison: Db-RRT vs ST-Db-RRT for Car and Planar Rotor Systems	72
A.3	Planner Comparison: Cost-Success Plots for Unicycle Systems	73
A.4	Planner Comparison: Cost-Success Plots for Car and Planar Rotor Systems	74
A.5	Solution Trajectories	75

1. Introduction

Robots are increasingly leaving structured and static workcells and entering spaces that they share with humans, machines, and other robots: mobile robots navigate warehouses, autonomous vehicles merge into flowing traffic, and fleets of manipulators cooperate in assembly tasks. A core capability behind all of these applications is motion planning, the problem of computing a collision-free motion that brings a robot from its current state to a goal.

However, in order to leave the structured workcell, a geometric path through a static world is not sufficient. The computed motion must be feasible for the robot, respecting its dynamics and actuation limits, and it must be able to handle moving obstacles. Working towards this ambition, the planned path becomes a planned trajectory that specifies not only *where* the robot moves, but also *when*.

1.1 Motivation

Planning feasible motions and avoiding dynamic obstacles have both been studied intensively, but largely in isolation. The first is addressed by *kinodynamic* motion planning [6], which takes kinematic and differential constraints of the robot into account, such as the turning rate of a car or the limited acceleration of a torque-constrained system. The second is addressed by planning in *known dynamic environments* [7]: when the trajectories of the moving obstacles are known in advance, as for example in multi-robot assembly planning, time can be treated as an additional planning dimension and the planner can anticipate the motion of the obstacles instead of merely reacting to it [4].

Each subproblem is solved well by the current state of the art. Recent kinodynamic planners such as Iterative Discontinuity-bounded Rapidly-exploring Random Tree (iDb-RRT) [3] and Generalized Bidirectional RRT (GBRRT) [8] find kinodynamically feasible trajectories fast for a wide range of systems, but only in static environments. By contrast, Space-Time-RRT* (ST-RRT*) [4] plans optimally through space-time with moving obstacles and unknown arrival time, but only for holonomic or steering-capable systems. The natural next step is to combine the two subproblems and plan kinodynamically feasible trajectories in dynamic environments with unknown arrival time.

Lifting ST-RRT* into the kinodynamic setting, however, is not straightforward, as it requires a steering function to connect two states directly. This is trivial for holonomic systems, but for kinodynamic systems it requires solving a two-point boundary value problem (BVP), which is computationally expensive or not available for many systems. The kinodynamic planners iDb-RRT and GBRRT, however, both avoid the steering function,

by deferring feasibility to one single optimization step or by never leaving the set of feasible motions. To the best of our knowledge, no existing kinodynamic motion planner for general dynamical systems simultaneously avoids steering functions and plans through space-time with dynamic obstacles and unknown, unbounded arrival time.

This thesis therefore combines the space-time search of ST-RRT* with the kinodynamic expansion strategies of iDb-RRT and GBRRT, yielding three new planners, ST-Db-RRT, ST-GBRRT and KIST, that plan for kinodynamic systems in space-time for dynamic environments without requiring a steering function or a known arrival time. All three planners share the same space-time search strategy but apply it to different kinodynamic search paradigms, which allows us to study how the established paradigms perform when lifted to space-time. Beyond the algorithms themselves, we analyse the theoretical consequences of the space-time extension, showing that probabilistic completeness of GBRRT is preserved, but the resolution completeness of Db-RRT is only preserved with bounded time. The asymptotic optimality of ST-RRT* through rewiring is not preserved in the kinodynamic setting, though it remains a promising direction for future work. We further evaluate the influence of the planners parameters, before comparing them in a benchmark of newly designed dynamic environments, demonstrating that planning in space-time solves problems on which planners lacking an explicit time dimension fail.

1.2 Research Objectives

In this thesis, we investigate the problem of kinodynamic motion planning in dynamic environments with unknown arrival time. The research objectives are to:

- **Completeness in Space-Time:** Investigate the theoretical and practical consequences of lifting kinodynamic planners into space-time, in particular with respect to probabilistic completeness.
- **Parameter Sensitivity:** Test and evaluate the influence of the planners central parameters on their performance.
- **Benchmark:** Compare three kinodynamic planners against each other in a benchmark of dynamic environments.

1.3 Contribution

To address the research objectives, this thesis makes the following contributions:

- **Dynamic Environments:** Design dynamic environments for testing and evaluating kinodynamic motion planners.
- **ST-Db-RRT:** Extending the kinodynamic planner Db-RRT to plan in space-time enabling it to deal with dynamic environments.
- **ST-GBRRT:** Extending the kinodynamic planner GBRRT to plan in space-time enabling it to deal with dynamic environments.
- **Novel Algorithm:** Develop KIST, an alternative kinodynamic RRT algorithm for planning in space-time.
- **Theoretical Analysis:** Analyse the consequences of the space-time extension on the theoretical properties of the planners, in particular on probabilistic completeness.
- **Benchmark:** Conduct a benchmark evaluating the influence of the central parameters of the presented planners and comparing them against each other in dynamic environments.

2. Related Work

This thesis lies at the intersection of motion planning in dynamic environments and kinodynamic motion planning. In the following we review both fields as well as the existing works at their intersection, and then position our work among them.

2.1 Planning in Dynamic Environments

Planning in dynamic environments is often approached reactively, without assuming prior knowledge of the obstacle trajectories: whenever the perceived environment changes and invalidates a previously computed path, a new one is generated. Execution-extended RRT [9] reuses waypoints of previous solutions to speed up replanning, Real-Time RRT* [10] interleaves online tree rewiring with path execution, and RRTX [11] maintains an asymptotically optimal search tree that is repaired online. When the obstacle trajectories are known in advance, as in this work, the motion of the obstacles can instead be anticipated. The classical path-velocity decomposition [12] does so in two stages, planning a geometric path first and a velocity profile along it that avoids the moving obstacles. Committing to a fixed path early, however, can render every velocity profile infeasible. A more general approach is to treat time as an explicit planning dimension, an idea introduced with state-time space [7]. Time-Based RRT [13] augments the configuration space with time to plan a rendezvous of two dynamic systems, but assumes that the arrival time of the goal configuration is known beforehand. SIPP [14] compresses the time dimension into safe intervals per configuration and computes optimal trajectories, but requires a discretization of the entire state space, restricting it to low-dimensional problems. Sampling-based planners avoid this state space discretization: T-PRM [15] augments the probabilistic roadmap with a time dimension and queries it with a time-aware variant of A*, while SI-RRT [16] transfers the safe-interval representation into an RRT-Connect search for high-DoF manipulators among moving obstacles, an idea also applied to multi-robot planning [17]. Both, however, connect configurations by linear interpolation and therefore remain restricted to holonomic systems. ST-RRT* [4] lifts RRT* into space-time without assuming prior knowledge of the arrival time, planning asymptotically optimal trajectories in dynamic environments. However, its extension step interpolates linearly between states, restricting it to holonomic systems with velocity limits, and kinodynamic constraints such as acceleration bounds cannot be handled. This thesis adopts the space-time search strategy of ST-RRT* and removes exactly this limitation, lifting it into the kinodynamic setting.

2.2 Kinodynamic Motion Planning

Kinodynamic motion planning accounts for differential constraints and actuation limits of the robot. There are mainly three categories of approaches:

Search-based methods discretize the control space with kinodynamically feasible motion primitives [18] to construct a graph of reachable states, which can then be searched with classical graph search algorithms like A*. This approach provides strong theoretical guarantees relative to the chosen discretization, but the size of the graph grows exponentially with the state dimension making exploration of high-dimensional state spaces infeasible. Sampling-based planners explore the state space more rapidly and RRT [19] established the idea of growing a tree iteratively towards randomly sampled states by propagating controls with the system dynamics, holding completeness using correct propagation [20]. RRT methods can achieve asymptotic optimality by planning in state-cost space (AO-RRT) [21], using witness sets (SST*) [22] or rewiring the tree (RRT*) [23], [11], [4]. Rewiring, however, requires connecting two states exactly which in the kinodynamic case can be described by a two-point boundary value problem (BVP) that can be solved in closed form only for restricted system classes such as linear dynamics [24], and is computationally expensive or unsolvable for general kinodynamic systems. Other works focus on improving convergence through goal biasing or bidirectional search [25]. Heuristics can be used to improve sampling with informed sampling regions [26] or AIT* [27], which computes a cost-to-go heuristic through an improving reverse search tree. GBRRT [8] makes this feasible for the general kinodynamic setting by avoiding the requirement of solving any BVP, by using a heuristic-guided bidirectional search with forward propagation.

Optimization-based methods such as CHOMP [28] or sequential convex optimization [29] optimize trajectories while incorporate complex constraints well. Their performance is highly dependent on an initial guess, otherwise they can suffer from the non-convexity of planning problems and only converge locally. T-CHOMP [30] extends the CHOMP [28] trajectory optimization framework to space-time, enabling it to handle time-dependent and complex constraints, but it has the same limitations and requires a prior fixed trajectory duration.

More recently, hybrid methods combine both worlds: db-A* [2] and [1] connect precomputed motion primitives while tolerating a bounded discontinuity at the junctions, which is subsequently repaired by nonlinear trajectory optimization to an optimal solution. iDb-RRT [3] embeds this discontinuity-bounded expansion into the RRT framework, removing the need of solving a BVP and finds initial solutions up to an order of magnitude faster than previous methods. We build upon these hybrid methods in this thesis and explain them more in detail in Section 3.5.4

These planners provide efficient and partly optimal algorithms for kinodynamic problems, but they are all formulated for static environments. Time is not an explicit planning dimension, and moving obstacles cannot be accounted for.

2.3 Kinodynamic Planning in Dynamic Environments

A smaller body of work addresses both challenges simultaneously. The randomized kinodynamic planner of Hsu et al. [31] is the earliest sampling-based approach in this setting: it builds a tree of milestones in state-time space by forward propagation of randomly sampled controls, avoiding moving obstacles with known trajectories. While general with

respect to the system dynamics, the planner expands only at random within a bounded time horizon, without goal-directed guidance and without any notion of solution quality. Chen et al. [32] extend the Space Exploration Guided Heuristic Search to the time domain, guiding a kinodynamic search with a time-dependent heuristic. The approach targets car-like robots and relies on a circle-based decomposition of the workspace. More recently, safe intervals have been combined with kinodynamic models: Ali and Yakovlev [33] extend SIPP with acceleration constraints while retaining completeness and optimality, but inherit its dependence on a discretized state space, and Huang et al. [34] couple a safe-interval graph search with gradient-based trajectory optimization to plan quadrotor trajectories in dynamic environments. These works demonstrate planning in dynamic environments for kinodynamic systems, but each of them is either restricted to a specific class of systems or depends on a discretization of the state space.

In summary, the reviewed planners for dynamic environments handle moving obstacles but assume holonomic systems, kinodynamic planners handle differential constraints but assume static environments, and the few works at their intersection are tied to specific system classes or state-space discretizations. We address this integrated problem with sampling-based planners that combine kinodynamic planning in dynamic environments with unknown arrival time. This thesis builds upon three algorithms that mark the state of the art of the respective subproblems: ST-RRT* [4] contributes the space-time search for unbounded time, iDb-RRT [3] contributes fast kinodynamic exploration and GBRRT [8] contributes an optimization-free approach. All three algorithms are described in detail in Chapter 3, and their combination yields the three planners presented in Chapter 4. As no established baseline exists for the integrated problem, we evaluate the three planners against each other in Chapter 5 on kinodynamic systems in dynamic environments, and demonstrate by comparing Db-RRT with its space-time extension ST-Db-RRT (Section 5.6.1) that a kinodynamic planner without an explicit time dimension fails precisely on the problems that motivate this work.

3. Background

The planners developed in Chapter 4 are obtained by combining ST-RRT* [4] with the kinodynamic planners Db-RRT [3] and GBRRT [8]. Following that construction requires familiarity with each of these components, which this chapter provides. It first defines the motion planning problem and introduces the sampling-based planners that all three build upon, before presenting ST-RRT* as the geometric foundation of the space-time search. It then turns to kinodynamic planning, where the differential constraints of the system make the direct connection of two states expensive or unavailable, and presents the two strategies that address this obstacle without a steering function: discontinuity-bounded search and BVP-free bidirectional search. These are the components that Chapter 4 lifts into space-time.

3.1 Motion Planning Definition

Motion planning is concerned with finding a continuous, collision-free path from a start state x_{start} to a goal state x_{goal} in a *state space* $\mathcal{X}$ [6]. The state space is divided into the obstacle space $\mathcal{X}_{\text{obs}}$, containing all states that are in collision with the environment and the free space $\mathcal{X}_{\text{free}} = \mathcal{X} \setminus \mathcal{X}_{\text{obs}}$. The problem can be formulated as the search problem to find a continuous path $x : [0, 1] \rightarrow \mathcal{X}_{\text{free}}$ satisfying $x(0) = x_{\text{start}}$ and $x(1) = x_{\text{goal}}$:

$$\text{find } x \tag{3.1}$$

$$\text{s.t. } x \in \mathcal{X}_{\text{free}} \tag{3.2}$$

$$x(0) = x_{\text{start}} \tag{3.3}$$

$$x(1) = x_{\text{goal}} \tag{3.4}$$

Motion planners for holonomic systems, like ST-RRT*, assume that every dimension of x can be controlled independently which allows planners to freely interpolate between states, decoupling path search from the robot's dynamics. In reality, the motions of a system like a robotic arm or a car are constrained by its kinematics and control limits. Geometric planners ignore these constraints, while kinodynamic planners explicitly consider them during planning, which makes the problem more complex but also more realistic.

There are two key properties that are commonly used to evaluate motion planners: *completeness* and *optimality*. A planner is *complete* if it guarantees finding a solution when one exists and correctly reports failure otherwise. The stronger property of *optimality* requires that the returned solution minimizes a user-specified cost, such as path length or execution time.

3.2 Search-Based Motion Planning

Search-based motion planners discretize the continuous state space $\mathcal{X}$ into a finite graph to make a search algorithm tractable. Formally, the continuous space is approximated by a graph $G = (V, E)$ with a fixed resolution r , where $V \subset \mathcal{X}_{\text{free}}$ is a finite set of discrete states and $E \subseteq V \times V$ is the set of collision-free transitions between neighboring states. The planning problem (3.1) then reduces to finding a sequence of nodes $(v_0, v_1, \dots, v_k)$ in V such that $(v_i, v_{i+1}) \in E$ for all i , with $v_0 = x_{\text{start}}$ and $v_k \in \mathcal{X}_{\text{goal}}$. Because the solution quality depends on the resolution of the discretization, these planners achieve *resolution completeness*: a solution is guaranteed to be found if one exists at the chosen resolution.

The most widely used search algorithm for motion planning is A* [35], which assigns each node $n \in V$ a priority

$$f(n) = g(n) + h(n), \quad (3.5)$$

where $g(n)$ is the cost of the best known path from x_{start} to n (cost-to-come), and $h(n)$ is a heuristic estimate of the remaining cost from n to x_{goal} (cost-to-go), and nodes are expanded in order of increasing f . When the heuristic is *admissible*, it never overestimates the true cost-to-go, ensuring that A* is guaranteed to return an optimal solution. The quality of the heuristic is crucial for A*'s efficiency: a more informed heuristic reduces the number of nodes expanded, while an uninformative one degrades toward exhaustive search. The most common heuristic is the euclidean distance to the goal, which is admissible but can be very uninformed in complex environments with many obstacles.

The main limitation of search-based planners is scalability: the number of nodes grows exponentially with the dimensionality of the state space, making the exhaustive search impractical for high-dimensional problems.

3.3 Sampling-Based Motion Planning

Sampling-based motion planners emerged as a scalable alternative to search-based methods, avoiding explicit discretization by instead randomly sampling the continuous state space and connecting them to build a collision-free graph. Because the state space is explored stochastically, they can satisfy a relaxed notion of completeness: *probabilistic completeness*. It requires only that the probability of finding a solution converges to 1 as the number of samples approaches infinity.

Two popular strategies exist for how the graph is constructed and searched for a solution path. *Probabilistic Roadmap (PRM)* algorithms first construct a graph by covering the state space with random samples and connecting them, to then conduct a graph search to find a solution path. This approach relies heavily on the ability to connect two states directly using a local planner. While for holonomic systems a simple straight-line local planner is sufficient, connecting two states directly for more constrained non-holonomic systems can be challenging and requires more expressive local planners, making PRM less suitable for such systems. *Rapidly-exploring Random Tree (RRT)* [19] algorithms can avoid connecting states directly by incrementally building tree graphs rooted at x_{start} and/or x_{goal} , which only requires extending the tree toward random samples, without needing to connect them directly.

Algorithm 1: RRT / RRT*

Input: $x_{\text{start}}, \mathcal{X}, \mathcal{X}_{\text{goal}}, K$
Output: path σ or FAILURE

```

1  $T \leftarrow \{x_{\text{start}}\}$ 
2 for  $k = 1$  to  $K$  do
3    $x_{\text{rand}} \leftarrow \text{SAMPLE}(\mathcal{X})$ 
4    $x_{\text{near}} \leftarrow \text{NEAREST}(T, x_{\text{rand}})$ 
5    $x_{\text{new}} \leftarrow \text{EXTEND}(x_{\text{near}}, x_{\text{rand}})$ 
6   if  $\text{COLLISIONFREE}(x_{\text{near}}, x_{\text{new}})$  then
7      $\mathcal{N} \leftarrow \text{NEAR}(T, x_{\text{new}}, r_k)$ 
8      $x_{\text{near}} \leftarrow \arg \min_{x \in \mathcal{N}, \text{CF}(x, x_{\text{new}})} c(x) + d(x, x_{\text{new}})$ 
9      $T \leftarrow T \cup \{x_{\text{new}}\}$ 
10     $\text{REWIRE}(T, x_{\text{new}}, \mathcal{N})$ 
11    if  $x_{\text{new}} \in \mathcal{X}_{\text{goal}}$  then return  $\text{EXTRACTPATH}(T, x_{\text{new}})$ 
12 return FAILURE

```

3.3.1 RRT

The fundamental RRT [19] algorithm, described in Alg. 1, grows a tree T rooted at x_{start} by iteratively sampling a random state $x_{\text{rand}} \in \mathcal{X}$ (line 3) and extending the tree from the nearest node toward it (lines 4-5). The original RRT was improved and extended in various ways by changing: the sampling strategy `SAMPLE`, the way the tree is extended with `NEAREST` and `EXTEND`. The design of these functions can significantly affect the performance and capabilities of the planner. The following paragraphs discuss each in turn, as they form the basis of the planners proposed in this thesis.

Sampling

The sampling strategy is a crucial component of RRT-based algorithms, as it determines how the state space is explored. The most basic sampling strategy to cover a continuous state space densely is to sample uniformly at random, which ensures that the planner is probabilistically complete, as the probability of sampling any particular point of the state space converges to 1 as the number of samples increases. In practice, randomness always carries *irregularity*, causing some areas of the state space to be sampled more densely than others, making random sampling inefficient for covering the state space evenly.

To overcome this issue, various deterministic sampling strategies have been proposed, such as low-dispersion sequences that show better convergence properties than random sampling [36]. Note, that deterministic sampling can only guarantee resolution completeness, as it relies on a fixed set of samples that may not cover the state space densely enough to find a solution.

A very popular sampling strategy is *goal biasing*, which samples the goal state x_{goal} with a certain probability, which can significantly speed up the convergence of the planner by guiding it toward the goal.

Nearest Neighbor

The nearest neighbor function NEAREST is responsible for selecting the node in the tree that should be extended towards the random sample (in most cases, the nearest neighbor). This computation relies on a distance metric d defined over the state space. For holonomic systems, the choice of the euclidean distance

$$d(x_1, x_2) = \|x_1 - x_2\|$$

is trivial, as it reflects the actual cost of moving between states.

With a growing tree size the runtime of the nearest neighbor search can become a bottleneck. In practice, efficient data structures such as k-d trees [37] are usually used to speed up queries.

Extend

The EXTEND step is responsible for generating a new state x_{new} by extending from x_{near} toward x_{rand} or directly connecting them. For holonomic systems, two states can simply be connected by interpolating between them using a steering function. How this step must be adapted for kinodynamic systems is discussed in Section 3.5.1.

3.3.2 RRT* and Asymptotic Optimality

While RRT is very efficient at finding an initial solution, it provides no optimality guarantee. Similar to *probabilistic completeness*, we can only guarantee a relaxed notion of optimality: *asymptotic optimality*. It requires that the cost of the returned path converges to the optimum as the number of samples approaches infinity. RRT* [23] achieves this by extending RRT with two modifications: The first modification is in the parent selection: instead of connecting x_{new} to the single nearest neighbor, a neighborhood $\mathcal{N}$ of nearby nodes is considered and the node that minimizes the accumulated path cost to x_{new} is selected as its parent. The second modification is the rewiring step: after adding x_{new} to the tree, the algorithm checks if any of the nodes in $\mathcal{N}$ can be reached with a lower cost by going through x_{new} , and if so, it rewires the tree accordingly.

3.3.3 Bidirectional Search

The idea of bidirectional search improved the efficiency of RRT-based planners by growing two trees simultaneously towards each other, one rooted at x_{start} and the other at x_{goal} . While the original RRT paper [19] already proposed a bidirectional search the RRT-Connect algorithm [25] introduced growing the two trees towards each other until they connect. Connecting the trees requires a steering function that can connect two states directly. For holonomic systems, this can be achieved with simple linear interpolation between the states.

3.4 Space-Time Motion Planning

3.4.1 Dynamic Environments

Standard motion planning assumes a static environment, where the state of obstacles does not change over time. However, many real-world scenarios involve dynamic environments with moving obstacles, such as pedestrians, vehicles, or other robots. In that case, a path that is collision-free at the time of planning may become infeasible during execution due to the movement of obstacles. To address this, motion planning can be formulated in a *space-time* state space $\mathcal{X} = \mathcal{Q} \times \mathcal{T}$, where $\mathcal{Q}$ is the robot's state space and $\mathcal{T}$ represents time.

Under this formulation, a configuration may be collision-free at one time t_a and in collision at another time t_b . Time-based RRT [13] first uses the space-time state space to plan motions in a unidirectional RRT search, while assuming a known arrival time for the goal. ST-RRT* [4] builds on this by introducing a more efficient bidirectional search strategy and a progressive goal sampling technique that allows it to find optimal paths with respect to arrival time, without assuming a known arrival time upfront.

In this work we consider the case where the trajectories of dynamic obstacles are known and deterministic, which allows us to predict their future positions and plan accordingly. For planning in unknown dynamic environments, the planner would need to be reactive or incorporate uncertainty and probabilistic predictions of obstacle trajectories, which is outside the scope of this work.

3.4.2 Space-Time RRT*

All planners that are presented in Chapter 4 use the bidirectional space-time search strategy of ST-RRT*, so we present its fundamental approach here and a more detailed description about the implementation can be found in Section 4.2.1.

ST-RRT* [4] is a sampling-based motion planner that extends RRT* to the space-time domain, enabling it to handle dynamic environments and unknown arrival times. The planner is probabilistically complete, asymptotically optimal with respect to arrival time, and can handle holonomic systems with velocity constraints. It achieves that by maintaining a forward tree rooted at x_{start} and sampling multiple reverse trees rooted at x_{goal} for different arrival times. The trees are grown by sampling in space-time and expanding towards each other, until they connect.

Progressive Goal Region Expansion

Progressive Goal Region Expansion is the strategy by which ST-RRT* addresses the unknown and unbounded arrival time: the optimal arrival time may lie anywhere along the time axis. ST-RRT* solves this, by sampling new goal states with progressively later times. The lower time bound for sampling these times is defined as the time it would take to reach the goal at maximum velocity along a straight line, which is the most optimistic scenario. The upper time bound is progressively increased during the planning process by a user-defined factor, which allows the planner to explore longer arrival times if a solution is not found within the initial optimistic bounds. Every new goal sampled this way, is added as a new root for a reverse tree and grown towards the forward tree offering a new solution with a different arrival time.

In order to prevent having a high sample density at earlier times when the upper bound is increased, the bounds and sampled goals are divided in a old and new batches, where the new time batch is sampled with a higher probability than the old one. This way a uniform sampling density is achieved over the entire time range.

After a first solution is found, the solutions goal time can be used to prune later goals and to set a upper bound for sampling new goals to focus on improving the solution with earlier arrival times.

Distance Metric

In space-time planning, the distance metric needs to consider the time dimension. Nodes that are backward in time from the current state are not reachable, and nodes that are too far in the future may also be unreachable due to velocity constraints. In ST-RRT* the pseudometric

$$d(x_1, x_2) = \begin{cases} \lambda d_Q(q_1, q_2) + (1 - \lambda)(t_2 - t_1), & \text{if } t_1 < t_2 \text{ and } v^i \leq v_{\max}^i \forall i \in [1, ||Q||] \\ \infty & \text{otherwise} \end{cases} \quad (3.6)$$

is used, where the time difference is added to the intrinsic state distance d_Q , while ensuring that the time consistency and velocity constraints $v_{\max}^i$ are met. The importance of both distances can be tuned with the weight parameter λ . The distance metric drives the nearest neighbor and action selection. We conduct experiments to research the influence of λ on the motion planner performance (Section 5.5.4).

Conditional Sampling

Conditional sampling is a strategy used by ST-RRT* to improve the sample efficiency in the time dimension, by preventing the sampling of states that cannot be part of a solution due to velocity constraints. First, they sample a random state q_{rand} without the time component. The lower time bound is defined as the time it would take to reach q_{rand} from the start at maximum velocity, so its the earliest time that q_{rand} can be reached from the start. The upper time bound is defined as the latest departure time from q_{rand} to reach the latest goal at maximum velocity. These time bounds define a feasible time window for q_{rand} to be in and the time is sampled uniformly within this window.

Rewiring

ST-RRT* achieves asymptotic optimality with respect to arrival time by rewiring nodes in the reverse tree. When a new node x_{new} is added to the reverse tree, nearby nodes are checked to see if they can improve their cost-to-go by going through x_{new} , and if so, they are rewired accordingly. In practice, this means that the rewired node is inserted to a reverse tree with a lower arrival time.

Limitation

ST-RRT* is designed for holonomic systems with velocity constraints and relies on straight-line interpolation between states in the Extend step, which is only valid for such systems. For systems with acceleration constraints or more complex dynamics, this approach is not feasible as it would generate trajectories that violate the system's constraints.

In this work, we extend the ST-RRT* algorithm to handle kinodynamic constraints, which requires significant modifications to the RRT components. Challenges that arise in kinodynamic planning will be introduced in the next section.

3.5 Kinodynamic Motion Planning

So far we only discussed planning with holonomic systems, where every dimension of the robot state can be controlled independently. Most real systems are constrained by their dynamics: a car cannot move sideways and is limited by its turning radius, and a torque-limited manipulator cannot achieve arbitrary joint acceleration.

In this section, we briefly describe the fundamentals of kinodynamic planning and present some existing solutions, while focusing on bidirectional search and its challenges.

3.5.1 Kinodynamic Constraints

Non-holonomic planning refers to motion planning problems, in which the motions of the system are kinematically constrained by its dynamics, expressed as a system of differential equations $\dot{x} = f(x, u)$, where u is the control input.

Kinodynamic motion planning deals with dynamics that not only impose kinematic constraints but also at least second-order constraints. This is the case, for example, when the state q includes velocities and the control u includes accelerations or forces. That means that the system cannot change velocity instantaneously, but is constrained by its acceleration limits. The Unicycle 2nd order model in Section 5.1 is an example of this, which includes velocities in the state space and imposes a limited acceleration control space.

In practice, continuous dynamics $\dot{x} = f(x, u)$ are made tractable through Euler integration with a small time step Δt , assuming a constant control input between steps:

$$x_{k+1} \approx \text{step}(x_k, u_k) = x_k + \dot{x}_k \Delta t. \quad (3.7)$$

This reduces the continuous trajectory to a finite sequence of states $X = [x_0, x_1, \dots, x_K]$ driven by a corresponding control sequence $U = [u_0, u_1, \dots, u_{K-1}]$. The approximation is accurate when Δt is chosen small enough relative to the system's dynamics.

Extending the basic formulation (3.1), the kinodynamic motion planning problem requires finding a discrete state sequence X and control sequence U of length K that minimizes a objective function J and satisfies the system dynamics:

$$\min_{X, U, K} J(X, U, K) \quad (3.8a)$$

$$\text{s.t. } x_{k+1} = \text{step}(x_k, u_k) \quad \forall k \in \{0, \dots, K-1\} \quad (3.8b)$$

$$u_k \in \mathcal{U} \quad \forall k \in \{0, \dots, K-1\} \quad (3.8c)$$

$$x_k \in \mathcal{X}_{\text{free}} \quad \forall k \in \{0, \dots, K\} \quad (3.8d)$$

$$x_0 = x_{\text{start}}, \quad x_K \in \mathcal{X}_{\text{goal}} \quad (3.8e)$$

The dynamics constraint (3.8b) prevents free interpolation between states: each transition is determined by applying a control input to the system, not chosen freely. The control constraint (3.8c) further restricts which motions the system can execute, for example by bounding acceleration or force. Together these two constraints make connecting arbitrary state pairs fundamentally harder than in the holonomic case, a challenge discussed in detail in Section 3.5.3. We focus on finding time-optimal trajectories and use the objective function $J(X, U, K) = K\Delta t$.

3.5.2 Tree Expansion

A popular approach for forward tree expansion was already introduced in the original RRT paper [19]. Rather than interpolating between states, the tree is expanded using *forward propagation*: the system's dynamics $f(x_k, u_k)$ from (3.7) are used to simulate the system forward in time from x_{near} with a control input u . Multiple candidate motions can be generated by sampling different control inputs and time durations, and the one landing closest to x_{rand} is added to the tree.

By discretizing the action space into motion primitives, expansion can be accelerated with a precomputed library M of feasible trajectories. Kinodynamically feasible primitives can be generated by forward propagation or constrained optimization. However, the discretization weakens the theoretical guarantees: only solutions achievable with the given set of actions are considered, a property called *resolution completeness*.

In practice, both methods struggle to connect two states directly, which is a problem that many kinodynamic planners face.

3.5.3 The Two-Point Boundary Value Problem

Although kinodynamic RRT works only by expanding *towards* a sample, connecting two states directly is desirable. Connecting two trees in the bidirectional search setting and rewire operations used in the optimal RRT [23] require direct connection of two states. In the kinodynamic setting, finding a feasible motion requires computing a control sequence $U = [u_0, \dots, u_{K-1}]$ that drives the system exactly from a state x_a to another state x_b while satisfying (3.8b) and (3.8c). This problem is called two-point boundary value problem (BVP). Closed-form solutions, often referred to as *steering function*, exist only for specific system classes such as linear systems [24] or Dubins vehicles [6]. For general kinodynamic systems, numerical solvers such as shooting methods are required, which are computationally expensive, numerically unstable, or sensitive to an initial guess [8].

3.5.4 Hybrid and Optimization-based Planning

Hybrid planning, introduced with db-A* [2], retains the speed of motion primitives while avoiding both a fixed action-space discretisation and repeated BVP solves.

Rather than connecting states exactly or propagating uninformative random controls, they use motion primitives and connect them while tolerating a bounded discontinuity δ at the junctions. The discontinuity costs are estimated by a lower-bound function $\underline{t}(q_1, q_2)$ that uses the system's control limits to bound the travel time between two configurations. They define a trajectory as a δ -discontinuity-bounded solution if $d(x_{k+1}, \text{step}(x_k, u_k)) \leq \delta$ at every junction (and $d(x_0, x_s), d(x_K, x_g) \leq \delta$ at start and goal) with all states collision-free. The resulting trajectory is not dynamically feasible, but it serves as a warm start for non-linear trajectory optimization, which repairs the discontinuities into a feasible trajectory. Allowing $\delta > 0$ makes the search with a discretised action space tractable, because a relatively small primitive set is sufficient as the states within δ are treated as equivalent.

The number of motion primitives and the size of the discontinuity bound δ are embedded in an iterative outer loop that grows the primitive set and shrinks δ through iterations, rendering it complete probabilistic and asymptotically optimal [1].

iDb-RRT

iDb-RRT transfers the same primitive-and-discontinuity mechanism to the RRT framework. Its search step, Db-RRT, replaces the RRT steering function with the application of a δ -bounded primitive toward a sampled state, retaining the Voronoi bias for rapid exploration. Expansion can either be *focused* by selecting the nearby primitive whose endpoint is closest to the target, or *random* by randomly choosing a collision-free primitive. It admits forward and bidirectional variants, the latter growing a reverse tree from the goal using time-reversed primitives. Again, the outer loop increases the primitive count when search fails and decreases δ when optimization fails. iDb-RRT is probabilistically complete under mild assumptions, but unlike the Db-A* it targets fast initial solutions rather than optimality. We describe how to build on the Db-RRT approach to plan in space-time in Section 4.3.

3.5.5 Bidirectional Planning Without BVP

Another approach, proposed in [8], avoids both the BVP and trajectory optimization: a bidirectional strategy that sidesteps both. Rather than connecting the two trees directly or building a discontinuity-bounded solution, the forward tree stays kinodynamically feasible throughout while the reverse tree's cost-to-go serves only as a guiding heuristic for the forward search. The algorithm proceeds in two phases. During *exploration*, both trees expand independently: the forward tree grows from x_{start} using forward propagation, while the reverse tree grows from x_{goal} without requiring kinodynamically valid backward integration. Once the trees are in close proximity, the *exploitation* phase begins: the forward tree uses the reverse tree's cost-to-go as a heuristic to guide its expansion toward the goal, effectively tracing the reverse tree without ever forming a direct edge between them. A fully feasible solution is obtained once the forward tree reaches a node inside the goal region.

We derive two new planners (see Section 4.4 and Section 4.5) from this approach. They use the reverse tree only as a guiding heuristic to solve the bidirectional challenge while adopting the ST-RRT* space-time approach.

4. Kinodynamic Space-Time Planning

To solve kinodynamic motion planning problems in dynamic environments, this thesis develops an approach to combine the bidirectional space-time search strategy of ST-RRT* [4] explained in section 3.4.2, with the kinodynamic planners Db-RRT and GBRRT. The result are three new space-time motion planner which will be presented in this chapter: the hybrid optimization-based ST-Db-RRT planner and the two optimization-free planners KIST and ST-GBRRT. All three planners are benchmarked and evaluated in Chapter 5. While all three planners share the same space-time search strategy, they differ in how much work they invest per iteration and in when kinodynamic feasibility is established. The algorithms have an increasing effort in their expansion step: ST-Db-RRT expands both trees fast with randomly selected discontinuity-bounded motion primitives and defers feasibility entirely to a single trajectory optimization at the end, KIST grows a kinodynamically feasible forward tree with best-input action rollouts, but exploits the reverse tree only in a dedicated completion phase once the trees connect and ST-GBRRT also maintains a feasible forward tree but additionally updates and follows a reverse-tree heuristic in every iteration. The benchmark in Chapter 5 evaluates how this trade-off between per-iteration effort and sampling throughput plays out under a fixed planning budget.

After introducing the notation and problem formulation, we present the shared space-time search strategy and then each planner in detail. Table 4.1 summarizes the three new planners together with the two Db-RRT baselines evaluated in Chapter 5, contrasting their theoretical properties and key algorithmic features before each planner is presented in detail.

Planner	ST	TO	Completeness		AO	Key features
			static	dynamic		
Db-RRT [3]	×	✓	RC	–	×	RAND db-expansion, bidirectional δ -connect
Naive Db-RRT	×	✓	RC	×	×	time-annotated nodes, time-dependent collision checks
ST-Db-RRT	✓	✓	RC	RC*	×	RAND db-expansion, BEST δ -connect in space-time
KIST	✓	×	PC	PC	×	PROP/BEST expansion, completion policies after tree connection
ST-GBRRT	✓	×	PC	PC	×	PROP/BEST expansion, continuous reverse-tree heuristic

Table 4.1: Overview of the planners presented in this chapter. The expansion and connection modes (RAND: random primitive; BEST: best primitive, PROP: forward propagation of the best primitive) are introduced in Sec. 4.2.2, PROP/BEST denotes forward/reverse tree. ST: plans in space-time; TO: uses trajectory optimization or is optimization-free; RC: resolution complete with respect to the provided motion primitives; RC*: resolution complete with respect to the provided motion primitives, but not complete in unbounded time; PC: probabilistically complete; AO: asymptotically optimal. Db-RRT does not consider moving obstacles (–). Naive Db-RRT loses completeness in dynamic environments through duplicate pruning (Remark 1, Sec. 4.3.1). ST-Db-RRT is only resolution complete if using randomly propagated (not time-optimal) motion primitives. KIST and ST-GBRRT retain probabilistic completeness through random action rollouts. None of the planners is asymptotically optimal, as all target fast initial solutions.

4.1 Notation and Problem Formulation

We adapt the state space of [4], the space-time $\mathcal{X} = \mathcal{Q} \times \mathcal{T}$, which pairs the configuration state space $\mathcal{Q}$ with the time axis $\mathcal{T}$. A space-time state $x = (q, t) \in \mathcal{X}$ therefore consists of a configuration state $q \in \mathcal{Q}$ and a time $t \in \mathcal{T}$. The system dynamics act on the configuration component only: applying a control $u \in \mathcal{U}$ for one timestep advances it by $q_{k+1} = \text{step}(q_k, u_k)$ following the Euler integration of Eq. 3.7, while the time component advances deterministically by the fixed step Δt . The resulting space-time transition is

$$x_{k+1} = (q_{k+1}, t_{k+1}) = (\text{step}(q_k, u_k), t_k + \Delta t). \quad (4.1)$$

Let $\mathcal{X}_{\text{free}} \subseteq \mathcal{X}$ be the collision-free states and x_{start} and x_{goal} the start and goal state and $\mathcal{X}_{\text{goal}}$ the goal region. We set the start time $t_{\text{start}} = 0$ and the goal time t_{goal} is sampled by the *Progressive Goal Region Expansion* explained in Sec. 3.4.2. The lower-bound time it takes to cover a certain distance between two configurations is notated with $\underline{t}(q_1, q_2)$.

We notate a single motion with $m = (X, U)$ consisting of a kinodynamically feasible sequence of states $X = [x_0, x_1, \dots, x_f]$ and actions $U = [u_0, u_1, \dots, u_{f-1}]$. In the algorithms, we grow trees T rooted in x_{start} with motions as edges and the corresponding landing state x_f as nodes. We use the notation $x_f = x \oplus m$ to indicate that a δ -discontinuity-bounded motion m is applied from a state x , landing in a final state x_f .

The output is a solution path $\mathcal{S} = (X, U)$ connecting x_{start} to the goal region $\mathcal{X}_{\text{goal}}$ with a state sequence $X = [x_{\text{start}}, x_1, \dots, x_{\text{goal}}]$, $x_{\text{goal}} \in \mathcal{X}_{\text{goal}}$, of length K and a control sequence $U = [u_0, u_1, \dots, u_{K-1}]$. All states are collision-free, $x \in \mathcal{X}_{\text{free}} \forall x_k \in X$. For KIST and ST-GBRRT the solution is kinodynamically feasible, $x_{k+1} = \text{step}(x_k, u_k) \forall x_k \in X$; for ST-Db-RRT it is only *discontinuity-bounded*. Since the cost is solely the trajectory duration, the cost of a feasible path is $c = K \cdot \Delta t$. For the discontinuity-bounded solution, node times follow Eq. 4.3, which additionally accounts for the lower-bound time at each junction.

4.2 Shared Components

4.2.1 Space-Time Search Strategy

All three planners presented in this chapter share the same *space-time search strategy*, which they adopt from ST-RRT* [4] and which is what enables them to plan in dynamic environments. Its fundamentals were introduced in Sec. 3.4.2 and here we fix the terminology used throughout this chapter and clarify the implementation details that the planners have in common. In every planner pseudocode the lines belonging to this shared strategy are printed in yellow and are identical across the planners, whereas the black lines show the individual kinodynamic expansion and connection of each planner. For a detailed pseudocode of the shared functions we refer to Algorithms 2-5 in the ST-RRT* paper [4].

In practice, we implemented three components from ST-RRT*: *Progressive Goal Region Expansion* (Sec. 3.4.2), *Conditional Sampling* (Sec. 3.4.2), and *Space-Time Pseudometric* (Eq. 3.6). The first two components are configured by two user-defined parameters: the *initial batch size* and the *range factor* which are unified in the struct P and used to initialize the time bound struct B . In every iteration, the $\text{UPDATEGOALREGION}(B, P)$ function checks if the current sampling batch is exhausted and if so, expands the goal region by updating the time bounds in B by the value of the *range factor*. Thus, the initial batch size controls how many samples are drawn in the time bounds before the goal region is expanded, and the range factor controls how much the goal region is expanded. The influence of the *initial batch size* and the *range factor* is studied empirically in Chapter 5.

The time bounds in B are used for the time sampling in $\text{SAMPLEGOAL}(x_g, B)$, where a new goal state is sampled with an arrival time drawn either from the newly expanded or the previous batch time bounds.

All planners are implemented in a anytime fashion: they keep track of the best solution found so far and return it when the time limit is reached. In the space-time search strategy this means, that when the planners find a first solution, the progressive goal region expansion is stopped. Instead the upper time bound is set to the arrival time of the best current solution, and the lower time bound is set to the lower-bound time from start to goal. This way only samples are drawn that could improve the current best solution.

ST-RRT* uses one more user-specified parameter *sampleRatio*, that we fixed to 0.1,

to control the ratio of goal samples to random samples. The conditional sampling is implemented in `SAMPLECONDITIONALLY`($\mathcal{X}, B$), which first samples a random configuration q_{rand} and then computes the lower-bound time $\underline{t}(q_{\text{rand}}, q_{\text{start}})$ to the start state, conditioning the sampled time on the reachability from start. The upper time bound is defined by the last valid time the system could still reach the latest goal arrival time of all currently existing goals.

The third shared component that also drives the expansion and connection decisions of the three planners is the space-time pseudometric of ST-RRT* (Eq. 3.6). It weights the intrinsic configuration distance d_Q against the temporal difference through the weight $\lambda \in (0, 1)$. Its case distinction enforces *time-consistency*: a state may only be reached from an earlier state, and only if the velocity required to cover the configuration gap within the available time respects the limits v_{max}^i ; states violating either condition are infinitely far apart and thus never selected. This metric is asymmetric and therefore incompatible with the symmetric k-d trees used for fast nearest-neighbour queries. We resolve this the same way in every planner: a small neighbourhood is queried with d_Q and the first candidate satisfying the time-consistency requirement (Eq. 4.2) is returned (Alg. 2). The weight λ influences the planner in multiple ways. It governs every decision that trades spatial against temporal proximity: the time-consistent nearest-neighbour selection that picks the parent to expand from, the ranking of candidate motion primitives during expansion, and deciding if the trees are considered to be connected. As its not clear how much importance should be attributed to the time dimension we treat λ as a design parameter and study its influence empirically in Section 5.5.4.

4.2.2 Expansion and Connection Functions

Beyond the sampling strategy, the three planners also assemble their trees from the same small set of building blocks, which we present once in this section. In practice these build on top of the *Dynoplan* [3] library and reuses its system dynamics, collision checking, lazy expansion and trajectory optimization. The planners differ only in how a candidate motion primitive is chosen and applied during expansion, which we capture in the *selection mode* variable:

$$\sigma \in \{\text{RAND}, \text{BEST}, \text{PROP}\}.$$

With $\sigma = \text{RAND}$, the first valid, randomly drawn primitive is applied as a δ -discontinuity-bounded jump $x \oplus m$. With $\sigma = \text{BEST}$, the primitives are ordered by the distance of their landing state to the expansion target before the first valid one is applied, again as a discontinuity-bounded jump. With $\sigma = \text{PROP}$, the primitives are ordered as like in BEST , but the actions $m.U$ of a candidate are forward propagated from the parent state with the system dynamics (`ACTIONROLLOUT`, Alg. 7), resulting in a kinodynamically feasible motion without discontinuity. Table 4.2 summarizes with which mode each planner instantiates the shared functions for its forward tree, reverse tree and connection attempts.

Planner	Forward expansion	Reverse expansion	Connection
ST-Db-RRT	RAND	RAND	BEST
KIST	PROP + random rollout	BEST	PROP / BEST
ST-GBRRT	PROP + random rollout	BEST	–

Table 4.2: Selection modes σ with which each planner instantiates the shared EXPAND (Alg. 4) and CONNECT (Alg. 6) functions. KIST connects with $\sigma = \text{PROP}$ from the forward tree and $\sigma = \text{BEST}$ from the reverse tree, ST-GBRRT never uses an explicit connection step. KIST and ST-GBRRT preserve probabilistic completeness by expanding the forward tree with a random action rollout (Alg. 8) with some probability.

Time-Consistency

All nearest-neighbor queries use the same time-consistency function, which ensures both conditions of the space-time pseudometric (Eq. 3.6): a tree node $x_n \in T$ is time-consistent with a query state x if the configuration gap between them can be traversed within their time difference,

$$\text{TIMECONSISTENT}(x_n, x, T) \iff \begin{cases} \underline{t}(q_n, q) \leq t - t_n & \text{if } T = T_{\text{fwd}}, \\ \underline{t}(q, q_n) \leq t_n - t & \text{if } T = T_{\text{rev}}. \end{cases} \quad (4.2)$$

Since $\underline{t} \geq 0$, this condition enforces the correct temporal ordering: a forward-tree node must lie earlier in time than the query state, and a reverse-tree node later. The time-consistent nearest-neighbour function (Alg. 2) queries a small neighbourhood in the k-d tree and returns the first candidate satisfying Eq. 4.2.

Motion Selection

SELECTMOTION (Alg. 3) implements all three selection modes in one function: the applicable primitives $\mathcal{L}$ are shuffled ($\sigma = \text{RAND}$) or sorted by the distance of their landing state to the target ($\sigma \in \{\text{BEST}, \text{PROP}\}$), and then applied one after another until the first valid new state is found. An expansion is valid if every transformed state of the motion is collision-free at its time, while a rollout (Alg. 7) checks validity at every integration step.

Expansion

The shared EXPAND function (Alg. 4) grows a tree T towards a sampled state x_{rand} . It queries the $k = \log(|T| + 1)$ nearest neighbours, skips the time-inconsistent ones (Eq. 4.2) and calls SELECTMOTION for the actual expansion. The first successful new state is added to the tree and returned. If the neighbourhood is exhausted, the expansion fails. Every new node keeps track of its time: a rollout advances it with the space-time transition (Eq. 4.1), while a discontinuity-bounded jump integrates the motion duration and the lower-bound time of discontinuity jump (Eq. 4.3). A more detailed explanation will follow in Sec. 4.3.1.

Algorithm 2: NEAREST - time-consistent nearest neighbor

Input: T, x
Output: x_n

```

1  $\mathcal{X}_n \leftarrow \text{NEARESTK}(T, x)$ 
2 foreach  $x_n \in \mathcal{X}_n$  do
3   if  $\text{TIMECONSISTENT}(x_n, x, T)$  then return  $x_n$ 
4 return NULL

```

Algorithm 3: SELECTMOTION - Applies the motions of $\mathcal{L}$ in the order given by σ and returns the first valid landing state

Input: $\mathcal{L}, x_{\text{near}}, x_{\text{target}}, \sigma$
Output: x_{new}

```

1 if  $\sigma = \text{RAND}$  then  $\mathcal{L}.\text{SHUFFLE}()$ 
2 else  $\mathcal{L}.\text{SORT}(d(x_{\text{near}} \oplus m, x_{\text{target}}))$ 
3 foreach  $m \in \mathcal{L}$  do
4   if  $\sigma = \text{PROP}$  then  $x_{\text{new}} \leftarrow \text{ACTIONROLLOUT}(x_{\text{near}}, m.U)$ 
5   else  $x_{\text{new}} \leftarrow x_{\text{near}} \oplus m$ 
6   if  $\text{VALID}(x_{\text{new}})$  then return  $x_{\text{new}}$ 
7 return NULL

```

Random Rollout

Selecting motion primitives from a discrete set restricts the planner to a fixed resolution and therefore precludes probabilistic completeness. The optimization-free planners KIST and ST-GBRRT therefore replace the best-input expansion of their forward tree by a random action rollout (RANDOMROLLOUT, Alg. 8) with a user-defined probability and fall back to it whenever the best-input expansion fails. It randomly draws a control $u \in \mathcal{U}$ and propagates it from the node for a random duration $t \in [0, t_{\text{max}}]$. This restores a positive probability of expanding into any reachable region (see the theoretical analyses in Sections 4.4 and 4.5).

Connection

The ISCONNECTED function (Alg. 5) checks whether a node x in one tree forms a connection to the *other* tree. It queries the other tree T_b within the discontinuity bound δ and returns the first time-consistent pair. Requiring Eq. 4.2 at the connection also guarantees that the gap between the two nodes can be traversed under ideal conditions. CONNECT (Alg. 6) attempts to greedily establish such a connection, by repeatedly expanding towards the nearest node of the other tree using SELECTMOTION, as long as the distance to the target decreases, checking every intermediate node with ISCONNECTED.

4.3 Space-Time Db-RRT

The idea of **Space-Time Discontinuity-Bounded RRT** is to plan with Db-RRT [3] (see Sec. 3.5.4) in space-time by using the bidirectional search approach of ST-RRT* [4] (see

Algorithm 4: EXPAND - Time-consistent neighborhood expansion with selection mode σ

Input: $T, x_{\text{rand}}, \mathcal{M}, \delta, \sigma$
Output: x_{new}

```

1  $k \leftarrow \log(|T| + 1)$ 
2  $N_k \leftarrow \text{NEARESTK}(T, x_{\text{rand}}, k)$ 
3 foreach  $x_{\text{near}} \in N_k$  do
4   if  $\neg \text{TIMECONSISTENT}(x_{\text{near}}, x_{\text{rand}}, T)$  then continue
5    $\mathcal{L} \leftarrow \text{APPLICABLEMOTIONS}(x_{\text{near}}, \mathcal{M}, \delta)$ 
6    $x_{\text{new}} \leftarrow \text{SELECTMOTION}(\mathcal{L}, x_{\text{near}}, x_{\text{rand}}, \sigma)$  // see Alg. 3
7   if  $\neg \text{VALID}(x_{\text{new}})$  then continue
8    $T \leftarrow T \cup x_{\text{new}}$ 
9   return  $x_{\text{new}}$ 
10 return NULL

```

Algorithm 5: ISCONNECTED - Checks if a node connects to the other tree

Input: T_b, x, δ
Output: (x, x_n) // tree connection nodes

```

1  $\mathcal{X}_n \leftarrow \text{NEASTR}(T_b, x, \delta)$ 
2 foreach  $x_n \in \mathcal{X}_n$  do
3   if  $\text{TIMECONSISTENT}(x_n, x, T_b)$  then return  $(x, x_n)$ 
4 return NULL

```

Sec. 3.4.2). Ortiz-Haro et al. note that the discontinuity-bounded expansion strategy can be integrated into other RRT variants and ST-Db-RRT realises this for the space-time search of ST-RRT*.

4.3.1 Naive Db-RRT for Dynamic Environments

A first naive approach to elevate Db-RRT to deal with dynamic environments is to keep the state space and only add dynamic collision checks to avoid moving obstacles. Since we know the duration of every motion primitive we can keep track of each node's time. This can be achieved by using the time as cost which makes the cost-to-come represent the time of each node. The time of the discontinuities between the motion primitives are accounted for using the lower time bound function $\underline{t}(q_1, q_2)$ [1]. A new nodes time t' can be calculated with

$$t' = t + m.t + \underline{t}(q, m.q_s) \quad (4.3)$$

where t is the time of its predecessor, $m.t$ the duration of the motion primitive and $\underline{t}(q, m.q_s)$ the lower-bound time estimate the systems needs to get from q to the start state of the motion $m.q_s$.

Knowing each node time enables time-dependent collision checks and allows the planner to find discontinuity-bounded solutions in environments with moving obstacles. In order to repair the discontinuity-bounded solution we also need to include dynamic collision checking into the trajectory optimization which is described further in Sec. 14. This Db-RRT variant with naive dynamic collision checks performs well in most dynamic environments (see Sec. 5.6.1) as it saves computational cost introduced by adding space-time.

Algorithm 6: CONNECT - Greedily expands from x towards the nearest node of the other tree

Input: $T_a, T_b, x, \mathcal{M}, \delta, \sigma$
Output: (x_a, x_b)

```

1  $x_{\text{target}} \leftarrow \text{NEAREST}(T_b, x)$ 
2 if  $x_{\text{target}} = \text{NULL}$  then return NULL
3  $d_{\text{prev}} \leftarrow \infty$ 
4 while  $d(x, x_{\text{target}}) < d_{\text{prev}}$  do
5    $d_{\text{prev}} \leftarrow d(x, x_{\text{target}})$ 
6    $\mathcal{L} \leftarrow \text{APPLICABLEMOTIONS}(x, \mathcal{M}, \delta)$ 
7    $x \leftarrow \text{SELECTMOTION}(\mathcal{L}, x, x_{\text{target}}, \sigma)$  // see Alg. 3
8   if  $\neg \text{VALID}(x)$  then return NULL
9    $T_a \leftarrow T_a \cup x$ 
10   $(x_a, x_b) \leftarrow \text{ISCONNECTED}(T_b, x, \delta)$  // see Alg. 5
11  if  $(x_a, x_b) \neq \text{NULL}$  then return  $(x_a, x_b)$ 
12 return NULL

```

Algorithm 7: ACTIONROLLOUT - Propagates Actions of a Motion Primitive from a State

Input: start state x , array of actions U
Output: final state x

```

1 foreach  $u_k \in U$  do
2    $x_{\text{next}} = \text{step}(x, u_k)$ 
3   if  $\neg \text{VALID}(x_{\text{next}})$  then return NULL
4    $x = x_{\text{next}}$ 
5 return  $x$ 

```

However, not including time in the state space comes at the cost of probabilistic completeness in dynamic environments.

Remark 1: Db-RRT with naive collision checks is not complete in dynamic environments.

Proof: Duplicate pruning ([3] Alg. 2, Line 14) of Db-RRT rejects a new node when a node would be placed in a δ -neighborhood i.e. it suppresses nodes too close in state space. Consider a configuration q that must be occupied at time t_1 to pass through a door within a short temporal window. If one branch reaches close to q before t_1 , duplication pruning can suppress a later branch that would reach q at t_1 and thus will never find a solution. Without planning in space-time, the requirement 'be at q , but at a later time' cannot be represented. In space-time, (q, t_0) and (q, t_1) are different states and duplicate pruning can distinguish between them, as the time is included in the distance metric (Eq. 3.6).

4.3.2 ST-Db-RRT

To enable planning in space-time with Db-RRT, we integrate the shared space-time search strategy (Sec. 4.2.1), while keeping the concept of a discontinuity-bounded solution that is repaired with trajectory optimization. The resulting algorithm ST-Db-RRT is presented in Alg. 9.

In contrast to iDb-RRT, we drop the iterative outer loop that grows the primitive set

Algorithm 8: RANDOMROLLOUT - Random duration and control rollout

Input: $x, \mathcal{U}, t_{\max}$
Output: x_{new}

```

1  $t \leftarrow \text{RANDOM}(0, t_{\max})$ 
2  $u \leftarrow \text{RANDOM}(\mathcal{U})$ 
3  $U \leftarrow [u]_0^t$  // constant control for duration  $t$ 
4 return ACTIONROLLOUT( $x, U$ ) // see Alg. 7
```

and shrinks the discontinuity bound δ across iterations. The outer loop relies on the inner search being complete and reporting failure when no solution exists, before the next iteration is started with more primitives and a smaller δ . In space-time this failure report never arrives, since the progressive goal region expansion keeps extending the search into the unbounded time dimension (see theoretical analysis in Sec. 14). Because of this the number of motion primitives and the discontinuity bound become user-defined parameters again, whose influence we study empirically in Sec. 5.5.2.

Algorithm Description**Algorithm 9:** ST-Db-RRT

Input: $x_{\text{start}}, x_{\text{goal}}, \mathcal{X}, \mathcal{X}_{\text{goal}}, \mathcal{M}, \delta, P$

```

1  $T_a \leftarrow \{x_{\text{start}}\}; T_b \leftarrow \emptyset$ 
2  $B \leftarrow \text{INITBATCHVARIABLES}(P)$ 
3 while  $\neg \text{TERMINATECONDITION}()$  do
4    $B \leftarrow \text{UPDATEGOALREGION}(B, P)$ 
5    $\text{SAMPLEGOAL}(x_g, B)$ 
6    $x_{\text{rand}} \leftarrow \text{SAMPLECONDITIONALLY}(\mathcal{X}, B)$ 
7    $x_{\text{new}} \leftarrow \text{EXPAND}(T_a, x_{\text{rand}}, \mathcal{M}, \delta, \text{RAND})$  // see Alg. 4
8   if  $\neg \text{VALID}(x_{\text{new}})$  then continue
9    $(x_a, x_b) \leftarrow \text{ISCONNECTED}(T_b, x_{\text{new}}, \delta)$  // see Alg. 5
10  if  $(x_a, x_b) \neq \text{NULL}$  then return EXTRACTSOLUTION( $x_a, x_b$ )
11   $(x_a, x_b) \leftarrow \text{CONNECT}(T_a, T_b, x_{\text{new}}, \mathcal{M}, \delta, \text{BEST})$  // see Alg. 6
12  if  $(x_a, x_b) \neq \text{NULL}$  then return EXTRACTSOLUTION( $x_a, x_b$ )
13   $\text{SWAP}(T_a, T_b)$ 
14 return FAILURE
```

The planner requires a start and goal state, the space-time state space $\mathcal{X}$, a library of motion primitives $\mathcal{M}$ (including reversed motions for reverse tree expansion), a discontinuity bound δ and the batch bound parameters P .

The start of the algorithm in line 1-6 is identical to ST-RRT*. The time bounds are updated, and if necessary, the goal region is expanded and a new goal root with later arrival time is added to the reverse tree. A random state x_{rand} in space-time is sampled that we want to expand towards to. The expansion in line 7 uses the shared EXPAND function (Alg. 4) with the selection mode $\sigma = \text{RAND}$, matching the fast randomized expansion of Db-RRT, but if it fails the iteration is skipped (line 8).

Lines 9-12 mirror the connection attempt of ST-RRT* in the discontinuity-bounded way: `ISCONNECTED` (Alg. 5) first checks whether the new node already connects to the other tree, and otherwise `CONNECT` (Alg. 6) greedily expands towards it with the focused mode $\sigma = \text{BEST}$.

When a connection is found, the discontinuity-bounded solution is extracted and used to warm start the trajectory optimization.

Theoretical Analysis

ST-Db-RRT does not suffer from the duplicate pruning problem that Db-RRT has in dynamic environments, as the distance function (Eq. 3.6) takes time into account. Planning in space-time, however, has other theoretical implications on probabilistic completeness, which we will show now.

The proof of probabilistic completeness for iDb-RRT and the family of Db-Algorithms ([2], [1], [3]) is derived from Kleinbort et al. [20] proof using an overlapping chain of balls on a solution. A core requirement for the proof is to show that there is a positive probability of reaching the next ball in the chain, which they provide for random forward propagation in Lemma 3. Instead of random forward propagation, the Db-Algorithms discretize the action space with motion primitives and thus have to show that they can still successfully expand to

the next ball. Their proof is based on iteratively adding more motion primitives and decreasing the discontinuity bound. This mechanism guarantees that at some point all required motions are considered to assign a positive probability to reach the successor ball of a solution. That argument builds on the assumption that generating and adding more motion primitives will cover the state space asymptotically (see Def. 4-5 and Remark 3 in [1]). Their motion primitive generation covers the state space by randomly sampling start and goal states and using trajectory optimization, resulting in time-optimal motions. This is the point where the proof breaks in space-time. If we want to cover space-time, we also need time-suboptimal motions. Fig. 4.1 illustrates the space-time of a one-dimensional system and the dashed triangle shows the bound of reachable states at the system's maximum velocity. Time-optimal motions (red) lie only on this boundary, whereas covering the full space-time also requires motions with later arrival times (a few shown in green). The completeness argument therefore needs the primitive-generating method to place positive measure on time-suboptimal motions of varied duration. Random forward propagation with randomised duration satisfies this requirement.

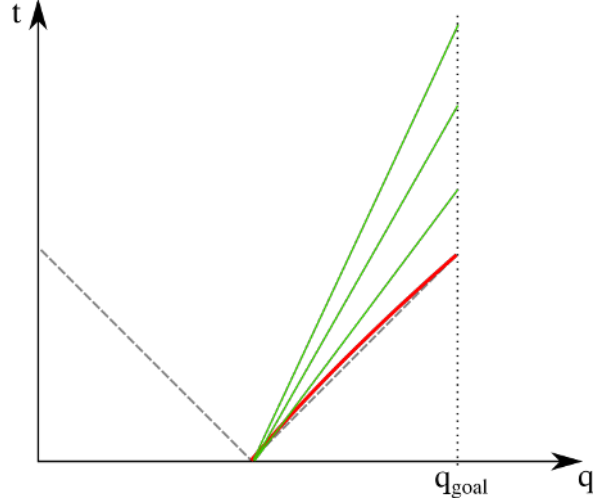


Figure 4.1: Space-time showing a time-optimal motion in red ■ and a few time-suboptimal motions in green ■ that would be needed for covering space-time.

However, even with a primitive-generating method that covers space-time asymptotically, ST-Db-RRT is still not complete because the unbounded time prevents it from reporting that no solution exists. While Db-RRT is resolution complete with respect to its provided

motion primitives, it is able to terminate and go into the next iteration of the iDb-RRT algorithm with more motion primitives. ST-Db-RRT on the other hand, will never terminate, as it will keep searching for solutions in the unbounded time dimension. This could be mitigated if an upper-bound of the time is known or estimated.

Trajectory Optimization

Repairing the discontinuity-bounded solution in dynamic environments requires the trajectory optimization to account for the time of each state. We adopt the Differential Dynamic Programming (DDP) repair of Db-RRT ([3], Chapter 4 E), chosen for its fast convergence and for operating on a trajectory of fixed length and fixed Δt . This fixed time grid makes the extension to dynamic environments straightforward as neither the number of stages nor Δt changes during optimization, the time at each state stays constant and known, so the obstacle configuration at that time can be determined directly. DDP solves trajectory optimization problems of the form

$$\min_{Q,U} \sum_{k=0}^{K-1} c(q_k, u_k) + c_K(q_K) \quad (4.4a)$$

$$s.t. \quad q_{k+1} = \text{step}(q_k, u_k) \quad \forall \in \{0, \dots, K-1\} \quad (4.4b)$$

$$x_0 = q_{\text{start}} \quad (4.4c)$$

and enforce constraints such as collision avoidance through a cost term evaluated at every iteration. We keep time separate from the state: it is not a decision variable but is passed to the cost function (Eq. 4.4a) solely to evaluate the collision constraint. In static environments, this collision cost was computed from a static signed distance function. In dynamic environments, the known time of each state is used to update every obstacle to its configuration at that time before the signed distance is computed.

4.4 KIST

We present **KIST**, a **KInodynamic Space-Time** planning algorithm for dynamic environments. As for ST-Db-RRT, we integrate the shared space-time search strategy (Sec. 4.2.1) on a high level, while on the lower level KIST takes inspiration from the bidirectional, BVP-free approach of GBRT [8] (Sec. 3.5.5) and uses motion primitives and action rollouts to enable fast kinodynamic planning. This bidirectional approach avoids solving any BVP to connect the trees directly by considering the tree connected when they are in δ proximity, just like Db-RRT. However, instead of expanding with discontinuities, KIST avoids optimization by growing a feasible forward tree with action rollouts. The reverse tree grows like in Db-RRT, with δ -discontinuity-bounded motion primitives, but after a connection with the forward tree, the algorithm uses the path of the reverse tree solely as a guiding heuristic to expand the forward tree greedily to the goal. We implemented two *Completion Policies* that differ in how the forward tree uses the reverse tree as heuristic to get to the goal. KIST expends more computation per expansion by evaluating all applicable motion primitives, whereas ST-Db-RRT relies primarily on fast random exploration.

The Figure 4.2 illustrates how the reverse and forward trees connect when within the discontinuity bound δ and how the reverse tree is used as a guiding heuristic

to complete the solution. The forward tree (light green) is kinodynamically feasible, while the reverse tree (red) is only discontinuity-bounded. When the two trees connect, the reverse tree path (dashed red) is used to guide the forward tree to the goal (dark green).

4.4.1 Algorithm Description

Algorithm 10: KIST

```

Input:  $x_{\text{start}}, x_{\text{goal}}, \mathcal{X}, \mathcal{X}_{\text{goal}}, \mathcal{M}, \delta, P, \mathcal{U}$ 
1  $T_a \leftarrow \{x_{\text{start}}\}; T_b \leftarrow \emptyset; T_{\text{rev}}^{\text{sol}} \leftarrow \emptyset$ 
2  $B \leftarrow \text{INITBATCHVARIABLES}(P)$ 
3 while  $\neg \text{TERMINATECONDITION}()$  do
4    $B \leftarrow \text{UPDATEGOALREGION}(B, P)$ 
5    $\text{SAMPLEGOAL}(x_g, B)$ 
6    $x_{\text{rand}} \leftarrow \text{SAMPLECONDITIONALLY}(\mathcal{X}, B)$ 
7    $\sigma \leftarrow \text{PROP}$  if  $T_a = T_{\text{fwd}}$  else  $\text{BEST}$ 
8   if  $\sigma = \text{BEST}$  then
9      $x_{\text{new}} \leftarrow \text{EXPAND}(T_a, x_{\text{rand}}, \mathcal{M}, \delta, \text{BEST})$  // see Alg. 4
10  else
11    if  $\text{RANDOM}(0, 1) \geq p_{\text{rand}}$  then  $x_{\text{new}} \leftarrow \text{EXPAND}(T_a, x_{\text{rand}}, \mathcal{M}, \delta, \text{PROP})$ 
12    if  $\neg \text{VALID}(x_{\text{new}})$  then
13       $x_{\text{new}} \leftarrow \text{RANDOMROLLOUT}(\text{NEAREST}(T_a, x_{\text{rand}}), \mathcal{U}, t_{\text{max}})$  if
14         $\text{VALID}(x_{\text{new}})$  then  $T_a \leftarrow T_a \cup x_{\text{new}}$ 
15  if  $\neg \text{VALID}(x_{\text{new}})$  then continue
16   $(x_a, x_b) \leftarrow \text{ISCONNECTED}(T_b, x_{\text{new}}, \delta)$  // see Alg. 5
17  if  $(x_a, x_b) = \text{NULL}$  then  $(x_a, x_b) \leftarrow \text{CONNECT}(T_a, T_b, x_{\text{new}}, \mathcal{M}, \delta, \sigma)$ 
18  if  $(x_a, x_b) \neq \text{NULL}$  then
19     $x_{\text{sol}} \leftarrow \text{COMPLETIONPOLICY}(x_a, x_b, T_{\text{fwd}}, T_{\text{rev}}, \mathcal{M})$ 
20    if  $x_{\text{sol}} \neq \text{NULL}$  then
21      return  $\text{EXTRACTSOLUTION}(x_{\text{sol}})$ 
22   $\text{SWAP}(T_a, T_b)$ 
23 return FAILURE

```

We describe KIST on a high-level in Alg. 10, where the colored text marks the shared space-time search strategy (Sec. 4.2.1). The planner requires a start and goal state, the space-time state space $\mathcal{X}$, a library of motion primitives $\mathcal{M}$ (including reversed motions for reverse tree expansion), a discontinuity bound δ , the batch bound parameters P and the control space $\mathcal{U}$ for random rollouts. In addition, the random-rollout probability p_{rand} and the maximum rollout duration t_{max} are user-defined parameters.

Before the main loop, the tree structures $T_{\text{fwd}}, T_{\text{rev}}$ for the forward and reverse tree are initialized. Additionally we initialize a tree structure $T_{\text{rev}}^{\text{sol}}$, which will only be used by the *Heuristic Completion* variant to populate a potential reverse tree solution as guiding heuristic in the final phase. Finally, the time bound sampling batch structure is initialized with the users parameter (line 2).

Then the algorithm enters a space-time sampling loop that resembles ST-RRT* [4] and

repeats until a connection of the forward and reverse tree is established. Each iteration begins with the shared space-time sampling phase (lines 4-6): the goal region and time bounds are updated, a goal state is sampled, and a random space-time state x_{rand} is drawn by conditional time sampling. The active tree T_a is then expanded towards x_{rand} with the shared EXPAND function (Alg. 4), where the selection mode σ depends on the active tree (line 7).

The forward tree expands with $\sigma = \text{PROP}$ and therefore remains completely kinodynamically feasible. To preserve probabilistic completeness, the forward tree is expanded with a random action rollout (RANDOMROLLOUT, Alg. 8) with probability p_{rand} , or whenever the best-input expansion fails (lines 12-14).

The reverse tree expands with $\sigma = \text{BEST}$ and thus only yields a δ -discontinuity-bounded tree towards the goal (line 9), which we accept since its role is solely to provide a guiding heuristic once the trees connect. Its node times start from the sampled goal arrival time and decrease along the path, describing the time-to-go.

If the expansion fails, the sampling phase continues (line 15). Otherwise, we check whether the new node x_{new} already connects the trees (ISCONNECTED, line 16) and, if not, greedily expand from x_{new} towards the other tree with CONNECT,

using the same selection mode σ as the expansion (line 17). As discussed in Sec. 4.2.2, such a connection avoids solving a BVP by allowing a δ -discontinuity at the junction, while the time-consistency function (Eq. 4.2) ensures that the forward and reverse path can be joined under ideal conditions. If no connection was found, the sampling phase continues with swapped trees (line 23). If a connection was found, the connecting nodes x_a and x_b (line 19) are passed to the completion policy to find a solution.

We followed different approaches to find a solution after the trees were connected. Two completion policies are considered. The first adapts the guided search of GBRRT [8] of using the reverse tree as guiding heuristic for a greedy A*-like search and the second replays the motion primitives of the reverse solution path from the forward connection node. If the completion policy fails to find a solution, the planners go back to sampling, but if a solution is found, the solution path is extracted and returned. Both variants are described in the following two sections.

4.4.2 Completion Policy: Heuristic Completion

The *Heuristic Completion* variant, described in Alg. 11, treats the reverse tree solution as a heuristic that guides an A*-like best-first search of the feasible forward tree towards

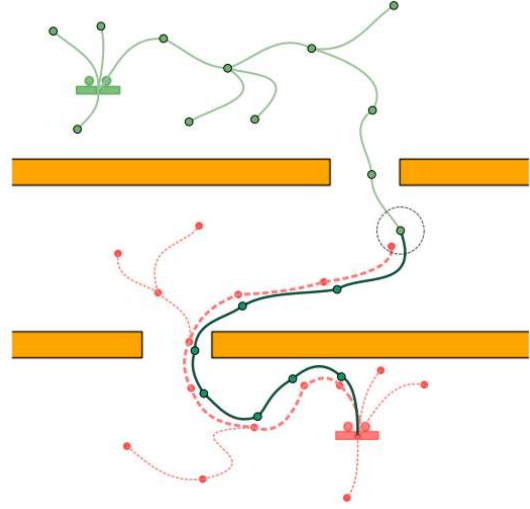


Figure 4.2: Illustration of KIST's bidirectional search with feasible forward tree (light green), discontinuity-bounded reverse tree (red), discontinuity-bounded solution (dashed red) and the completion policy solution (dark green).

the goal. It adapts the idea of [8], but operates in space-time and grows the search with feasible action rollouts instead of discontinuity-bounded motions.

Reverse Solution as Heuristic

Once the trees connect, we follow the parent pointers from the reverse connecting node x_{rev} up to the goal and store the resulting (discontinuity-bounded) path in $T_{\text{rev}}^{\text{sol}}$ (line 2-5) and only these nodes are kept to guide the forward search. Every reverse node x_{rev} carries its cost-to-come $g(x_{\text{rev}})$, i.e. the accumulated motion duration from the goal to x_{rev} . Because the reverse tree is rooted at the goal, this *cost-to-come* is the time still needed to travel from x_{rev} to the goal. From the perspective of the forward search it is therefore a *time-to-go*, which is the quantity a heuristic needs to estimate.

The Heuristic h

For every candidate forward node x the search estimates how much travel time remains to the goal. We compute this estimate in two steps. First, we find the node on the reverse solution path that lies closest to x ,

$$x_{\text{rev}} = \text{NEAREST}(T_{\text{rev}}^{\text{sol}}, x).$$

Second, we add the lower-bound time $\underline{t}(x, x_{\text{rev}})$ required to bridge the spatial gap between x and x_{rev} to its stored time-to-go $g(x_{\text{rev}})$:

$$h(x) = \underline{t}(x, x_{\text{rev}}) + g(x_{\text{rev}}). \quad (4.5)$$

Intuitively, $h(x)$ approximates the remaining time-to-go by estimating the time difference between forward node to the reverse solution path and then following that path to the goal: the first term pays for reaching the path, the second for traveling along it. Since $\underline{t}$ is a lower bound on the true traversal time, h does not over-estimate the cost of this detour, which keeps the forward search concentrated in a corridor around the reverse solution.

Best-first Expansion

The search maintains a priority queue Q , initialized with the forward connecting node x_{fwd} and ordered by $f(x) = g(x) + h(x)$, where $g(x)$ is the forward time-to-come already tracked during forward expansion via the space-time transition (Eq. 4.1). In each iteration we pop the node with the smallest f -score, roll out its applicable motions into new feasible states, evaluate each of them with the heuristic (Eq. 4.5), and push them back onto the queue. The search returns as soon as a popped node reaches the goal region, and fails once the queue is empty or its size limit Q_{max} is exceeded. To keep the branching factor low, we do not expand every applicable motion but only greedily roll out the k best ones, i.e. those whose landing state scores lowest under h . This concentrates the available compute on the motions that follow the reverse solution most closely.

4.4.3 Completion Policy: Replay

The *Replay* variant, described in Alg. 12, avoids any search and instead replays the reverse tree solution directly. Like *Heuristic Completion* it starts from the forward connecting node x_{fwd} , but instead of expanding a guided search towards the goal, it rolls out the

Algorithm 11: HEURISTIC COMPLETION — Best-first search from connection point to goal, guided by the reverse solution

Input: $x_{\text{fwd}}, x_{\text{rev}}, T_{\text{fwd}}, \mathcal{M}, \delta, k$

```

1  $Q \leftarrow \{x_{\text{fwd}}\}$  // priority queue ordered by  $f = g + h$ 
2  $x \leftarrow x_{\text{rev}}$ 
3 while  $x \neq \text{NULL}$  do
4    $T_{\text{rev}}^{\text{sol}} \leftarrow T_{\text{rev}}^{\text{sol}} \cup \{x\}$ 
5    $x \leftarrow x.\text{PARENT}$ 
6 while  $Q \neq \emptyset \wedge |Q| < Q_{\text{max}}$  do
7    $x_{\text{best}} \leftarrow Q.\text{POP}()$  // lowest  $f$ -score node
8   if  $d(x_{\text{best}}, x_{\text{goal}}) < \epsilon_{\text{goal}}$  then
9     return  $x_{\text{best}}$ 
10   $\mathcal{L} \leftarrow \text{APPLICABLEMOTIONS}(x_{\text{best}}, \mathcal{M}, \delta)$ 
11   $\mathcal{L}.\text{SORT}(h(x_{\text{best}} \oplus m))$  // rank by heuristic of landing state
12  foreach  $m \in \mathcal{L}[1:k]$  do
13     $x_{\text{new}} \leftarrow x_{\text{best}} \oplus m$ 
14    if  $x_{\text{new}} \notin \mathcal{X}_{\text{free}}$  then continue
15     $x_{\text{rev}} \leftarrow \text{NEAREST}(T_{\text{rev}}^{\text{sol}}, x_{\text{new}})$ 
16     $x_{\text{new}}.h \leftarrow \underline{t}(x_{\text{new}}, x_{\text{rev}}) + g(x_{\text{rev}})$  // Eq. 4.5
17     $x_{\text{new}}.f \leftarrow x_{\text{new}}.g + x_{\text{new}}.h$  //  $g$ : forward time-to-come
18     $Q.\text{PUSH}(x_{\text{new}})$ 
19 return  $\text{NULL}$ 

```

motions of the reverse solution path one after another.

Starting from the reverse connecting node x_{rev} , it follows the parent pointers up to the goal and collects the corresponding motions in M (line 1-5). Beginning at x_{fwd} , we then apply the action sequence of each motion with a feasible ACTIONROLLOUT (Alg. 7), performing collision and validity checks at every step and appending the resulting states to the forward tree. If any rollout leaves the free space the completion fails; otherwise the trajectory is accepted once its final state reaches the goal region.

Compared to *Heuristic Completion*, this saves computational cost by avoiding to find, evaluate, sort and roll out multiple candidate motions per node. However, since the reverse solution path is only δ -discontinuity-bounded, replaying its motions from x_{fwd} with the exact system dynamics does not reproduce it precisely: the per-junction discontinuity accumulates along the path, so the rolled-out trajectory drifts from the reverse solution. For a large δ or a long path this drift can grow significant, causing the trajectory to miss the goal region or to collide with an obstacle, in which case the completion fails and KIST returns to sampling. Because of this drift, the Replay policy solved only a small fraction of instances in preliminary trials and the evaluation in Chapter 5 therefore reports only the Heuristic Completion policy.

Algorithm 12: REPLAY — Rollout δ -discontinuity-bounded reverse tree solution

Input: $x_{\text{fwd}}, x_{\text{rev}}, T_{\text{fwd}}, \mathcal{M}, \delta$

```

1  $M \leftarrow \{\}$ 
2  $x \leftarrow x_{\text{rev}}$ 
3 while  $x \neq \text{NULL}$  do
4    $M \leftarrow M \cup x.m$ 
5    $x \leftarrow x.\text{PARENT}$ 
6  $x \leftarrow x_{\text{fwd}}$ 
7 foreach  $m \in M$  do
8    $x_{\text{new}} \leftarrow \text{ACTIONROLLOUT}(x, m.U)$ 
9   if  $\neg \text{VALID}(x_{\text{new}})$  then return NULL
10   $T_{\text{fwd}} \leftarrow T_{\text{fwd}} \cup x_{\text{new}}$ 
11   $x \leftarrow x_{\text{new}}$ 
12 if  $d(x, x_{\text{goal}}) > \epsilon_{\text{goal}}$  then
13   return NULL
14 return  $x$ 

```

4.4.4 Theoretical Analysis

KIST resembles the structure of ST-RRT* on a high-level, but uses a mixture of motion primitive and action rollouts for expansion. We use ST-RRT*'s proof sketch and the proof of Kleinbort et al. [20] to show the probabilistic completeness for KIST. The proof is based on induction along a sequence of balls of a reference solution, which includes showing that there is a positive probability of 1) selecting the current i th ball and 2) expanding to the next $(i + 1)$ th ball. While ST-RRT* relies on a steering function to connect two states, we need to show that there is a positive probability of expanding into the $i + 1$ -ball in space-time. In the expand step (Alg. 4) we primarily use a best-input approach ($\sigma \in \{\text{BEST}, \text{PROP}\}$, SELECTMOTION, Alg. 3) for efficient expansion, which alone would mean the loss of completeness, as the set of primitives is limited. We restore probabilistic completeness by expanding the forward tree with a random action rollout (Alg. 10, lines 12-14, and Alg. 8) that propagates a random action from $\mathcal{U}$ for a random duration $t \in [0, t_{\text{max}}]$.

Finally, in the case of unbounded goal time, we adapt the argument of ST-RRT*, that through progressive goal region expansion and conditional sampling we have a positive probability of eventually sampling a feasible goal arrival time and also any node that is part of a feasible solution.

4.5 ST-GBRRT

ST-GBRRT is a direct transfer of GBRRT [8] (Sec. 3.5.5) to dynamic environments. GBRRT introduced the idea of guiding a feasible forward tree towards the goal with a cost-to-goal heuristic taken from a reverse tree, so that the two trees never have to be connected and no boundary value problem needs to be solved. As for the other two planners in this chapter, we obtain the space-time variant through two modifications: we integrate the shared space-time search strategy (Sec. 4.2.1), and we grow the reverse tree with

fast δ -discontinuity-bounded motion primitives instead of GBRRT’s reverse propagation. The forward tree keeps GBRRT’s feasible action rollouts, so that ST-GBRRT and KIST end up sharing the same building blocks: a feasible, heuristic-guided forward tree and a δ -discontinuity-bounded, heuristic-providing reverse tree. The two planners differ only in how the heuristic is used. KIST exploits the reverse tree in a dedicated completion phase once the trees connect, whereas ST-GBRRT follows GBRRT’s original scheme and guides the forward tree towards the goal throughout the whole search, interleaving exploration of the state space with exploitation of the reverse-tree heuristic. Because the forward tree is kinodynamically feasible and grows directly into the goal region, ST-GBRRT returns a solution without any completion policy or trajectory optimization.

4.5.1 Algorithm Description

We describe ST-GBRRT in Alg. 13, where the colored text again marks the shared space-time sampling of ST-RRT*. Before the main loop, the forward tree is rooted in the start and the reverse tree in the goal, and a priority queue Q that orders forward-tree nodes by their heuristic h is initialized empty (lines 1-3). The queue is ordered greedily by the time-to-go h to reach an initial feasible solution as fast as possible.

Each iteration begins with the shared space-time sampling phase (lines 5-7): the goal region and time bounds are updated, a goal state is sampled if needed, and a random space-time state x_{rand} is drawn by conditional time sampling. We then compute the heuristic radius r (line 8), which shrinks as the reverse tree grows and bounds the neighborhood within which the two trees influence each other. Following GBRRT, we compute the heuristic radius with the parameter γ , d the dimension of $\mathcal{X}$ and r_{max} a user-defined upper bound. First the reverse tree is expanded towards x_{rand} with the shared EXPAND function ($\sigma = \text{BEST}$, line 9). The reverse node’s time-to-come $g(x_{\text{rev}})$ is used as heuristic for the forward tree nodes. The cost-to-go $h(x)$ for every forward node within the heuristic radius r of the new reverse node, is updated with the new estimate (Eq. 4.5) if it improves its current one (lines 10-13).

After the reverse expansion ST-GBRRT either exploits the reverse tree’s heuristic or explores randomly, depending on the probability p_{exp} . In the exploit case, if the trees have encountered each other within the heuristic radius r before, the lowest time-to-go forward node x_{pop} is popped from the queue and expanded towards its most promising reverse-tree neighbor x_{best} , i.e. the neighbor minimizing the estimated remaining time-to-goal (line 18). The expansion rolls out the best applicable primitive towards x_{best} (SELECTMOTION, $\sigma = \text{PROP}$), growing the forward tree towards and along the reverse node’s solution path (line 19).

If the trees have not encountered each other yet, the queue is empty and the forward tree instead expands towards the sampled x_{rand} with the shared EXPAND function ($\sigma = \text{PROP}$, line 21). In the explore case, or whenever the exploitation fails, the forward tree expands with a random action rollout from its nearest node (RANDOMROLLOUT, Alg. 8, lines 22-25). As in KIST, the random rollout is what preserves probabilistic completeness.

Since the forward tree remains kinodynamically feasible, a new node that lands in the goal region can be used to extract the solution path (line 26). Otherwise, if the new forward node has a reverse node within its heuristic radius r , its heuristic (Eq. 4.5) is computed with respect to the best neighbor and the node is pushed onto Q for later exploitation (lines 27-30). Note that, unlike ST-Db-RRT and KIST, the trees are *not* swapped: the reverse tree and forward tree are expanded in every iteration.

Theoretical Analysis

ST-GBRRT is a direct transfer of GBRRT to space-time, so we can adapt the proof of probabilistic completeness of GBRRT [8] to ST-GBRRT. The best-input selection of motion primitives (SELECTMOTION, Alg. 3) is not complete, but completeness is restored by the random action rollout (Alg. 13, lines 22-25, and Alg. 8). The shared space-time search strategy (Sec. 4.2.1) ensures that the goal region is expanded and that there is a positive probability of sampling a feasible goal arrival time.

Algorithm 13: ST-GBRRT

Input: $x_{\text{start}}, x_{\text{goal}}, \mathcal{X}, \mathcal{X}_{\text{goal}}, \mathcal{M}, \delta, P, \mathcal{U}$

```

1  $Q \leftarrow \emptyset$  // forward nodes ordered by  $h$ 
2  $T_{\text{fwd}} \leftarrow \{x_{\text{start}}\}; T_{\text{rev}} \leftarrow \{x_{\text{goal}}\}$ 
3  $B \leftarrow \text{INITBATCHVARIABLES}(P)$ 
4 while  $\neg \text{TERMINATECONDITION}()$  do
5    $B \leftarrow \text{UPDATEGOALREGION}(B, P)$ 
6    $\text{SAMPLEGOAL}(x_g, B)$ 
7    $x_{\text{rand}} \leftarrow \text{SAMPLECONDITIONALLY}(\mathcal{X}, B)$ 
8    $r \leftarrow \min(\gamma (\frac{\log |T_{\text{rev}}|}{|T_{\text{rev}}|})^{1/(d+1)}, r_{\text{max}})$ 
   // reverse tree: best-input primitive expansion
9    $x_{\text{rev}} \leftarrow \text{EXPAND}(T_{\text{rev}}, x_{\text{rand}}, \mathcal{M}, \delta, \text{BEST})$  // see Alg. 4
10  if  $\text{VALID}(x_{\text{rev}})$  then
11    foreach  $x \in \text{NEARESTTR}(T_{\text{fwd}}, x_{\text{rev}}, r)$  do
12       $h \leftarrow \underline{t}(x, x_{\text{rev}}) + g(x_{\text{rev}})$  // Eq. 4.5
13      if  $h < x.h$  then  $x.h \leftarrow h; Q.\text{UPDATE}(x)$ 
   // forward tree: exploit reverse-tree heuristic
14   $x_{\text{new}} \leftarrow \text{NULL}; c \sim \text{RANDOM}(0, 1)$ 
15  if  $c < p_{\text{exp}}$  then
16     $x_{\text{pop}} \leftarrow Q.\text{POP}()$ 
17    if  $x_{\text{pop}} \neq \text{NULL}$  then
18       $x_{\text{best}} \leftarrow \arg \min_{x \in \text{NEARESTTR}(T_{\text{rev}}, x_{\text{pop}}, r)} \underline{t}(x_{\text{pop}}, x) + g(x)$ 
19       $x_{\text{new}} \leftarrow \text{SELECTMOTION}(\text{APPLICABLEMOTIONS}(x_{\text{pop}}, \mathcal{M}, \delta), x_{\text{pop}}, x_{\text{best}}, \text{PROP})$ 
20      if  $\text{VALID}(x_{\text{new}})$  then  $T_{\text{fwd}} \leftarrow T_{\text{fwd}} \cup x_{\text{new}}$ 
21      if  $\neg \text{VALID}(x_{\text{new}})$  then  $x_{\text{new}} \leftarrow \text{EXPAND}(T_{\text{fwd}}, x_{\text{rand}}, \mathcal{M}, \delta, \text{PROP})$ 
        // expand to random sample
   // explore: random rollout for PC
22  if  $\neg \text{VALID}(x_{\text{new}})$  then
23     $x_{\text{new}} \leftarrow \text{RANDOMROLLOUT}(\text{NEAREST}(T_{\text{fwd}}, x_{\text{rand}}), \mathcal{U}, t_{\text{max}})$ 
24    if  $\neg \text{VALID}(x_{\text{new}})$  then continue
25     $T_{\text{fwd}} \leftarrow T_{\text{fwd}} \cup x_{\text{new}}$ 
26  if  $d(x_{\text{new}}, \mathcal{X}_{\text{goal}}) < \epsilon_{\text{goal}}$  then return  $\text{EXTRACTSOLUTION}(x_{\text{new}})$ 
27   $x_{\text{rev}} \leftarrow \arg \min_{x \in \text{NEARESTTR}(T_{\text{rev}}, x_{\text{new}}, r)} \underline{t}(x_{\text{new}}, x) + g(x)$ 
28  if  $x_{\text{rev}} \neq \text{NULL}$  then
29     $x_{\text{new}}.h \leftarrow \underline{t}(x_{\text{new}}, x_{\text{rev}}) + g(x_{\text{rev}})$  // Eq. 4.5
30     $Q.\text{PUSH}(x_{\text{new}})$ 
31 return FAILURE

```

5. Experiments and Benchmark

This chapter evaluates the three planners presented in Chapter 4. The experiments are designed to answer and support the three introduced research objectives. First, investigate the influence of parameters on the performance of the planners to identify a common parameter set for all planners and environments. Secondly, investigate the practical consequence of planning in space-time by comparing the space-time variant of Db-RRT with its original version. Third, compare the three space-time planners against each other to identify their strengths and weaknesses.

The presented algorithms were implemented in *Dynoplan* [3], which already provides the used dynamical systems including motion primitives and infrastructure to run motion planning algorithms in environments. For this work, we extended the existing infrastructure to deal with dynamic obstacles. In the following sections we describe the dynamical systems, environments and algorithm configurations used in the benchmark. The actual evaluation is then presented in two parts: a parameter study (Section 5.5) that examines the influence of the planner parameters and selects a common parameter set, followed by the planner comparison of Db-RRT vs ST-Db-RRT (Section 5.6.1) and ST-Db-RRT vs KIST vs ST-GBRRT (Section 5.6.2).

The individual instances are each run on a single-core in parallel on a AMD Ryzen Threadripper PRO 5975WX @ 3,60 GHz with 128GB running Ubuntu 22.04.05.

5.1 Dynamical Systems

In our benchmark, we use four dynamical systems which are all adopted from *Dynoplan* [3] and the Planar Rotor from [1].

Unicycle 1st Order

The first system used is the 1st order unicycle [6, Eq. 13.18] with a 3-dimensional state space $[x, y, \theta] \in SE(2)$ and a 2-dimensional velocity control space $[u_v, u_\omega] \in U$. In the benchmark we stick to version (v0) with control bounds $u_v \in [-0.5, 0.5]$ m/s and $u_\omega \in [-0.5, 0.5]$ rad/s. With that the dynamics $\dot{x} = f(x, u)$ are defined by

$$\begin{aligned}\dot{x} &= u_v \cdot \cos(\theta) \\ \dot{y} &= u_v \cdot \sin(\theta) \\ \dot{\theta} &= u_\omega.\end{aligned}$$

Unicycle 2nd Order

The 2nd Order Unicycle [6, Eq. 13.46] includes velocities in the 5-dimensional state space $[x, y, \theta, v, \omega] \in \mathcal{X} \subset \mathbb{R}^5$ and has an acceleration control space $[u_a, u_\alpha] \in \mathcal{U} \subset \mathbb{R}^2$. We use the control bounds $u_a \in [-0.25, 0.25]$ m/s² and $u_\alpha \in [-0.25, 0.25]$ rad/s² and state bounds $v \in [-0.5, 0.5]$ m/s and $\omega \in [-0.5, 0.5]$ rad/s. The dynamics are defined with

$$\begin{aligned}\dot{x} &= v \cdot \cos(\theta) \\ \dot{y} &= v \cdot \sin(\theta) \\ \dot{\theta} &= \omega \\ \dot{v} &= u_a \\ \dot{\omega} &= u_\alpha.\end{aligned}$$

Car with trailer

The car with one trailer has a 4-dimensional state space $[x, y, \theta_0, \theta_1] \in \mathbb{R}^4$ with a trailer orientation θ_1 and 2-dimensional control space $[u_v, u_\phi] \in U$. We use $L = 0.25$, hitch length $d_1 = 0.5$ and the control bounds $u_v \in [-0.1, 0.5]$ m/s and $u_\phi \in [-\pi/3, \pi/3]$ rad. The dynamics are defined with

$$\begin{aligned}\dot{x} &= u_v \cdot \cos(\theta_0) \\ \dot{y} &= u_v \cdot \sin(\theta_0) \\ \dot{\theta}_0 &= \frac{u_v}{L} \cdot \tan(u_\phi) \\ \dot{\theta}_1 &= \frac{u_v}{d_1} \cdot \sin(\theta_0 - \theta_1).\end{aligned}$$

Planar Rotor

This system resembles a quadrotor living on a plane with only two rotors. It has a 6-dimensional state space $[x, z, \theta, v_x, v_z, \omega] \in \mathbb{R}^6$ including the position x, z , orientation θ and linear and angular velocities v_x, v_z, ω . The control space consists of two rotor forces $[f_1, f_2] \in \mathbb{R}^2$ with the control bounds $f_1, f_2 \in [0, 1.3]$ N. The dynamics are defined with

$$\begin{aligned}\dot{x} &= v_x \\ \dot{z} &= v_z \\ \dot{\theta} &= \omega \\ \dot{v}_x &= -\frac{f_1 + f_2}{m} \sin(\theta) \\ \dot{v}_z &= \frac{f_1 + f_2}{m} \cos(\theta) - g \\ \dot{\omega} &= \frac{l}{I}(f_1 - f_2)\end{aligned}$$

with mass $m = 0.034$, inertia $I = 0.0004$ and length $l = 0.1$ and gravity g .

5.2 Motion Primitives

All presented planners use motion primitives for fast kinodynamic tree expansion. In the theoretical analysis for ST-Db-RRT in Section 14, we argue that completeness in space-time is only preserved if the motion primitives also include time-suboptimal motions. However, in the practical evaluation, we use the time-optimal motion primitives included in Dynoplan [3] for the following reasons: First, the incompleteness established in Section 14 is an asymptotic property whose effect on finite-time performance is not clear and Section 5.5.1 tests this directly. Second, how to cover the unbounded time dimension is an open question and in practice the planners can still perform time-suboptimal expansions by chaining together multiple time-optimal motion primitives. Third, generating motion primitives with random forward propagation that have a good state space coverage for more sensitive systems, like flying robots such as the Planar Rotor, is non-trivial. We validate this choice empirically at the beginning of the parameter study (Section 5.5.1).

Figure 5.1 shows a plot of 200 motion primitives for each system with for a fixed start configuration and $\delta = 0.1$ (except unicycle2 $\delta = 0.25$).

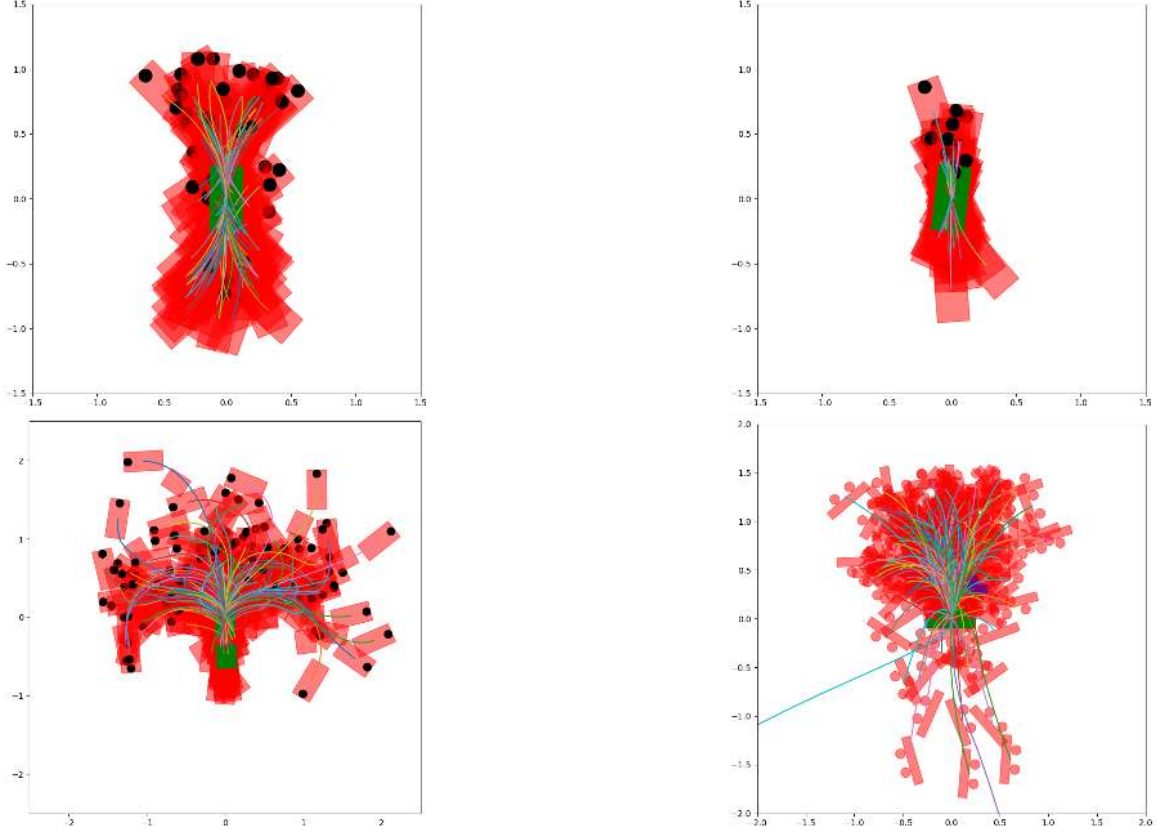


Figure 5.1: Plot of 200 motion primitives for each dynamical system with fixed start configuration and $\delta = 0.1$. 1st Order Unicycle (top left), 2nd Order Unicycle (top right), Car with trailer (bottom left) and Planar Rotor (bottom right).

5.3 Environments

The evaluation uses a suite of 16 dynamic environments, including 10 newly designed and 6 dynamic adaptations of static environments from *dynobench* [1]. An overview of

the environments is given in Table 5.1, grouped by the dynamics model they target. Beyond their spatial layout, the environments differ in their *temporal structure*, that is, when a passage becomes traversable and for how long it stays open. Most impose only a loose timing requirement, where the goal is reachable across a wide range of arrival times, whereas the *Wait* and *Elevator* admit a solution only within a narrow window at a specific moment within a confined space. This temporal dimension is precisely what the space-time sampling schedule must resolve (Section 5.5.3), so the relevant timing of each environment is highlighted in Table 5.1 and a detailed description of the environments follows in the next sections.

Environment	System	Source	Challenge
Revolving Door	Unicycle (1st + 2nd)	new	single rotating opening
Double Revolving Door	Unicycle (1st + 2nd)	new	two rotating openings
Multi Revolving Door	Unicycle (1st + 2nd)	new	several rotating openings
Double Moving Window	Unicycle (1st + 2nd)	adapted	two moving narrow gaps
Shortcut	Unicycle (1st + 2nd)	new	3 s opening at 12 s
Bugtrap	Unicycle (1st + 2nd)	adapted	exit blocked for a few seconds
Wait	Unicycle (1st + 2nd)	new	30 s wait, 1 s window
Elevator	Unicycle (1st + 2nd)	new	tight motion among obstacles
Lane Switch	Unicycle (1st + 2nd)	new	merge into moving traffic gap
Lane Switch	Car + trailer	new	merge into moving traffic gap
Park	Car + trailer	adapted	parallel park, spot blocked
Parking Lot	Car + trailer	new	traverse lot, avoid moving cars
Moving Window	Planar rotor	adapted	single translating gap
Double Moving Window	Planar rotor	adapted	two opposing gaps
Moving Small Window	Planar rotor	adapted	narrow translating gap
Elevator	Planar rotor	new	tight motion among obstacles

Table 5.1: Dynamic evaluation environments grouped by dynamics model.

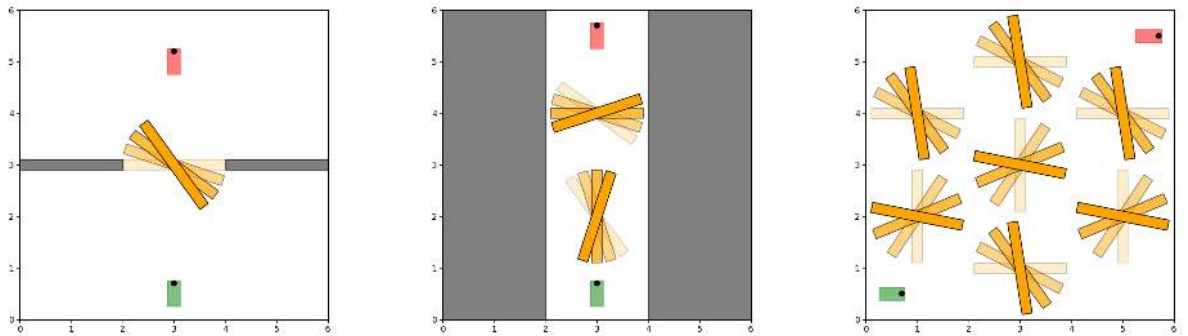


Figure 5.2: Visualization of the dynamic environments from left to right: *Revolving Door*, *Double Revolving Door* and *Multi Revolving Door* with start state (green), goal state (red), dynamic obstacles (yellow) and static obstacles (grey).

Unicycle

The first- and second-order unicycle share the same environments. Most of these environments stress navigation through gaps that open and close over time. In the *Revolving Door*, *Double Revolving Door*, and *Multi Revolving Door* environments (Figure 5.2), the robot must pass through one, two, or several revolving-door openings, respectively. Each additional door raises the temporal difficulty, as the robot must now time its passage through several openings that each align only briefly. The *Double Moving Window* (Figure 5.3), adapted from the planar-rotor environment of the same name, requires the unicycle to pass through two continuously moving narrow gaps. The *Shortcut* environment offers a suboptimal path around a wall alongside a door that opens after 12 s and stays open for roughly 3 s, revealing a direct route to the goal. Reaching the goal optimally requires exploiting this brief later time window. The *Bugtrap* environment adapts the classic motion-planning benchmark by blocking the trap exit for a few seconds. The *Wait* environment (Figure 5.5, top) is designed specifically to demonstrate the incompleteness of Db-RRT (Remark 1, Section 4.3.1): it forces the robot to wait about 30 s in a tightly restricted region and admits only a 1 s window in which it can proceed. The *Elevator* environment requires the unicycle to enter an elevator and move in unison with it to remain collision-free. The goal and its reverse trees are isolated until the elevator arrives, so here a late arrival time is coupled with the ability to navigate in a confined space. Finally, the *Lane Switch* environment places the unicycle in crowded traffic, where it must merge into a gap between moving obstacles, so the feasible window travels with the traffic.

Car with trailer

For the car-with-trailer dynamics, we designed environments that reflect realistic driving scenarios. The *Lane Switch* environment (Figure 5.4, top) simulates a lane-change maneuver in crowded traffic. The *Park* environment is a parallel-parking scenario in which another car initially blocks the parking space. The *Parking Lot* environment (Figure 5.4, bottom) requires the car to traverse a car park into a parking spot while avoiding other moving cars.

Planar rotor

The Planar Rotor environments require the rotor to fly through narrow gaps that translate back and forth. The *Moving Window* environment (Figure 5.6, top) is the simplest and *Double Moving Window* (bottom) and *Moving Small Window* tighten this timing further with two oppositely moving gaps and a narrower gap, respectively. As with the unicycle, the *Elevator* environment (Figure 5.5, bottom) requires the rotor to move tightly alongside other obstacles to avoid collision with a late feasible arrival time.

5.4 Algorithms

In the following evaluation, the space-time planners use the same goal region of 0.3, are configured with the same distance metric (Section 3.4.2) and use the same motion primitives (Section 5.2). The central parameters of the planners are tuned in a parameter study (Section 5.5) and then fixed for the planner comparison (Section 5.6)

We set the planner specific parameters as follows: For KIST, the maximal queue size of

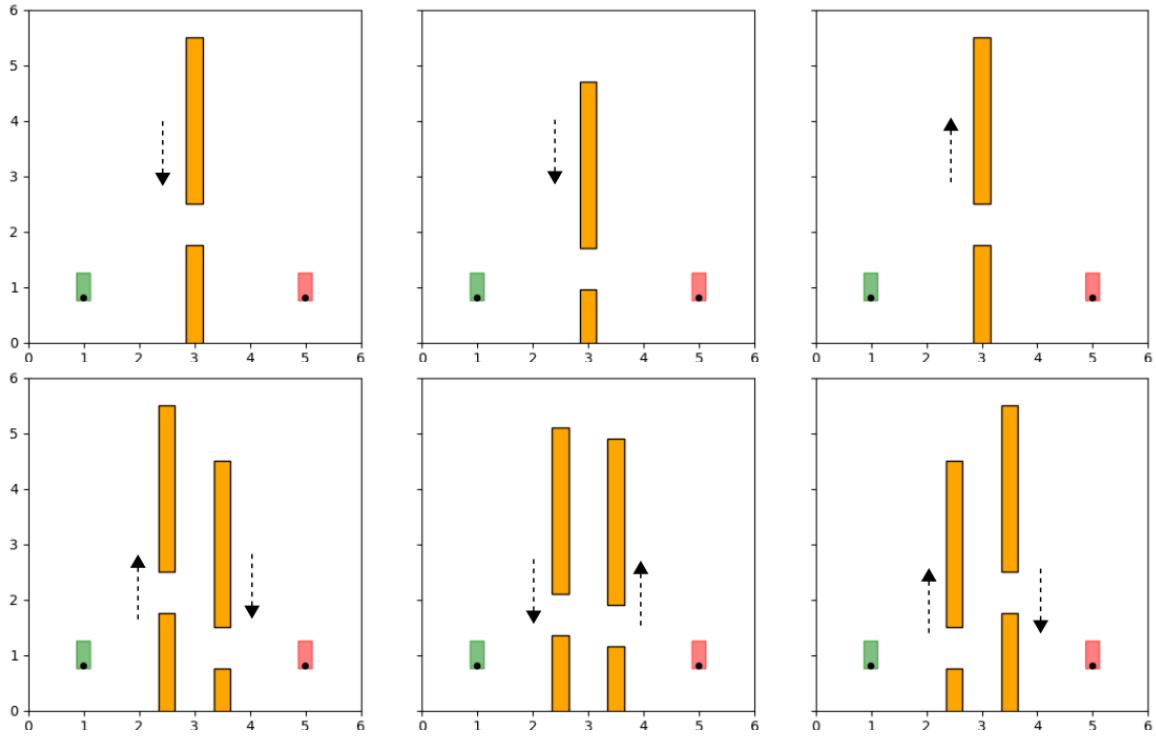


Figure 5.3: Top row: *Moving Window* environment. One window moving up and down. Bottom row: *Double Moving Window* environment. Two concatenated windows moving in opposite directions up and down.

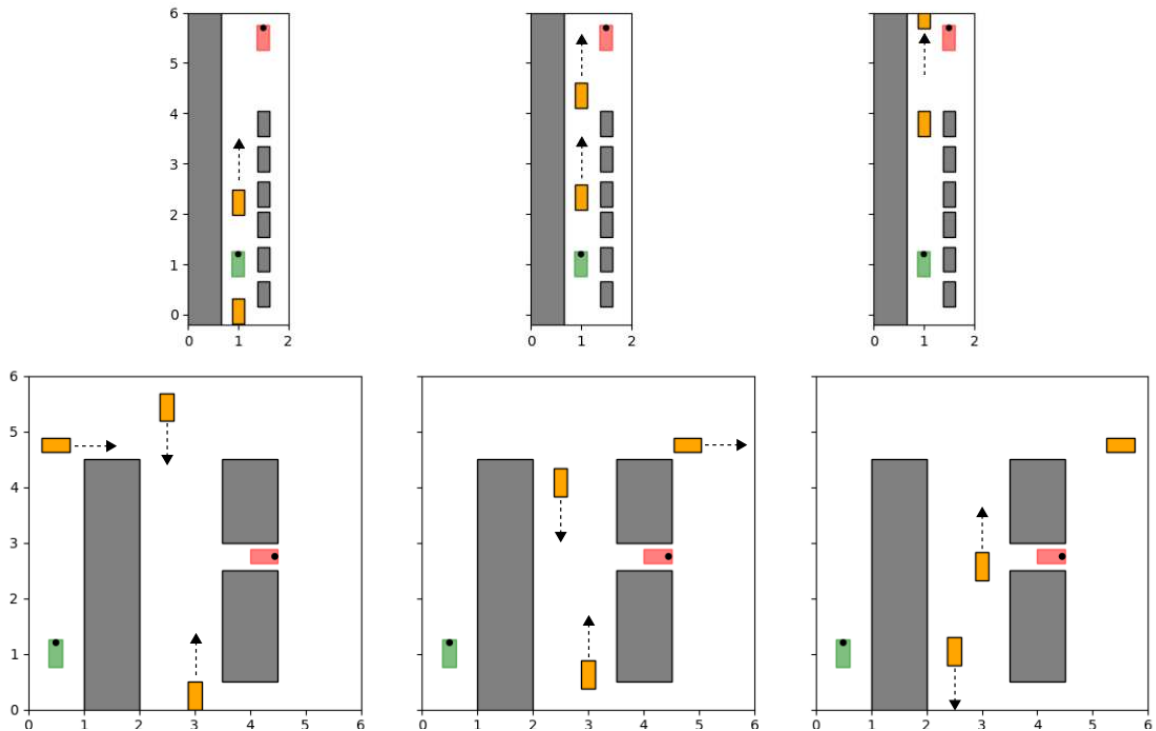


Figure 5.4: Top row: Lane Switch, the robot needs to move with the adjacent obstacles and switch the lane. Bottom row: The robot needs to navigate a parking lot, with several moving obstacles.

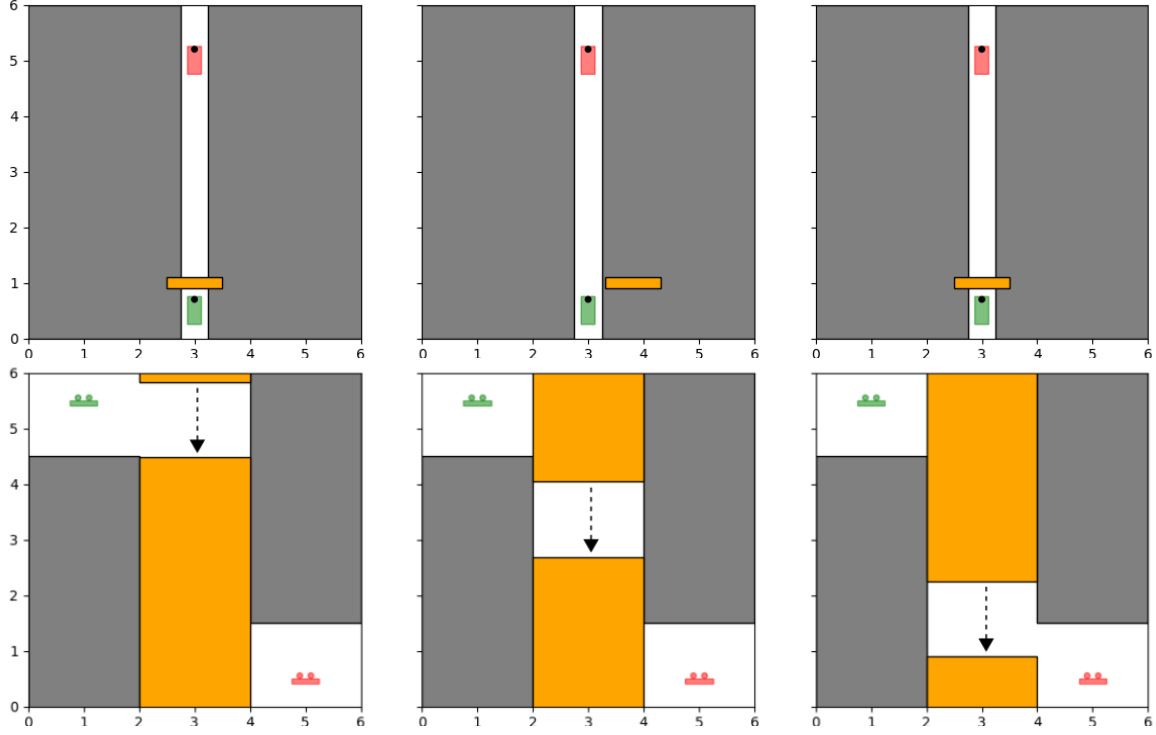


Figure 5.5: Top row: The Wait environment where the door opens for 1s. Bottom row: The Elevator environment.

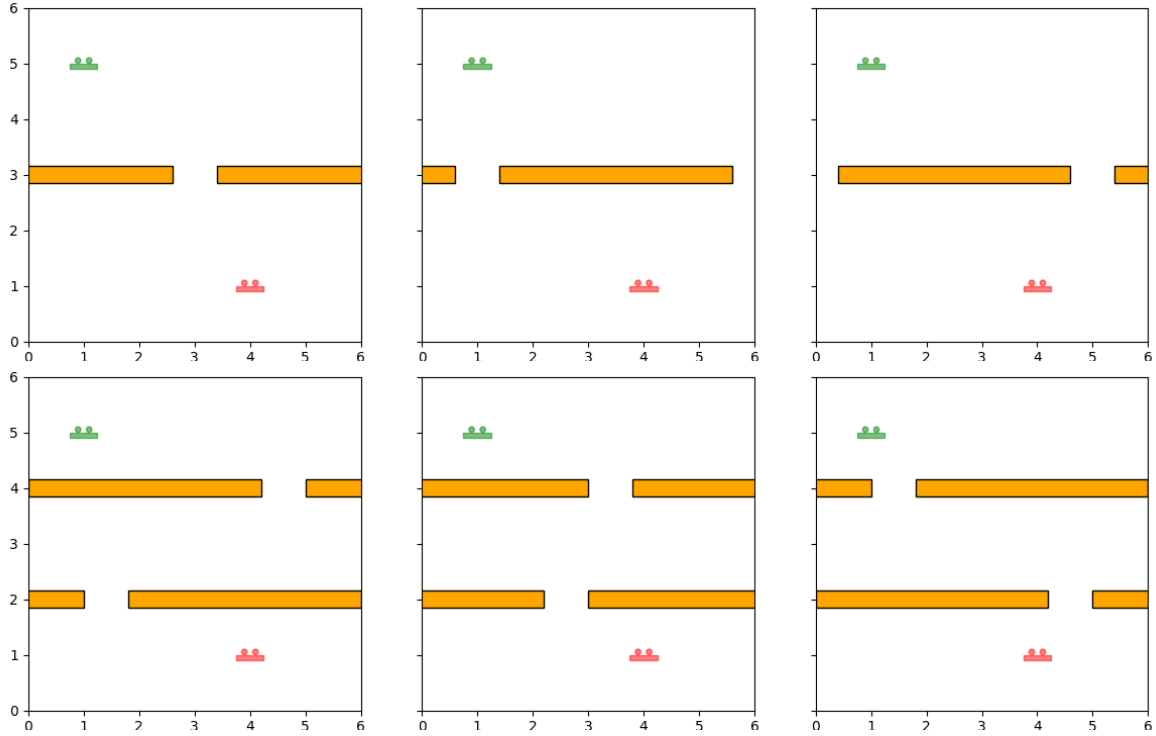


Figure 5.6: Visualization of two dynamic environments for the Planar Rotor with moving windows. Top row: One window moving left and right. Bottom row: Two concatenated windows moving in opposite directions left and right.

the *Heuristic Completion* policy is $|Q| = 500$. For ST-GBRRT, the maximal heuristic

radius is set to $r_{\max} = 0.3$.

5.5 Parameter Study

The performance of all three planners depends on a set of shared parameters. For best performance, these can be tuned for a specific environment and dynamical system, e.g. using the Hyperplan framework [38], but in this work we aim to identify general trends across all environments and dynamical systems. For that we divide each parameter set into the coarse categories of *low*, *medium* and *high*. Before studying the parameters, we first validate the choice of time-optimal motion primitives made in Section 5.2 (Section 5.5.1). We then investigate three categories of parameters: 1) the *primitive density and discontinuity bound* (Section 5.5.2), 2) the *sampling time bounds* (Section 5.5.3) and 3) the *time weight* (Section 5.5.4). Based on the observed trends, we conclude the study by selecting a common parameter set for the subsequent planner comparison (Section 5.5.5).

The parameter study is restricted to a representative subset of environments. For all dynamics we include the *Empty* environment to have a static instance to compare to. For the unicycle we use the environments *Revolving Door*, *Double Revolving Door*, *Multi Revolving Door* to see effects on increasingly more dynamic environments and for the 1st Order Unicycle we additionally use the environments *Elevator* and *Wait* to see effects on temporally constrained environments. For the car we added the *Parking Lot* environment to have a environment which requires to plan a relatively long trajectory and for the Planar Rotor we use the *Moving Window* as a simple dynamic environment and the *Elevator* as a temporally and spatially constrained environment.

All benchmarks in this chapter share the same protocol: each table reports the average over 20 runs of the solution cost c , the time to the first solution t_1 , and the success rate sr for each algorithm across the benchmark instances, with a time limit of 180s per run. In the parameter study, the best value per metric across the three parameter settings is highlighted in bold for every instance.

5.5.1 Time - Optimal vs. Random Motion Primitives

This experiment validates the claims of Section 5.2 by comparing the time-optimal with random motion primitives for ST-Db-RRT and KIST. We generated random motion primitives using forward propagation of randomly sampled controls for a random duration ensuring that both primitive sets contain motions of comparable duration. Deviating from the other benchmarks, Figure 5.7 reports the metrics over 10 runs on a subset of the environments, including the time-sensitive *Wait* environment. The results show that the time-optimal motion primitives perform generally better, with lower cost and faster first solution times. Most importantly, the success rate is not negatively affected for the time-optimal motion primitives, especially in the time-sensitive *Wait* environment. This shows that in practice the time-optimal motion primitives are sufficient to solve the benchmark environments, and we therefore use them for the rest of the evaluation.

5.5.2 Primitive Density and Discontinuity Bound

With the loss of the iterative adaption of the amount of motion primitives and the discontinuity bound (see Section 4.3), these parameters need to be set manually again.

Dynamics	Instance	ST-Db-RRT: Opt. Prim.			ST-Db-RRT: Rand. Prim.			KIST: Opt. Prim.			KIST: Rand. Prim.		
		c [s]	t_1 [s]	sr [p]	c [s]	t_1 [s]	sr [p]	c [s]	t_1 [s]	sr [p]	c [s]	t_1 [s]	sr [p]
1st Order Unicycle	Empty	14.7	0.5	1.0	39.9	2.2	1.0	15.2	1.7	1.0	15.0	1.8	1.0
1st Order Unicycle	Multi Revolving Door	17.8	3.7	1.0	84.8	43.0	0.9	18.0	7.4	1.0	17.9	7.0	1.0
1st Order Unicycle	Wait	39.5	4.0	1.0	-	-	0.0	39.1	30.6	0.7	38.9	25.7	0.7
1st Order Unicycle	Elevator	17.4	2.0	0.3	-	-	0.0	16.7	1.0	0.8	16.7	0.9	0.8
2nd Order Unicycle	Empty	19.6	0.8	1.0	20.8	0.7	1.0	19.4	1.8	1.0	19.8	1.1	1.0
2nd Order Unicycle	Multi Revolving Door	22.2	4.1	1.0	24.2	4.3	1.0	35.5	6.8	1.0	30.7	8.1	1.0
2nd Order Unicycle	Wait	44.1	6.7	0.2	42.5	5.5	0.1	-	-	0.0	-	-	0.0
2nd Order Unicycle	Lane Switch	30.0	5.3	1.0	29.6	7.1	1.0	29.9	6.8	1.0	29.6	6.4	1.0
1st Order Car	Lane Switch	32.2	3.9	1.0	33.4	6.1	1.0	29.1	64.5	1.0	29.1	53.5	1.0
1st Order Car	Park	8.1	0.4	1.0	13.2	3.7	1.0	7.5	2.1	1.0	7.6	0.5	1.0

Figure 5.7: Comparing time-optimal and random motion primitives for ST-Db-RRT and KIST on a subset of the environments.

We investigate the influence of these parameters on the performance of the planner for all environments and dynamical systems, by conducting three experiments with different amount of motion primitives and discontinuity bounds. In general we use

- **Low Density:** 300 motion primitives and a discontinuity bound of $\delta = 0.3$ and 3000 motion primitives and a discontinuity bound of $\delta = 0.6$ for the planar rotor.
- **Medium Density:** 3000 motion primitives and a discontinuity bound of $\delta = 0.2$ and 16000 motion primitives and a discontinuity bound of $\delta = 0.5$ for the planar rotor.
- **High Density:** 10000 motion primitives and a discontinuity bound of $\delta = 0.1$ and 32000 motion primitives and a discontinuity bound of $\delta = 0.4$ for the planar rotor.

Dynamics	Instance	$ \mathcal{M} = 300/3000, \delta = 0.3/0.6$			$ \mathcal{M} = 3000/16000, \delta = 0.2/0.5$			$ \mathcal{M} = 10000/32000, \delta = 0.1/0.4$		
		c [s]	t_1 [s]	sr [%]	c [s]	t_1 [s]	sr [%]	c [s]	t_1 [s]	sr [%]
1st Order Unicycle	Empty	14.7	0.2	1.0	15.4	1.3	1.0	18.0	4.3	1.0
1st Order Unicycle	Revolving Door	10.6	0.5	1.0	10.9	1.9	1.0	12.7	12.8	1.0
1st Order Unicycle	Double Revolving Door	12.7	0.6	1.0	13.5	4.5	1.0	16.0	14.5	1.0
1st Order Unicycle	Multi Revolving Door	16.7	1.4	1.0	19.2	8.4	1.0	23.1	24.2	1.0
1st Order Unicycle	Elevator	16.6	0.9	0.4	16.5	1.0	0.1	17.2	0.8	0.1
1st Order Unicycle	Wait	-	-	0.0	39.0	5.9	1.0	39.6	11.7	0.8
2nd Order Unicycle	Empty	19.3	0.3	1.0	21.4	1.1	1.0	28.7	2.2	0.8
2nd Order Unicycle	Revolving Door	12.2	0.9	1.0	15.2	4.4	1.0	22.1	5.5	0.6
2nd Order Unicycle	Double Revolving Door	19.2	1.0	1.0	19.8	5.0	1.0	24.8	7.7	0.8
2nd Order Unicycle	Multi Revolving Door	21.0	1.4	1.0	24.8	7.1	1.0	31.6	9.8	0.9
1st Order Car	Empty	15.3	0.3	1.0	15.2	3.2	1.0	17.6	4.2	1.0
1st Order Car	Parking Lot	25.4	1.4	1.0	30.7	10.7	1.0	34.1	17.9	1.0
Planar Rotor	Elevator	14.1	43.9	1.0	15.3	159.5	0.4	-	-	0.0
Planar Rotor	Empty	1.9	3.3	1.0	2.3	22.3	1.0	2.9	51.9	0.9
Planar Rotor	Moving Window	3.3	18.9	1.0	4.7	104.4	0.9	5.4	119.6	0.7

Figure 5.8: ST-Db-RRT benchmark comparing low, medium and high density of motion primitives.

Dynamics	Instance	$ \mathcal{M} = 300/3000, \delta = 0.3/0.6$			$ \mathcal{M} = 3000/16000, \delta = 0.2/0.5$			$ \mathcal{M} = 10000/32000, \delta = 0.1/0.4$		
		c [s]	t_1 [s]	sr [%]	c [s]	t_1 [s]	sr [%]	c [s]	t_1 [s]	sr [%]
1st Order Unicycle	Empty	15.4	0.4	1.0	15.0	2.7	1.0	14.9	2.5	1.0
1st Order Unicycle	Revolving Door	10.8	0.6	1.0	10.8	3.6	1.0	10.8	3.7	1.0
1st Order Unicycle	Double Revolving Door	12.9	0.9	1.0	13.8	5.1	1.0	15.3	6.0	1.0
1st Order Unicycle	Multi Revolving Door	16.9	1.3	1.0	20.7	13.0	1.0	20.9	22.5	1.0
1st Order Unicycle	Elevator	16.8	11.1	0.7	16.6	9.4	0.8	16.9	2.3	0.7
1st Order Unicycle	Wait	-	-	0.0	39.0	23.9	0.9	38.9	19.3	0.9
2nd Order Unicycle	Empty	20.3	0.4	1.0	20.4	2.1	1.0	20.6	1.0	1.0
2nd Order Unicycle	Revolving Door	13.1	2.6	1.0	15.2	5.8	1.0	13.2	2.6	1.0
2nd Order Unicycle	Double Revolving Door	22.1	2.5	1.0	27.9	9.1	1.0	23.5	5.5	1.0
2nd Order Unicycle	Multi Revolving Door	22.8	3.1	1.0	36.7	11.9	1.0	27.7	4.9	1.0
1st Order Car	Empty	15.4	0.2	1.0	13.7	0.4	1.0	15.5	0.8	1.0
1st Order Car	Parking Lot	29.5	72.5	0.8	24.5	13.6	1.0	24.2	8.6	1.0
Planar Rotor	Elevator	15.6	143.1	0.7	-	-	0.0	-	-	0.0
Planar Rotor	Empty	2.4	1.8	1.0	2.7	10.6	1.0	2.9	24.1	1.0
Planar Rotor	Moving Window	3.7	19.0	1.0	5.5	83.8	0.8	6.3	96.3	0.8

Figure 5.9: KIST benchmark comparing low, medium and high density of motion primitives.

Dynamics	Instance	$ \mathcal{M} = 300/3000, \delta = 0.3/0.6$			$ \mathcal{M} = 3000/16000, \delta = 0.2/0.5$			$ \mathcal{M} = 10000/32000, \delta = 0.1/0.4$		
		c [s]	t_1 [s]	sr [%]	c [s]	t_1 [s]	sr [%]	c [s]	t_1 [s]	sr [%]
1st Order Unicycle	Empty	14.4	0.8	1.0	14.3	3.8	1.0	14.3	5.0	1.0
1st Order Unicycle	Revolving Door	10.5	1.1	1.0	10.6	5.1	1.0	10.6	8.7	1.0
1st Order Unicycle	Double Revolving Door	12.5	1.3	1.0	12.6	11.0	1.0	12.7	21.6	1.0
1st Order Unicycle	Multi Revolving Door	15.7	3.6	1.0	17.5	25.9	1.0	17.1	63.4	1.0
1st Order Unicycle	Elevator	16.3	10.0	1.0	16.7	14.7	1.0	16.9	43.3	0.9
1st Order Unicycle	Wait	-	-	0.0	-	-	0.0	-	-	0.0
2nd Order Unicycle	Empty	18.1	16.1	1.0	19.1	14.2	0.9	21.2	10.5	0.9
2nd Order Unicycle	Revolving Door	12.2	11.8	1.0	12.3	15.2	1.0	11.9	14.1	1.0
2nd Order Unicycle	Double Revolving Door	19.9	52.4	0.2	-	-	0.0	-	-	0.0
2nd Order Unicycle	Multi Revolving Door	21.5	56.9	0.8	20.5	75.6	0.5	22.3	48.1	0.8
1st Order Car	Empty	14.4	5.6	1.0	12.8	4.5	1.0	12.6	4.2	1.0
1st Order Car	Parking Lot	-	-	0.0	-	-	0.0	-	-	0.0
Planar Rotor	Elevator	-	-	0.0	-	-	0.0	-	-	0.0
Planar Rotor	Empty	2.5	91.1	0.2	2.7	88.4	0.2	-	-	0.0
Planar Rotor	Moving Window	-	-	0.0	-	-	0.0	2.4	124.4	0.1

Figure 5.10: ST-GBRRT benchmark comparing low, medium and high density of motion primitives.

ST-Db-RRT

Figure 5.8 shows the results for ST-Db-RRT. Refining the primitive set and decreasing the delta lowers the performance on nearly every instance. Both solution cost and time to first solution increase monotonically as the primitive count grows and δ tightens. Modestly for the first-order unicycle and car and steeply for the second-order unicycle, where cost on *Revolving Door* nearly doubles from the coarse to the refined setting. Success remains at 1.0 across the coarse and intermediate settings for the simpler dynamics, but at the refined setting it drops for the second-order unicycle and the planar rotor, the latter failing entirely on *Elevator*. The single exception is *Wait*, which is unsolvable in the coarse setting and only becomes solvable with at least intermediate amount of primitives.

KIST

As Figure 5.9 shows, KIST finds a solution fastest with the low density setting, except for the *Parking Lot* environment. The low and medium setting have similar costs in the unicycle environments, with the low setting having a few environments with slightly lower costs. The low density setting is not able to solve the *Wait* environment. The high density setting has lower costs in only three environments.

ST-GBRRT

For ST-GBRRT (Figure 5.10), refinement of the primitive set yields no consistent improvement in solution cost: across the unicycle and car instances the cost remains within a fraction of a second of its coarse-setting value, and on *Multi Revolving Door* it degrades slightly. The time to first solution, in contrast, grows by up to an order of magnitude as the primitive count increases and δ tightens. ST-GBRRT therefore performs best in its coarsest configuration. Its coverage is also the most limited of the three planners; it fails entirely on *Parking Lot*, *Double Window*, and *Wait*.

5.5.3 Sampling Time Bounds

All three planners embed the same space-time sampling strategy, which draws sample times uniformly between a lower and an upper time bound and enlarges the upper bound as the search progresses: the initial batch size determines how many samples are drawn before increasing the time bounds and the range factor determines how much the sampling time bounds are increased after each batch. This schedule is the only component of the planner that couples directly to the temporal structure of a problem, which makes it an interesting parameter to investigate. We compare three configurations spanning two qualitatively different behaviours:

- **Gradual Schedule:** a gradual schedule ($b=1000$, $f=1.5$) that advances the horizon in finer steps
- **Aggressive Schedule:** an aggressive schedule ($b=512$, $f=2.0$) that pushes the bound outward quickly while sampling each window only sparsely
- **Conservative Schedule:** a conservative schedule ($b=3000$, $f=2.0$) that samples each horizon densely before advancing it in a large jump

Since the schedules govern only how the time horizon is explored and not which trajectories are available, we expect the solution cost c to be determined predominantly by the environment and to remain largely invariant across schedules. The informative signal should instead appear in the success rate sr and the time to first solution t_1 , and be concentrated in temporally relevant environments, where the feasible arrival window must be sampled within a limited time. In temporally unconstrained environments, where the goal is reachable across a wide range of arrival times, we expect the schedules to perform comparably.

Dynamics	Instance	batch=1000, factor=1.5			batch=512, factor=2.0			batch=3000, factor=2.0		
		c [s]	t_1 [s]	sr [%]	c [s]	t_1 [s]	sr [%]	c [s]	t_1 [s]	sr [%]
1st Order Unicycle	Empty	15.1	1.5	1.0	15.4	0.7	1.0	15.5	1.2	1.0
1st Order Unicycle	Revolving Door	11.0	1.7	1.0	10.9	2.0	1.0	10.8	2.8	1.0
1st Order Unicycle	Double Revolving Door	13.3	3.8	1.0	13.2	1.9	1.0	13.7	6.7	1.0
1st Order Unicycle	Multi Revolving Door	18.6	7.3	1.0	19.2	4.2	1.0	18.4	9.7	1.0
1st Order Unicycle	Elevator	17.1	2.2	0.1	16.4	0.4	0.1	17.3	5.7	0.5
1st Order Unicycle	Wait	39.2	6.1	1.0	39.3	1.1	0.2	39.2	6.9	1.0
2nd Order Unicycle	Empty	21.7	1.2	1.0	21.7	0.7	1.0	21.6	1.1	1.0
2nd Order Unicycle	Revolving Door	14.4	3.7	1.0	16.4	20.2	0.8	13.9	4.8	1.0
2nd Order Unicycle	Double Revolving Door	19.8	5.4	1.0	20.0	7.5	0.9	20.0	4.9	1.0
2nd Order Unicycle	Multi Revolving Door	24.3	7.5	1.0	26.1	53.8	0.7	24.2	7.2	1.0
1st Order Car	Empty	14.7	3.9	1.0	16.9	1.1	1.0	15.0	3.2	1.0
1st Order Car	Parking Lot	32.2	11.8	1.0	32.9	2.7	1.0	32.5	9.4	1.0
Planar Rotor	Elevator	15.3	159.6	0.4	14.9	42.5	1.0	15.3	150.9	0.6
Planar Rotor	Empty	2.5	20.6	1.0	2.5	8.6	1.0	2.5	23.5	1.0
Planar Rotor	Moving Window	4.2	74.9	0.9	4.0	26.2	1.0	5.9	102.2	0.9

Figure 5.11: ST-Db-RRT benchmark comparing slow, medium and fast time bound expansion.

ST-Db-RRT

As Figure 5.11 shows, ST-Db-RRT is able to solve every instance with every schedule and, as expected, the effects concentrate in sr and t_1 . It achieves a relatively low success rate in the environments *Elevator* and *Wait*, which both have a late feasible arrival time and tight constrained spaces. The aggressive schedule (b=512, f=2.0) is uniquely strong for the *Planar Rotor Elevator* at $sr = 1.0$ ($t_1 = 42.5s$) against $sr = 0.4/0.6$ (with $t_1 \approx 150s$) and for the *Planar Rotor Moving Window* instance with a low $t_1 = 26.2s$ against $t_1 = 74.9s/102.2s$ for the gradual and conservative schedule. The aggressive schedule is weak on the other hand, just like the gradual schedule in the *Unicycle Elevator* instance with $sr = 0.1$ compared to the stronger conservative schedule with $sr = 0.5$. The success rate of the aggressive schedule also collapses in the *Wait* with short arrival time window, dropping to $sr = 0.2$ against $sr = 1.0$ for the other schedules.

KIST

Dynamics	Instance	batch=1000, factor=1.5			batch=512, factor=2.0			batch=3000, factor=2.0		
		c [s]	t_1 [s]	sr [%]	c [s]	t_1 [s]	sr [%]	c [s]	t_1 [s]	sr [%]
1st Order Unicycle	Empty	15.0	1.6	1.0	14.9	1.2	1.0	15.1	1.0	1.0
1st Order Unicycle	Revolving Door	10.9	2.7	1.0	10.8	2.7	1.0	10.8	4.8	1.0
1st Order Unicycle	Double Revolving Door	13.1	3.1	1.0	15.0	2.5	1.0	13.6	8.7	1.0
1st Order Unicycle	Multi Revolving Door	18.9	7.3	1.0	20.3	5.4	1.0	19.5	16.3	1.0
1st Order Unicycle	Elevator	16.7	13.4	0.7	16.8	1.9	0.7	16.8	2.6	0.8
1st Order Unicycle	Wait	39.0	19.9	0.9	38.9	33.8	0.9	40.0	94.9	0.9
2nd Order Unicycle	Empty	20.2	1.3	1.0	20.3	1.6	1.0	19.9	1.3	1.0
2nd Order Unicycle	Revolving Door	14.1	6.6	1.0	16.4	6.0	1.0	13.7	4.7	1.0
2nd Order Unicycle	Double Revolving Door	24.6	10.1	1.0	26.1	6.3	1.0	25.4	11.9	1.0
2nd Order Unicycle	Multi Revolving Door	34.0	9.7	1.0	37.2	12.7	1.0	33.0	18.1	1.0
1st Order Car	Empty	14.6	0.6	1.0	14.3	0.5	1.0	14.2	0.4	1.0
1st Order Car	Parking Lot	24.5	15.1	1.0	25.9	6.1	1.0	25.0	62.3	1.0
Planar Rotor	Elevator	-	-	0.0	16.8	132.5	0.8	-	-	0.0
Planar Rotor	Empty	2.6	10.4	1.0	2.6	8.1	1.0	2.6	14.0	1.0
Planar Rotor	Moving Window	5.7	65.8	0.9	7.9	74.3	0.9	5.2	93.0	0.8

Figure 5.12: KIST benchmark comparing slow, medium and fast time bound expansion.

KIST (Figure 5.12) is able to solve all instances except the Planar Rotor environment

Elevator with the gradual and conservative schedule, while the aggressive schedule clearly wins with a success rate of $sr = 0.8$. Other than that, compared to ST-Db-RRT it shows a more robust success rate of $sr \geq 0.7$ in all other environments and schedules and very consistent success rates throughout the schedules.

The only other signal we observe lives in the t_1 which inflates markedly for the conservative schedule on temporally constrained and late arrival time instances (*Wait* 94.9s, *Parking Lot* 62.3s, *Multi Revolving Door* 16.3s), while the aggressive and gradual schedule generally reaches the first solution faster.

ST-GBRRT

Dynamics	Instance	batch=1000, factor=1.5			batch=512, factor=2.0			batch=3000, factor=2.0		
		c [s]	t_1 [s]	sr [%]	c [s]	t_1 [s]	sr [%]	c [s]	t_1 [s]	sr [%]
1st Order Unicycle	Empty	14.3	5.2	1.0	14.3	4.4	1.0	14.3	4.3	1.0
1st Order Unicycle	Revolving Door	10.6	7.5	1.0	10.5	3.5	1.0	10.6	6.0	1.0
1st Order Unicycle	Double Revolving Door	12.7	11.6	1.0	12.6	6.2	1.0	12.6	6.4	1.0
1st Order Unicycle	Multi Revolving Door	16.5	24.3	1.0	16.5	37.1	1.0	16.5	26.9	1.0
1st Order Unicycle	Elevator	16.5	20.1	0.9	17.0	30.0	1.0	16.8	17.7	0.9
1st Order Unicycle	Wait	-	-	0.0	-	-	0.0	-	-	0.0
2nd Order Unicycle	Empty	18.5	23.0	0.9	19.1	21.1	0.8	19.3	16.5	0.9
2nd Order Unicycle	Revolving Door	12.0	15.8	1.0	12.3	17.5	1.0	12.3	14.4	1.0
2nd Order Unicycle	Double Revolving Door	-	-	0.0	-	-	0.0	-	-	0.0
2nd Order Unicycle	Multi Revolving Door	22.0	71.4	0.8	21.6	113.7	0.6	22.0	81.2	0.8
1st Order Car	Empty	12.7	7.3	1.0	12.8	9.1	1.0	12.7	6.3	1.0
1st Order Car	Parking Lot	-	-	0.0	-	-	0.0	-	-	0.0
Planar Rotor	Elevator	-	-	0.0	-	-	0.0	-	-	0.0
Planar Rotor	Empty	2.4	82.3	0.1	2.6	128.9	0.1	2.5	72.9	0.1
Planar Rotor	Moving Window	-	-	0.0	-	-	0.0	-	-	0.0

Figure 5.13: ST-GBRRT benchmark comparing slow, medium and fast time bound expansion.

ST-GBRRT (Figure 5.13) has the most limited coverage, failing entirely ($sr = 0$ in every schedule) on *Wait*, the 2nd Order Unicycle *Double Revolving Door*, *Parking Lot*, *Planar Rotor Elevator* and *Moving Window*, and reaching only $sr = 0.1$ on *Planar Rotor Empty*. On the instances it does solve, the schedules attain comparable success rates, and the parameter affects almost exclusively t_1 . No schedule dominates: the aggressive schedule is fastest on the single revolving-door instances (e.g. 3.5s vs 7.5s), the conservative on the empty and 2nd Order instances, and the gradual only on the multi-revolving-door cases.

5.5.4 Time Weight

The time weight λ is used to control the importance of the configuration space d_Q and time t of the state space. It is used in the space-time distance metric (see Section 3.6) and thus influences the nearest-neighbour selection, the selection of a motion primitives during expansion, and the definition of a connection between trees. We investigate the influence by running benchmarks with low $\lambda = 0.2$, equal $\lambda = 0.5$ and high $\lambda = 0.7$ weight.

ST-Db-RRT

For ST-Db-RRT (Figure 5.14) the lowest time weight $\lambda = 0.2$ is best on the large majority of environments. While the costs c are only slightly better for $\lambda = 0.2$, the time to first solution t_1 increases strongly and sometimes monotonically with λ , e.g. for the 2nd

Dynamics	Instance	$\lambda = 0.2$			$\lambda = 0.5$			$\lambda = 0.7$		
		c [s]	t_1 [s]	sr [p]	c [s]	t_1 [s]	sr [p]	c [s]	t_1 [s]	sr [p]
1st Order Unicycle	Empty	15.1	1.0	1.0	16.5	1.9	1.0	16.6	3.1	1.0
1st Order Unicycle	Revolving Door	10.8	1.3	1.0	11.1	4.2	1.0	11.2	6.4	1.0
1st Order Unicycle	Double Revolving Door	13.6	3.0	1.0	14.0	8.6	1.0	13.9	6.3	1.0
1st Order Unicycle	Multi Revolving Door	18.9	7.2	1.0	20.2	14.3	1.0	20.6	14.5	1.0
1st Order Unicycle	Elevator	17.1	2.0	0.3	17.5	5.8	0.2	16.9	4.6	0.1
1st Order Unicycle	Wait	39.1	4.7	0.9	39.2	5.7	0.8	39.3	6.1	0.6
2nd Order Unicycle	Empty	21.7	0.8	1.0	21.9	1.5	1.0	22.3	2.1	1.0
2nd Order Unicycle	Revolving Door	15.0	3.3	1.0	15.1	5.3	0.9	14.2	6.1	0.8
2nd Order Unicycle	Double Revolving Door	19.8	4.2	1.0	30.4	15.8	0.9	20.0	9.5	0.9
2nd Order Unicycle	Multi Revolving Door	24.6	4.8	1.0	24.9	10.0	1.0	26.2	20.6	1.0
1st Order Car	Empty	16.8	3.3	1.0	15.7	3.6	1.0	15.4	3.3	1.0
1st Order Car	Parking Lot	30.4	8.8	1.0	31.9	8.3	1.0	32.7	11.0	1.0
Planar Rotor	Elevator	16.1	139.5	0.8	15.5	139.0	0.7	16.2	140.1	0.6
Planar Rotor	Empty	2.4	21.4	1.0	2.7	29.2	1.0	2.4	39.4	1.0
Planar Rotor	Moving Window	3.9	70.0	1.0	4.4	65.9	1.0	4.5	73.1	0.8

Figure 5.14: ST-Db-RRT benchmark comparing low, medium and high time weight.

Order Unicycle Multi Revolving Door ($t_1 = 4.8s/10.0s/20.6s$). On the temporally more constrained instances the success rate decreases with λ , as in the Elevator for Planar Rotor ($sr = 0.8/0.7/0.6$) and 1st Order Unicycle Elevator ($sr = 0.3/0.2/0.1$) and Wait ($sr = 0.9/0.8/0.6$). Same can be observed for the 2nd Order Unicycle Single and Double Revolving Door.

KIST

Dynamics	Instance	$\lambda = 0.2$			$\lambda = 0.5$			$\lambda = 0.7$		
		c [s]	t_1 [s]	sr [p]	c [s]	t_1 [s]	sr [p]	c [s]	t_1 [s]	sr [p]
1st Order Unicycle	Empty	15.0	1.5	1.0	15.3	0.5	1.0	16.0	0.6	1.0
1st Order Unicycle	Revolving Door	10.7	2.2	1.0	10.8	3.1	1.0	11.4	3.8	1.0
1st Order Unicycle	Double Revolving Door	13.9	1.4	1.0	14.2	3.2	1.0	15.0	4.4	1.0
1st Order Unicycle	Multi Revolving Door	21.1	5.8	1.0	37.6	51.0	0.9	34.6	21.2	1.0
1st Order Unicycle	Elevator	16.6	1.1	0.6	18.7	15.9	0.5	19.5	6.9	0.5
1st Order Unicycle	Wait	38.9	20.1	0.9	39.3	22.4	0.9	39.3	9.0	0.9
2nd Order Unicycle	Empty	20.6	1.4	1.0	21.0	3.1	1.0	61.9	12.6	1.0
2nd Order Unicycle	Revolving Door	16.5	4.9	1.0	26.6	32.3	0.9	53.5	46.4	0.7
2nd Order Unicycle	Double Revolving Door	25.3	7.4	1.0	58.0	41.4	0.9	69.5	34.4	0.9
2nd Order Unicycle	Multi Revolving Door	40.5	14.1	1.0	262.9	88.3	1.0	196.8	98.6	0.6
1st Order Car	Empty	14.3	0.3	1.0	14.1	1.2	1.0	17.1	3.5	1.0
1st Order Car	Parking Lot	25.4	6.5	1.0	28.9	16.3	1.0	34.3	38.9	0.8
Planar Rotor	Elevator	16.4	130.7	0.8	16.9	120.0	0.6	18.1	113.2	0.6
Planar Rotor	Empty	2.8	8.8	1.0	2.9	9.2	1.0	3.1	13.5	1.0
Planar Rotor	Moving Window	6.1	39.1	0.9	7.2	46.3	0.9	6.0	47.9	0.9

Figure 5.15: KIST benchmark comparing low, medium and high time weight.

KIST (Figure 5.15) also shows a clear preference for a small time weight $\lambda = 0.2$. Unlike for ST-Db-RRT this preference is also reflected in significantly lower cost c for some environments. For the 1st Order Unicycle *Multi Revolving Door* environment the cost goes from $c = 21.1s$ for $\lambda = 0.2$ to $c = 37.6s/34.6s$, and in all 2nd Order Unicycle Revolving Door environments the cost more than doubles, e.g. in the *Revolving Door* with $c = 16.5s/26.6s/53.5s$.

The time to first solution t_1 is generally the lowest for $\lambda = 0.2$ and often increases monotonically with λ . However, there are a few instances, where the high time weight

$\lambda = 0.7$ finds solution significantly faster than the medium and even low time weight. Most outstanding is the 1st Order Unicycle *Wait* environment, where KIST with $\lambda = 0.7$ finds a solution in $t_1 = 9.0s$, less than half of the time than the other parameters with $t_1 = 20.1s/22.4s$. A weaker form of the same effect appears in the 1st Order Unicycle *Elevator* and *Multi Revolving Door* environments, where the high time weight beats at least the equal time weight.

The success rate is highest for the small time weight and decreases slightly for higher λ in a few environments. Examples are 1st and 2nd Order Unicycle *Elevator* $sr = 0.6/0.5/0.5$ and $sr = 0.8/0.6/0.6$ or the 2nd Order Unicycle *Revolving Door* environment $sr = 1.0/0.9/0.7$.

ST-GBRRT

Dynamics	Instance	$\lambda = 0.2$			$\lambda = 0.5$			$\lambda = 0.7$		
		c [s]	t_1 [s]	sr [p]	c [s]	t_1 [s]	sr [p]	c [s]	t_1 [s]	sr [p]
1st Order Unicycle	Empty	14.3	3.4	1.0	14.3	5.4	1.0	14.3	5.5	1.0
1st Order Unicycle	Revolving Door	10.5	4.6	1.0	10.6	3.6	1.0	10.6	6.8	1.0
1st Order Unicycle	Double Revolving Door	12.6	10.8	1.0	12.6	11.7	1.0	12.6	16.5	1.0
1st Order Unicycle	Multi Revolving Door	16.3	34.8	1.0	16.6	49.6	1.0	17.2	49.6	0.9
1st Order Unicycle	Elevator	16.7	18.5	1.0	16.5	26.2	1.0	16.7	40.6	0.9
1st Order Unicycle	Wait	-	-	0.0	-	-	0.0	-	-	0.0
2nd Order Unicycle	Empty	18.4	13.0	0.9	18.1	24.3	1.0	18.7	24.9	1.0
2nd Order Unicycle	Revolving Door	12.2	12.0	1.0	11.8	14.2	1.0	11.8	22.3	1.0
2nd Order Unicycle	Double Revolving Door	20.4	108.2	0.1	19.0	70.1	0.1	19.0	74.9	0.1
2nd Order Unicycle	Multi Revolving Door	21.2	75.1	0.8	21.5	62.6	0.8	21.4	84.3	0.8
1st Order Car	Empty	13.0	5.3	1.0	12.9	6.0	1.0	12.9	6.6	1.0
1st Order Car	Parking Lot	-	-	0.0	-	-	0.0	-	-	0.0
Planar Rotor	Elevator	-	-	0.0	-	-	0.0	-	-	0.0
Planar Rotor	Empty	2.6	127.5	0.1	2.6	78.5	0.2	2.5	102.1	0.2
Planar Rotor	Moving Window	-	-	0.0	-	-	0.0	-	-	0.0

Figure 5.16: ST-GBRRT benchmark comparing low, medium and high time weight.

For ST-GBRRT (Figure 5.16) the trend for a lower time weight is less clear than for the two other planners.

First, we observe that the cost remains very constant across all parameters settings. The time to first solution t_1 is generally lowest for the small time weight and increases monotonically with λ for most instances. In three exceptions (the 2nd Order *Multi* and *Double Revolving Door* and the *Planar Rotor Empty* instance) the equal time weight $\lambda = 0.5$ finds solutions faster.

Success rates remain relatively constant across all time weights, with unusually low success rate $sr = 0.1$ for the 2nd Order Double Revolving Door.

5.5.5 Parameter Selection

We use the insights from the previous parameter benchmarks to select a common parameter set for the following planner comparisons:

- $|\mathcal{M}| = 1000$, $\delta = 0.3$: Between lowest and medium density ($|\mathcal{M}| = 3000$ for Planar Rotor)
- batch= 1200, factor= 1.7: Gradual schedule with more sample density
- $\lambda = 0.2$: Clearly preferred small time weight

For the amount of motion primitives we increased the number of primitives from the lowest preferred amount of $|\mathcal{M}| = 300$ to increase the success rate in difficult environments. We choose the gradual schedule with slightly increased batch size and factor to find a better middle ground between sample density and expansion speed. The time weight was set to the clearly preferred smallest value.

5.6 Planner Comparison

With the parameter set selected in the preceding study (Section 5.5.5), we now turn to the comparative benchmarks. We first isolate the effect of the space-time extension by comparing Db-RRT directly with its space-time variant ST-Db-RRT (Section 5.6.1), before comparing the three space-time planners against each other (Section 5.6.2).

For the planner comparison we now use the full suite of dynamic environments and like in the parameter benchmark share all benchmarks the same protocol: each table reports the average over 20 runs of the solution cost c , the time to the first solution t_1 , and the success rate sr for each algorithm across the benchmark instances, with a time limit of 180s per run. In the parameter study, the best value per metric across the three parameter settings is highlighted in bold for every instance.

5.6.1 Db-RRT vs ST-Db-RRT

We study the influence of the space-time planning by comparing Db-RRT with its space-time variant ST-Db-RRT. The comparison is split into two parts: first we look at the dynamic environments, where we expect the space-time extension to be beneficial, and then we look at the static environments, where we expect the space-time extension to be a disadvantage.

Since ST-Db-RRT is a bidirectional planner, we run Db-RRT also in its bidirectional variant with the default parameter of $p_{\text{forward}} = 0.5$ probability to expand the forward tree.

Dynamic Environments

In this Section we take a detailed look at the performance of Db-RRT and ST-Db-RRT in the dynamic environments. We use the selected parameter set (Section 5.5.5) for ST-Db-RRT and for Db-RRT we use the naive dynamic collision checks (see Section 4.3.1) to handle the dynamic environments. Both planners use the same amount of motion primitives, discontinuity bound and the same fixed-time trajectory optimization. Figure 5.17 shows the results. Db-RRT solves a majority of the dynamic environments with a success rate of $sr = 1.0$ and up to $42\times$ faster than ST-Db-RRT in the *Bugtrap* environment ($t_1 = 0.3s$ against $t_1 = 12.6s$). However, in the *Wait* environment, which requires the robot to wait for a long time in a confined space, Db-RRT fails entirely ($sr = 0.0$) while ST-Db-RRT solves it with $sr = 1.0$ for the 1st Order Unicycle and $sr = 0.1$ for the 2nd Order Unicycle. Also in the high dimensional state space of the Planar Rotor, Db-RRT fails more often in the more dynamic environments like *Double Windows* ($sr = 0.7$) or *Elevator* ($sr = 0.8$) while ST-Db-RRT solves it with $sr = 1.0$. Planning in space-time also pays off in the solution cost: ST-Db-RRT reaches a lower cost in 22 of the 25 instances. On the unicycle and car systems the advantage is moderate, typically between 4% and 17% with the largest gain in the 2nd Order Unicycle *Double Revolving Door* ($c = 18.7s$ against $c = 22.6s$). It

grows substantially for the Planar Rotor, where the cost is reduced by 12% to 46% and nearly halved in all moving-window instances, e.g. the *Double Window* with $c = 4.4s$ against $c = 8.1s$. Db-RRT achieves a notably lower cost only in the *Bugtrap* environment ($c = 27.9s$ against $c = 31.9s$).

Dynamics	Instance	Db-RRT			ST-Db-RRT		
		c [s]	t_1 [s]	sr [p]	c [s]	t_1 [s]	sr [p]
1st Order Unicycle	Empty	15.6	0.0	1.0	14.5	0.5	1.0
1st Order Unicycle	Revolving Door	11.6	0.1	1.0	10.6	0.6	1.0
1st Order Unicycle	Double Revolving Door	14.2	0.1	1.0	12.6	1.7	1.0
1st Order Unicycle	Multi Revolving Door	16.7	0.2	1.0	16.8	2.6	1.0
1st Order Unicycle	Double Moving Windows	11.5	0.1	1.0	10.9	2.8	1.0
1st Order Unicycle	Shortcut	14.8	0.1	1.0	14.0	0.9	1.0
1st Order Unicycle	Bugtrap	27.9	0.3	1.0	31.9	12.6	0.8
1st Order Unicycle	Wait	-	-	0.0	38.8	2.1	1.0
1st Order Unicycle	Elevator	17.8	26.5	0.9	16.6	1.1	0.3
2nd Order Unicycle	Empty	21.1	0.1	1.0	19.1	0.4	1.0
2nd Order Unicycle	Revolving Door	13.7	0.4	1.0	12.1	1.3	1.0
2nd Order Unicycle	Double Revolving Door	22.6	0.4	0.9	18.7	1.8	1.0
2nd Order Unicycle	Multi Revolving Door	24.7	0.6	1.0	22.3	2.4	1.0
2nd Order Unicycle	Double Moving Windows	16.5	0.2	1.0	16.3	2.9	1.0
2nd Order Unicycle	Wait	-	-	0.0	40.9	1.9	0.1
2nd Order Unicycle	Lane Switch	30.4	0.7	1.0	28.7	2.2	1.0
1st Order Car	Empty	14.6	0.1	1.0	14.0	0.8	1.0
1st Order Car	Lane Switch	29.2	0.3	1.0	27.8	1.4	1.0
1st Order Car	Park	7.8	0.2	1.0	6.7	0.1	1.0
1st Order Car	Parking Lot	24.6	0.4	1.0	26.2	2.2	1.0
Planar Rotor	Empty	2.7	2.2	1.0	1.9	2.4	1.0
Planar Rotor	Moving Window	5.7	4.8	0.9	3.2	13.7	1.0
Planar Rotor	Moving Small Window	6.7	31.1	0.8	3.7	16.6	1.0
Planar Rotor	Double Window	8.1	24.2	0.7	4.4	19.2	1.0
Planar Rotor	Elevator	16.3	13.8	0.8	14.3	27.0	1.0

Figure 5.17: Comparing Db-RRT with the new space-time variant ST-Db-RRT.

Static Environments

While the dynamic environment benchmark shows what planning in space-time gains, the static environments quantify what it costs when the environment carries no temporal structure. We compare both planners on the original static instances from *dynobench* [1] with the same parameter set as before. Since the environments are static, we can additionally include SST [22] as a classical kinodynamic sampling-based baseline. Figure 5.18 shows the results.

As expected, the advantage reverses and Db-RRT is the strongest planner across all metrics: it reaches the first solution fastest on every instance, within $t_1 = 0.4s$ on all unicycle and car instances, solves every instance reliably ($sr = 1.0$ except *Recovery Obstacles* with $sr = 0.8$) and achieves the best cost on 7 of the 10 instances.

ST-Db-RRT pays for the additional time dimension with a consistently later first solution, typically by an order of magnitude and up to $60\times$ in the 1st Order Unicycle *Bugtrap* ($t_1 = 12.1s$ against $t_1 = 0.2s$). Its solution costs remain competitive on the unicycle

instances, in the 1st Order Unicycle *Kink* it even attains the lowest cost ($c = 13.9s$ against $c = 14.1s$), but degrade for the 1st Order Car *Kink* ($c = 34.1s$ against $c = 18.8s$) and the Planar Rotor. The struggle concentrates in the *Bugtrap* instances: the success rate drops to $sr = 0.9$ and $sr = 0.6$ for the 1st and 2nd Order Unicycle, and for the Planar Rotor the average cost inflates to $c = 428.9s$ against $c = 6.6s$ for Db-RRT.

SST solves all unicycle and car instances reliably ($sr = 1.0$) with first solution times comparable to Db-RRT, but converges to substantially higher costs on the 2nd Order systems, e.g. $c = 53.4s$ against $c = 19.8s$ in the 2nd Order Unicycle *Kink*, and fails on the high dimensional Planar Rotor, where it solves only the *Empty* instance ($sr = 0.3$).

Dynamics	Instance	Db-RRT			ST-Db-RRT			SST		
		c [s]	t_1 [s]	sr [p]	c [s]	t_1 [s]	sr [p]	c [s]	t_1 [s]	sr [p]
1st Order Unicycle	Kink	14.1	0.1	1.0	13.9	2.1	1.0	16.7	0.1	1.0
1st Order Unicycle	Bugtrap	22.6	0.2	1.0	28.1	12.1	0.9	28.5	0.4	1.0
2nd Order Unicycle	Kink	19.8	0.2	1.0	20.4	2.5	1.0	53.4	1.8	1.0
2nd Order Unicycle	Bugtrap	32.5	0.4	1.0	38.8	8.8	0.6	59.9	3.6	1.0
1st Order Car	Kink	18.8	0.3	1.0	34.1	5.7	1.0	46.7	0.8	1.0
1st Order Car	Park	3.4	0.1	1.0	3.7	0.3	1.0	3.3	0.1	1.0
Planar Rotor	Empty	1.4	0.3	1.0	17.2	7.8	1.0	0.4	73.1	0.3
Planar Rotor	Bugtrap	6.6	5.5	1.0	428.9	61.0	1.0	-	-	0.0
Planar Rotor	Hole	6.6	13.1	1.0	65.1	28.6	0.8	-	-	0.0
Planar Rotor	Recovery Obstacles	5.7	4.5	0.8	15.8	37.5	0.7	-	-	0.0

Figure 5.18: Comparing Db-RRT with the new space-time variant ST-Db-RRT in static environments.

5.6.2 Space-Time Planner Comparison

Before turning to the quantitative comparison, Figure 5.19 shows three representative solution trajectories found by ST-Db-RRT, illustrating the kind of kinodynamically feasible space-time motions the planners produce across systems and environments. Figure 5.20 shows the exhaustive benchmark of all three new planners in all environments, using the parameter set selected in Section 5.5.5. ST-Db-RRT is generally the strongest planner in this comparison. It is the only planner that solves every instance, it reaches first solutions within seconds where the optimization-free planners need minutes, and its final solution cost stays close to the best observed value on every instance. The strengths of the two optimization-free planners are more localized: KIST solves most of the instances almost as reliably but consistently slower, while ST-GBRRT converges to the lowest costs on the simpler instances but fails on large parts of the benchmark.

This pattern is visible in the cost-success plots (Figure 5.21): ST-Db-RRT reaches its first solution fastest, while ST-GBRRT converges to slightly lower costs. On the higher-dimensional 2nd Order Unicycle Multi Revolving Door instance, ST-GBRRT does not reach a success rate of 100%. The cost-success plot in Figure 5.21 (bottom row) show two cases where ST-Db-RRT doesn't reach 100% success rate. In the bugtrap environment, we can see that if ST-Db-RRT finds a solution, it finds it very quickly, but only does so in $< 80\%$ of the runs. In the elevator environment, ST-Db-RRT finds a solution in only 30% of the runs, while KIST gets up to 60% and ST-GBRRT even up to 100%.

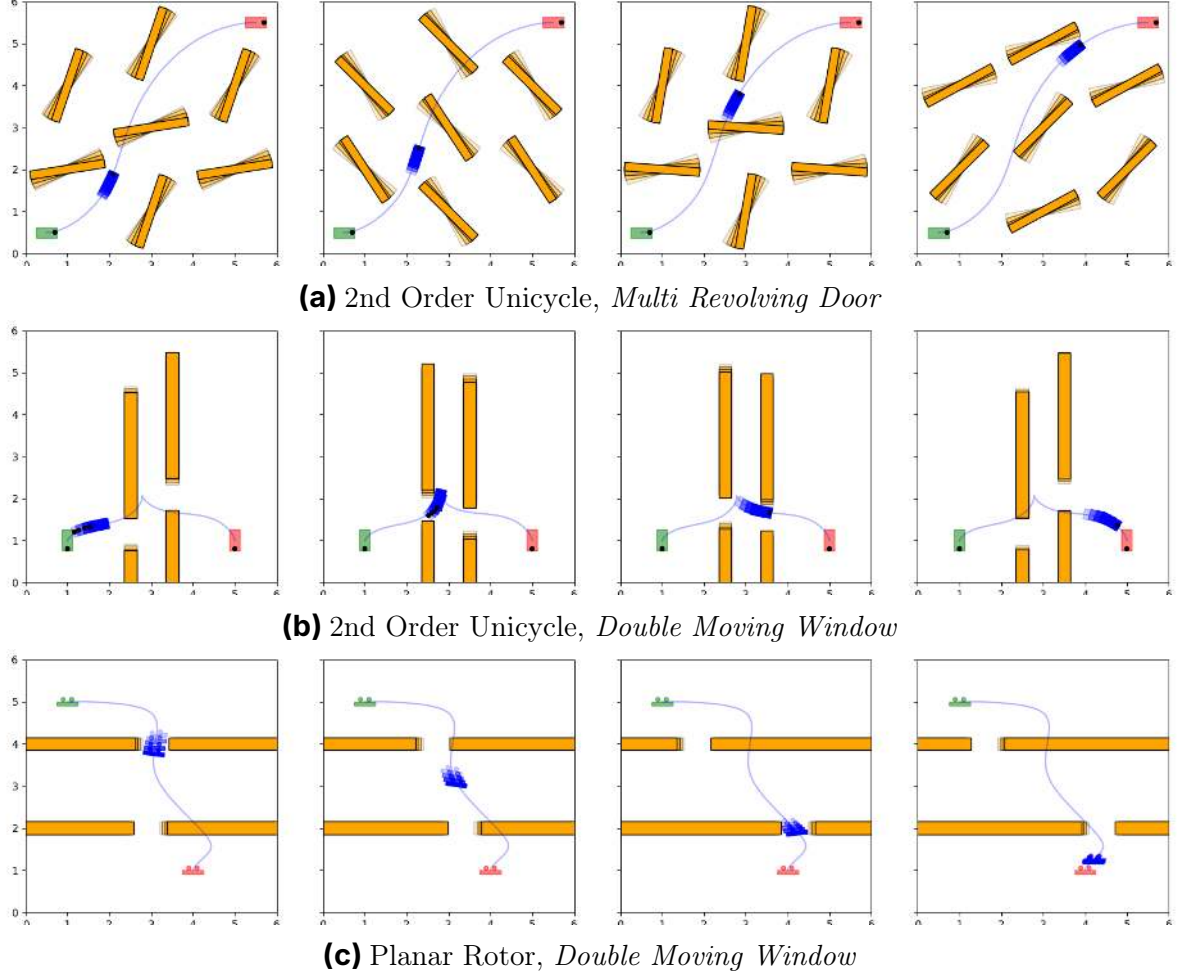


Figure 5.19: Snapshots of representative solution trajectories found by ST-Db-RRT for two different dynamical systems in 3 different dynamic environments.

Success Rate

ST-Db-RRT achieves 100% on 22 of the 25 instances and is the only planner that solves every instance at least once. Most notably, it is the only planner to find any solution in the 2nd Order Unicycle *Wait* instance ($sr = 0.1$), which forces the system to wait roughly 30s in a confined region. Its success rate is surpassed only twice: in the *Bugtrap* ($sr = 0.8$ against $sr = 1.0$ for KIST) and in the 1st Order Unicycle *Elevator* ($sr = 0.3$ against $sr = 0.6$ for KIST and $sr = 1.0$ for ST-GBRRT), confirming the recurring weakness of ST-Db-RRT in this instance already observed in the parameter benchmarks. KIST is similarly reliable on the unicycle and car systems ($sr = 1.0$ on 19 of the 25 instances), but its success rate declines on the temporally hardest problems: it drops to $sr = 0.6$ in the 1st Order Unicycle *Wait* and *Elevator*, fails the 2nd Order Unicycle *Wait* entirely, and decreases with the temporal difficulty of the Planar Rotor instances, down to $sr = 0.5$ in the *Elevator*. ST-GBRRT has the most limited coverage and fails entirely on 10 of the 25 instances. The failures concentrate on the long-horizon problems *Bugtrap*, *Wait*, *Lane Switch* and *Parking Lot*, and on the Planar Rotor, where only the *Empty* instance is solved ($sr = 0.2$). On the 2nd Order Unicycle its success rate additionally degrades in more dynamic environments, down to $sr = 0.1$ in the *Double Revolving Door*.

Dynamics	Instance	ST-Db-RRT			KIST			ST-GBRRT		
		c [s]	t_1 [s]	sr [p]	c [s]	t_1 [s]	sr [p]	c [s]	t_1 [s]	sr [p]
1st Order Unicycle	Empty	14.5	0.5	1.0	15.1	1.3	1.0	14.3	1.7	1.0
1st Order Unicycle	Revolving Door	10.6	0.6	1.0	10.7	2.0	1.0	10.5	2.5	1.0
1st Order Unicycle	Double Revolving Door	12.6	1.7	1.0	12.8	1.8	1.0	12.6	4.2	1.0
1st Order Unicycle	Multi Revolving Door	16.8	2.6	1.0	17.7	6.2	1.0	15.9	17.0	1.0
1st Order Unicycle	Double Moving Windows	10.9	2.8	1.0	11.2	2.1	1.0	10.5	6.8	1.0
1st Order Unicycle	Shortcut	14.0	0.9	1.0	14.6	2.3	1.0	14.0	3.9	1.0
1st Order Unicycle	Bugtrap	31.9	12.6	0.8	36.7	42.1	1.0	-	-	0.0
1st Order Unicycle	Wait	38.8	2.1	1.0	39.1	52.5	0.6	-	-	0.0
1st Order Unicycle	Elevator	16.6	1.1	0.3	16.7	4.5	0.6	16.5	8.2	1.0
2nd Order Unicycle	Empty	19.1	0.4	1.0	19.6	1.4	1.0	20.3	23.7	0.9
2nd Order Unicycle	Revolving Door	12.1	1.3	1.0	13.1	3.0	1.0	12.1	10.2	1.0
2nd Order Unicycle	Double Revolving Door	18.7	1.8	1.0	22.7	7.2	1.0	19.8	102.9	0.1
2nd Order Unicycle	Multi Revolving Door	22.3	2.4	1.0	33.9	6.7	1.0	21.6	71.7	0.7
2nd Order Unicycle	Double Moving Windows	16.3	2.9	1.0	24.9	5.3	1.0	15.6	99.5	0.6
2nd Order Unicycle	Wait	40.9	1.9	0.1	-	-	0.0	-	-	0.0
2nd Order Unicycle	Lane Switch	28.7	2.2	1.0	30.0	6.9	1.0	-	-	0.0
1st Order Car	Empty	14.0	0.8	1.0	14.0	0.4	1.0	13.3	7.0	1.0
1st Order Car	Lane Switch	27.8	1.4	1.0	29.2	55.9	1.0	-	-	0.0
1st Order Car	Park	6.7	0.1	1.0	7.7	0.4	1.0	6.9	0.6	1.0
1st Order Car	Parking Lot	26.2	2.2	1.0	25.0	14.2	1.0	-	-	0.0
Planar Rotor	Empty	1.9	2.4	1.0	2.6	2.5	1.0	2.4	67.6	0.2
Planar Rotor	Moving Window	3.2	13.7	1.0	4.0	22.1	1.0	-	-	0.0
Planar Rotor	Moving Small Window	3.7	16.6	1.0	5.6	28.8	0.9	-	-	0.0
Planar Rotor	Double Window	4.4	19.2	1.0	6.6	63.6	0.9	-	-	0.0
Planar Rotor	Elevator	14.3	27.0	1.0	16.2	125.2	0.5	-	-	0.0

Figure 5.20: Comparing all three new planners on all environments.

Time to First Solution

The clearest separation between the planners appears in the time to the first solution. ST-Db-RRT is the fastest planner on 23 of the 25 instances and stays below $t_1 = 3s$ on every unicycle and car instance except the *Bugtrap* ($t_1 = 12.6s$). Its t_1 is thereby largely insensitive to the temporal constraints that slow the optimization-free planners down by an order of magnitude: KIST needs $t_1 = 52.5s$ against $t_1 = 2.1s$ in the 1st Order Unicycle *Wait*, $t_1 = 42.1s$ against $t_1 = 12.6s$ in the *Bugtrap*, and $t_1 = 55.9s$ against $t_1 = 1.4s$ in the 1st Order Car *Lane Switch*, a $39\times$ speed-up for ST-Db-RRT. The Planar Rotor instances take longer for all planners, but the ordering persists, e.g. with $t_1 = 27.0s$ against $t_1 = 125.2s$ for KIST in the *Elevator*. KIST reaches a first solution faster than ST-Db-RRT only on two of the easiest instances (1st Order Car *Empty* and 1st Order Unicycle *Double Moving Windows*), both by less than a second. ST-GBRRT is the slowest planner throughout: on the 2nd Order Unicycle it takes $t_1 > 100s$ on instances that ST-Db-RRT solves within $3s$, which also explains its reduced success rates there.

Solution Cost

The solution cost discriminates the planners the least. On the 1st Order Unicycle and Car instances that all three planners solve, the average costs lie within a few percent of each other. Remarkably, ST-GBRRT converges to the lowest or shared lowest cost on 11 of the 15 instances it solves, despite reaching its first solution last. For the more demanding dynamics the ranking changes: ST-Db-RRT attains the best cost on all Planar Rotor instances, and on the 2nd Order Unicycle the best cost is always held by ST-Db-RRT or ST-GBRRT, while the cost of KIST inflates in the dynamically crowded instances.

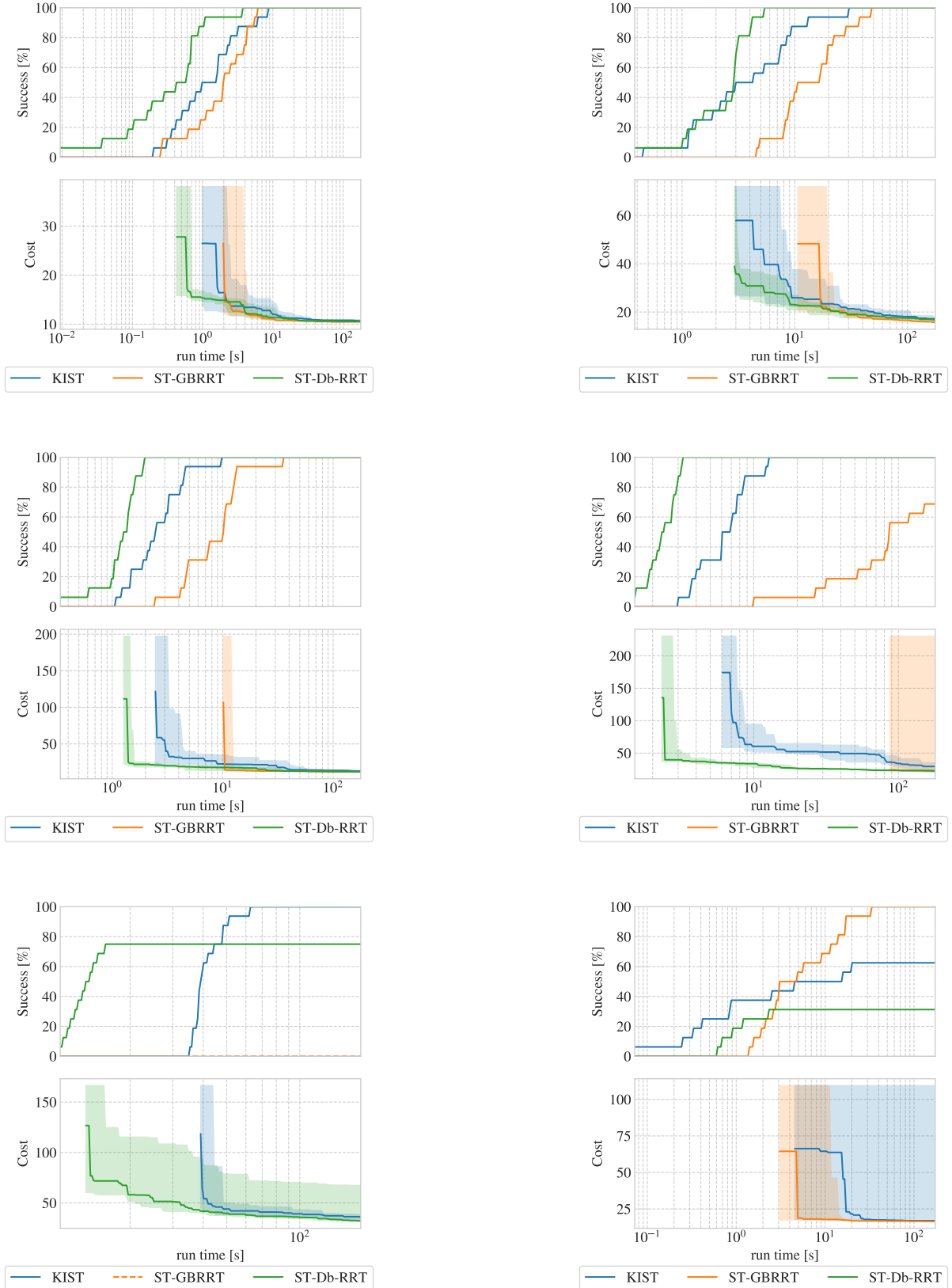


Figure 5.21: Cost-Success plots over 20 runs, showing average cost starting 50% solved instances. **Top Left:** 1st Order Unicycle: Revolving Door **Top Right:** 1st Order Unicycle: Multi Revolving Door **Middle Left:** 2nd Order Unicycle: Revolving Door **Middle Right:** 2nd Order Unicycle: Multi Revolving Door **Bottom Left:** 1st Order Unicycle: Bugtrap **Bottom Right:** 1st Order Unicycle: Elevator

It exceeds the cost of ST-Db-RRT by roughly 50% in the 2nd Order Unicycle *Multi Revolving Door* ($c = 33.9s$ against $c = 22.3s$) and *Double Moving Windows* ($c = 24.9s$ against $c = 16.3s$) as well as in the Planar Rotor *Moving Small Window* ($c = 5.6s$ against $c = 3.7s$). KIST attains the best cost on a single instance, the *Parking Lot* ($c = 25.0s$ against $c = 26.2s$ for ST-Db-RRT). ST-Db-RRT, in contrast, is never far off: its cost stays within 6% of the best observed value on every instance of the benchmark.

6. Discussion

This chapter interprets the benchmark results of Chapter 5. Following the structure of the evaluation, Section 6.1 discusses the parameter study and the mechanisms behind the observed trends, and Section 6.2 discusses the comparison of Db-RRT with ST-Db-RRT and of the three space-time planners against each other. Section 6.3 condenses the discussion into the key findings, followed by the limitations of the presented planners (Section 6.4) and directions for future work (Section 6.5).

6.1 Parameter Study

The parameter study (Section 5.5) varied three groups of parameters that all three planners share. This section discusses each group in turn and explains the mechanisms behind the observed trends, which motivate the common parameter set selected in Section 5.5.5.

6.1.1 Primitive Density and Discontinuity Bound

The benchmarks on parameter selection showed a clear result. Changing the amount of motion primitives and the discontinuity bound δ is a trade-off that is visible for all three planners: a denser set is a finer and more expressive library of maneuvers, but it also enlarges the number of candidate motions that are evaluated at every expansion.

This affects the performance of the planners and is reflected by the results in three different observations. First, planning with more primitives slows the planner down. On nearly every instance that a planner is able to solve with the coarse set, the time to first solution t_1 grows monotonically as the primitive amount rises, while the solution cost stays within a fraction of a second of its coarse value. The sole exception, KIST on the *Parking Lot*, is discussed below. The most extreme case is KIST on the *Planar Rotor Empty* environment, where t_1 increases from 1.8s to 24.1s for a cost decrease of only 0.5s.

Not finding any solution within the time limit can be another extreme consequence of a slower search speed caused by a fine density. This can be observed for KIST, which solves the *Planar Rotor Elevator* only at the low density setting, or for ST-GBRRT, which solves the *Double Revolving Door* only with the coarse set. Here, the refinement buys expressiveness that these instances do not need and pays for it in search speed, so the coarsest configuration remains preferable wherever it already suffices.

Secondly, more primitives might be needed to remain resolution complete, which is the second finding common to all planners: a minimum primitive density is required to cover the hardest instances. The *Wait* environment is unsolvable at the low setting for both ST-Db-RRT and KIST and becomes reliably solvable only from the medium setting onwards,

as it requires the systems to wait in a tight space with small maneuvers. For KIST the *Parking Lot* also improves from $sr = 0.8$ at low density to $sr = 1.0$ at medium and high density, with t_1 falling from 72.5s to 8.6s. This dependence on access to a sufficient number of primitives is strongest for the optimization-free KIST and ST-GBRRT, which must assemble a feasible solution directly from the primitives. KIST reaches the *Parking Lot* and *Wait* only once the set is dense enough while ST-GBRRT can not find any solution.

The third finding is the decisive difference between the planners in how strongly they depend on the number of primitives, and here another advantage of ST-Db-RRT becomes clear. A coarse primitive set is sufficient as it only needs to discover a δ -discontinuity-bounded solution, which is repaired with optimization, while KIST and ST-GBRRT need to find a feasible solution only with the available actions from the primitives. Consequently, ST-Db-RRT profits more from a coarse primitive set than the optimization-free algorithms KIST and ST-GBRRT. On the other hand, ST-Db-RRT struggles in the tightly constrained *Elevator* environment for the unicycle. It is the only instance where it cannot compete with optimization-free planners. In this highly dynamic and tight environment, a too loose δ -discontinuity bound leaves the optimizer unable to repair the discontinuities into a feasible trajectory. The effect does not appear in the corresponding Planar Rotor instance. One explanation is the smaller integration step Δt used for that system, which yields a finer trajectory discretisation and may make the discontinuities easier to repair; isolating this would require an ablation over Δt .

6.1.2 Sampling Time Bounds

The effect of the schedule is confined almost entirely to the success rate sr and the time to first solution t_1 , and does so mostly in temporally constrained instances, where reaching the goal depends on drawing samples within a particular arrival-time window, especially the environments *Elevator* and *Wait*. The three schedules show different strategies on 1) how quickly the upper time bound is advanced and 2) how densely each horizon is sampled before advancing. The aggressive schedule advances fastest, but samples each window sparsely, the conservative schedule samples densely but reaches distant horizons late, and the gradual schedule lies between the two. Although an already passed sampling time window keeps being sampled after the time bound moves on, it receives less sample probability than the window it is currently expanding. Which strategy pays off is dictated entirely by where the environment's feasible arrival window sits and how narrow it is. No single schedule dominates across planners and environments, which itself indicates that the schedule interacts strongly with both the planner's expansion strategy and the temporal structure of the instance. We expected two failure modes from this, which are partly reflected by the results.

First, when the feasible arrival time is much later than the initial estimate, the planner must reach that horizon quickly, and the aggressive schedule is superior. This becomes clear for KIST and ST-Db-RRT in the *Planar Rotor Elevator* environment, where the closed elevator blocks the goal state for an extended time and only the aggressive schedule solves it reliably for both planners. However, this effect is not fully validated in the same environment for the 1st Order Unicycle, where for KIST the success rate remains relatively constant and for ST-Db-RRT the conservative schedule even achieves the highest success rate. This could be explained with the different initial arrival time estimates of the Unicycle and the Planar Rotor. It also confirms the recurring struggle of ST-Db-RRT

in the *Elevator* environment, presumably because of the failing optimizer in the highly constrained space. However, while the success rate doesn't fully reflect the late arrival time issue for KIST, it can still be observed by the t_1 for the late arrival time environments *Wait*, *Parking Lot* and *Multi Revolving Door*. For these environments the t_1 is substantially later for the slowly exploring conservative schedule, as it takes more time to fill its large batch and advance to the arrival-time window.

The other failure mode we expected is related to the sample density: when the arrival time window is really short, dense sampling of that window matters, and the aggressive schedule risks advancing past it before covering it sufficiently. The *Wait* environment shows this for ST-Db-RRT: the gradual and conservative schedules solve it reliably, whereas the success rate of the aggressive schedule collapses, since success then hinges on drawing the right sample in one particular batch before the bound moves on. The very low t_1 of the aggressive schedule in its few successful runs shows that if it draws the right samples, it finds a solution substantially quicker. KIST does not confirm this for the same environment, showing an essentially constant success rate for all schedules.

6.1.3 Time Weight

We can see a clear preference for a small time weight λ across all algorithms. The distance metric influences the planner's behavior mainly in two places: the nearest-neighbor selection after sampling and the evaluation when selecting motion primitives.

In the nearest neighbor selection, a small time weight $\lambda = 0.2$ makes the planner prioritize proximity in the configuration space, rather than temporal neighbors, which could lead to a better uniform exploration of the configuration space. For ST-Db-RRT, this seems to be especially beneficial in highly spatially constrained environments like *Elevator*, *Wait* and *Multi Revolving Door*. However, we can observe the opposite effect for KIST in the 1st Order *Wait* environment, where a high time weight finds solutions within half the time of the lower time weights. Instead of mostly random expansion, like ST-Db-RRT, KIST invests computation into selecting motions that come closest to the random sample. Preferring time proximity for a more uniform time exploration could help in the tightly time-constrained *Wait* environment to find the right motion to traverse the short open time window.

Secondly, a lower time weight is generally better when evaluating motion primitives in the expand or connect step, as the configuration-space distance loses meaningful precision with a high time weight. The connect step (Alg. 6) evaluates candidate motion primitives by computing the distance to the target state. With a high time weight it will be drawn to primitives reducing mostly the time distance, but not necessarily configuration space distance. This could result in selecting primitives that do not get closer in the configuration space, slowing the approach toward the other tree.

6.2 Planner Comparison

The comparative benchmarks (Section 5.6) are discussed in two steps that follow the evaluation: first the effect of the space-time state space in isolation, and then the differences between the three space-time planners.

6.2.1 Db-RRT vs ST-Db-RRT

The direct comparison between Db-RRT and ST-Db-RRT (Section 5.6.1) isolates the effect of the space-time state space: both planners share the same motion primitives, discontinuity bound and trajectory optimization, and differ only in whether time is part of the state. The dynamic and static benchmarks show the two sides of this single design change, what the time dimension buys when the environment moves, and what it costs when it does not.

In the dynamic environments the benefit concentrates in success rate and solution cost. The complete failure of Db-RRT in the *Wait* environment is the incompleteness analysed in Section 4.3.1. Without time in the state, duplicate pruning suppresses branches that revisit a configuration at a later time, so the required waiting in place cannot be represented. The environment was designed to trigger exactly this failure mode and the benchmark confirms it. The same mechanism explains, in a milder form, the reduced success rates and higher costs in the strongly dynamic instances: Db-RRT commits to the first arrival time with which its search reaches a region, while ST-Db-RRT distinguishes states that differ in time and explores paths at different time windows. Accordingly, the cost gap scales with how strongly the achievable duration depends on the timing of the passage. Notably, wherever its naive time handling suffices, Db-RRT still reaches the first solution substantially earlier. The speed cost of the additional dimension is thus present in dynamic environments as well, but ST-Db-RRT wins because it converts this cost into completeness and solution quality.

The static benchmark exposes the same cost without any compensating benefit. In a static environment, time carries no information, so the additional dimension only enlarges the space to be covered and constrains the search with time consistency, which is reflected in first solutions that arrive an order of magnitude later across all instances. Beyond this constant overhead, the results deteriorate where the initial arrival-time estimate is poor. The sampling time bounds are initialized from the lower-bound time of the direct distance to the goal, which in the *Bugtrap* instances drastically underestimates the duration of the trajectory leading out of the trap. The sampling schedule must therefore expand the time bounds far before feasible arrival times are drawn at all, which delays the first solution and biases it towards unnecessarily late arrivals, culminating in the Planar Rotor *Bugtrap* cost of $c = 428.9s$ against $c = 6.6s$. In the *Kink* instances, in contrast, the direct estimate is nearly correct and the unicycle costs of both planners essentially coincide, reducing the overhead to a constant factor in t_1 . This sensitivity to the initial estimate is not specific to static environments, but static instances with long detours expose it most clearly, making a more informed arrival-time estimate a natural direction for future work.

6.2.2 Space-Time Planner Comparison

The comparing benchmark (Section 5.6.2) produced three main results: ST-Db-RRT solves every instance and reaches first solutions dramatically earlier, KIST is nearly as reliable but consistently slower and loses solution quality in crowded dynamic scenes, and ST-GBRRT converges to the best costs on the simpler instances while failing on large parts of the benchmark. As all planner use the same space-time sampling strategy, the observations can be traced back to two design decision: how much work a planner invests during the search into 1) selecting a motion primitive and 2) producing a kinodynamically feasible trajectory.

Concretely, ST-Db-RRT selects primitives randomly, except in its connect step and defers feasibility entirely to the trajectory optimization, resulting in two rapidly exploring δ -discontinuity-bounded trees. KIST and ST-GBRRT, on the other hand, evaluate and select the best motion primitive from a set of candidates and maintain a heuristic guidance for the forward tree, KIST only in a separate completion step, ST-GBRRT in every iteration. Both pay for feasibility during the search, growing their forward trees with exact action rollouts. The three planners thus form a spectrum of increasing search effort per iteration, and the benchmark shows the consequences of this spectrum under a fixed time budget.

The first reason for the large gap in the time to the first solution is the cost of the primitive selection itself. An expansion of ST-Db-RRT selects a random applicable motion primitive, shifts it to the tree node and collision-checks the resulting states. KIST and ST-GBRRT instead invest into a more focused expansion (Section 4.4): they compute the landing states of all applicable primitives, sort them by their distance to the random sample and expand only with the best candidate. ST-GBRRT additionally updates the heuristic of nearby forward nodes after every reverse expansion and maintains its priority queue. Every one of their expansions is therefore better informed, but in the same planning time ST-Db-RRT completes many random expansions, and the benchmark indicates that for reaching a first solution the higher sampling throughput outweighs the higher expansion quality.

The second reason is how the planners achieve feasibility. KIST and ST-GBRRT maintain it in every iteration: the actions of the selected primitive are rolled out step by step with the system dynamics to keep the forward tree feasible, and a rollout that collides discards the invested work. ST-Db-RRT never integrates the dynamics during the search. Both of its trees grow with shifted, δ -discontinuity-bounded primitives and feasibility is established once per solution, by a single DDP solve that is warm-started with the near-feasible space-time trajectory and converges quickly (Section 14). The cost of feasibility is thereby outsourced into one repair step at the end instead of being paid in every expansion, which is substantially cheaper as long as the repair succeeds, to which we return below.

The higher sampling advantage resulting from both effects compounds in the environments with difficult temporal structure. The space-time sampling schedule advances the upper time bound after a certain amount of samples, so the planners runtime needed to reach a late feasible arrival window scales with the per-iteration cost. In environments whose feasible windows are late or narrow, such as *Wait*, KIST and ST-GBRRT take much longer advancing the sampling horizon towards the feasible window, which ST-Db-RRT reaches much faster. Additionally, the fast sampling could allow ST-Db-RRT to work with bigger batch sizes to achieve higher sampling densities, increasing the probability of finding the right sample in narrow time windows. Together, this explains why t_1 is comparable for all planners in the *Empty* environments but drifts apart by up to a factor of 39 in the temporally constrained ones.

The success rates mirror the same mechanics at the 180s time limit. The failures of ST-GBRRT appear to be caused by the time limit rather than by an algorithmic incapacity: in environments it solves only partially, its t_1 already approaches the limit (e.g. $t_1 = 102.9s$ in the 2nd Order Unicycle *Double Revolving Door*), and the long-horizon problems *Bugtrap*, *Wait*, *Lane Switch* and *Parking Lot* require fast exploration. Confirming this interpretation, however, would require runs with a substantially larger time budget. KIST occupies the middle ground because its greedy connection step and completion

policy can result in fast solutions, sparing it the slow guided growth over the full horizon. The notable exception inverts the picture: in the 1st Order Unicycle *Elevator*, the design decision of ST-Db-RRT to defer feasibility to the optimizer does not pay off. The optimizer cannot reliably repair the δ -discontinuities inside the tight corridor formed by the moving elevator, so found discontinuity-bounded solutions are discarded. KIST only achieves $sr = 0.6$, presumably because of a similar issue: its greedy Heuristic Completion frequently fails, and restarting it at every new tree connection is costly, while ST-GBRRT solves the same instance with $sr = 1.0$ profiting from the maintained priority queue and gradual expansion of the forward tree. The other exception is the *Bugtrap* environment, where ST-Db-RRT either finds solutions relatively quickly within the first 20s or fails entirely. Because the initial arrival time estimate in this environment is far from the actual arrival time, the amount drawn samples from the sampling schedule that are actual valid might be small. ST-Db-RRT seems to struggle to connect the trees with its random expansions, while KIST solves the environment more reliably with its more directed expansions.

The two environments in which ST-Db-RRT is outperformed by the other planners highlights the regime in which best-input expansion and optimization-free planning remains valuable: where the repair step is fragile, investing into feasibility during the search pays off.

Where all planners succeed on the simpler systems, the costs lie within a few percent of each other, confirming once more that the achievable cost is largely fixed by the environment. That ST-Db-RRT stays within 6% is not a consequence of the trajectory optimization, which in our setting only repairs discontinuities but does not improve arrival time. Instead, it results entirely from higher sample density combined with its early first solutions, which leave almost the entire time budget for finding improved solutions within upper cost bounds. The optimization-free planners follow the same mechanism more slowly, achieving lower sample density within the budget. For ST-GBRRT this works remarkably well where its search is fast enough: it attains the lowest or shared lowest cost on 11 of the 15 instances it solves, so solution quality is not its weakness, convergence speed is. KIST suffers from the same time budget problem while its greedy approach finds solution faster, but may neglect other low cost nodes, which explains the roughly 50% higher costs on the 2nd Order Unicycle *Multi Revolving Door* and *Double Moving Windows*.

6.3 Key Findings

The preceding discussion condenses into the following key findings:

- **Space-time planning is necessary for completeness in dynamic environments** Db-RRT fails on the *Wait* environment, constructed from Remark 1 (Section 4.3.1) and confirms the incompleteness in dynamic environments, while its space-time variant solves it. In static environments, however, the additional dimension costs roughly an order of magnitude in time to first solution without a compensating benefit. The space-time extension is therefore best treated as an optional, more expensive mode, to be enabled when the temporal structure of the environment requires it.

- **Space-time planning achieves lower costs in dynamic environments** ST-Db-RRT achieves lower costs than Db-RRT in almost all dynamic environments. Exploring different times for the same configuration allows ST-Db-RRT to find better solutions, while Db-RRT commits to the first arrival time it reaches a configuration and is therefore more likely to miss better solutions. In static environments, the additional time dimension does not carry any information and only slows the search down, resulting in higher costs for ST-Db-RRT.
- **Deferring feasibility wins under a time budget.** The combination of cheap, discontinuity-bounded search in space-time with a single optimization-based repair (ST-Db-RRT) dominates the benchmark in coverage, time to first solution and cost.
- **Optimization-free planning remains valuable where the repair step is fragile.** In the tight and highly dynamic *Elevator* corridor the optimizer often fails to repair the discontinuities, and the planners KIST and ST-GBRRT outperform ST-Db-RRT.
- **The primitive set should be kept as coarse as possible.** Additional primitives only pay off when the coarse library results in incompleteness. ST-Db-RRT is the least sensitive to a sparse set because its optimization step repairs the remaining δ -discontinuities.
- **No time bound schedule dominates.** Which schedule pays off is dictated by where the feasible arrival window lies and how narrow it is: the aggressive schedule wins where the window is late or distant, the slower schedules where it is early or must be sampled densely.
- **A small time weight is the robust default.** Prioritizing configuration-space over temporal proximity ($\lambda = 0.2$) is preferable in both the nearest-neighbor and the primitive selection. Only strongly time-constrained instances occasionally reward a larger λ .

6.4 Limitations

Although combining the approaches of the original algorithms made the planners capable of solving new types of problems, not all properties of the original planners were preserved. ST-Db-RRT is resolution complete only if a time bound is given (Section 14), as it otherwise cannot report that no solution exists with the current primitive set. KIST and ST-GBRRT circumvent this problem only with a theoretical trick: the random forward propagation step in their planning lifts them out of resolution completeness into probabilistic completeness. Practically, however, the time to find a solution with this single propagation step alone is prohibitive. The loss of resolution completeness also has a direct practical consequence: the iterative approach of iDb-RRT, which increases the resolution until it is sufficient, is no longer directly applicable so the primitive amount and the discontinuity bound or a upper timebound must be set manually. This feeds into the second limitation, the high parameter dependency and sensitivity of all planners. Beyond the primitive set and the discontinuity bound, their performance strongly depends on the time bound parameters of the space-time sampling, as seen in the results of the time bound benchmark (Section 5.5.3).

Furthermore, none of the presented planners has an optimality guarantee. Although ST-RRT* achieves asymptotic optimality through its rewire operation, we were unable to find a feasible formulation of rewiring in space-time.

Lastly, the additional time dimension enlarges the state space and thereby further limits the ability of the planners to scale to high-dimensional systems.

6.5 Future Research Directions

There are a few promising directions for future work that could improve the performance of the planners and address some of their limitations. First, a better heuristic for the initial time bound estimate could improve the performance of the planners in environments with late feasible arrival windows. The current estimate lower-bounds the time by the direct distance to the goal, which can be a poor approximation in environments with long detours. A more informed estimate could help the planners reach the feasible arrival window faster and improve their success rates.

Second, kinodynamic rewiring in space-time could be explored for ST-Db-RRT to achieve asymptotic optimality. The discontinuity-bound can be leveraged to avoid solving BVPs for connecting nodes, but rewiring a node in space-time means a time shift of the entire subtree, which requires a complete re-evaluation of the subtree's feasibility. This is a non-trivial problem that could be addressed in future work.

In this work we only used fixed-time trajectory optimization (Section 14). Investigating variable-time optimization in dynamic environments could improve the solution quality and allow the planners to better adapt to different time constraints. [1] already explores different optimization strategies for the iDb-A* algorithm, which could be adapted to ST-Db-RRT.

7. Conclusion

In this thesis, we present, analyze and evaluate three new kinodynamic motion planners that plan in space-time for dynamic environments. The planners ST-Db-RRT, ST-GBRRT and KIST combine the space-time sampling strategy of ST-RRT* with the kinodynamic expansion strategies of iDb-RRT and GBRRT, enabling them to plan kinodynamically feasible trajectories through dynamic environments without requiring a steering function or a known arrival time. As all three share the same sampling strategy, they allowed us to study how the established kinodynamic search paradigms perform when lifted into space-time.

The theoretical analysis shows that the completeness properties of the underlying planners are not preserved automatically: a naive application of Db-RRT to dynamic environments is incomplete, which the benchmarks confirm. Lifting the planners into space-time restores completeness, with ST-Db-RRT being resolution complete once a time bound is given, while KIST and ST-GBRRT retain probabilistic completeness. In static environments, however, the additional dimension delays the first solution by roughly an order of magnitude, making space-time planning a more expensive variant to be enabled when the temporal structure of the environment demands it.

A parameter focused benchmark shows that all three planners are sensitive to their central parameters. The motion primitive set should be kept as coarse as the environments permit, a small time weight in the distance metric is the robust default, and the best time bound schedule depends on where the feasible arrival window lies and how narrow it is.

The benchmark comparing the planners across four dynamical systems shows that they reach similar solution costs where they all succeed, but differ strongly in success rate and runtime. ST-Db-RRT is the only planner to solve every dynamic instance, reaches the first solution up to $39\times$ faster than the optimization-free planners, and stays within 6% of the best observed cost, demonstrating that deferring feasibility to a single repair step wins under a fixed time budget. ST-GBRRT converges to the lowest costs on the instances its slower search can cover, and KIST occupies the middle ground. Only the tight and highly dynamic *Elevator* environment inverts this picture, revealing that optimization-free planning remains valuable when the optimization struggles.

The planners leave clear room for improvement. As outlined in Section 6.5, a more informed arrival-time estimate could reach late feasible arrival windows more reliably, kinodynamic rewiring in space-time could recover the asymptotic optimality of ST-RRT*, and variable-time trajectory optimization could further improve solution quality. However, we believe that by developing ST-Db-RRT, ST-GBRRT and KIST, the first sampling-based planners for general kinodynamic systems that plan through space-time without requiring a steering function or a known arrival time, this thesis takes a step towards general-purpose motion planning in dynamic environments, and towards robots that plan not only *where* to move, but also *when*.

Bibliography

- [1] J. Ortiz-Haro, W. Hönig, V. N. Hartmann, and M. Toussaint, “iDb-A*: Iterative search and optimization for optimal kinodynamic motion planning,” *IEEE Transactions on Robotics*, vol. 41, pp. 2031–2049, 2025. DOI: 10.1109/TR0.2024.3502505.
- [2] W. Hönig, J. Ortiz-Haro, and M. Toussaint, “db-A*: Discontinuity-bounded search for kinodynamic mobile robot motion planning,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 13 540–13 547. DOI: 10.1109/IROS47612.2022.9981577.
- [3] J. Ortiz-Haro, W. Hönig, V. N. Hartmann, M. Toussaint, and L. Righetti, “iDb-RRT: Sampling-based kinodynamic motion planning with motion primitives and trajectory optimization,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 10 702–10 709. DOI: 10.1109/IROS58592.2024.10802168.
- [4] F. Grothe, V. N. Hartmann, A. Orthey, and M. Toussaint, “ST-RRT*: Asymptotically-optimal bidirectional motion planning through space-time,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 3314–3320. DOI: 10.1109/ICRA46639.2022.9811814.
- [5] W. Guo et al., *The Open Motion Planning Library 2.0*, 2026. arXiv: 2605.29301 [cs.R0]. [Online]. Available: <https://arxiv.org/abs/2605.29301>.
- [6] S. LaValle, *Planning Algorithms*. Cambridge University Press, 2006, ISBN: 9781139455176. [Online]. Available: <https://books.google.de/books?id=PwLBAAAQBAJ>.
- [7] T. Fraichard, “Trajectory planning in a dynamic workspace: A ‘state-time space’ approach,” *Advanced Robotics*, vol. 13, no. 1, pp. 75–94, 1998. DOI: 10.1163/156855399X00928.
- [8] S. Nayak and M. W. Otte, “Bidirectional sampling-based motion planning without two-point boundary value solution,” *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3636–3654, 2022. DOI: 10.1109/TR0.2022.3181947.
- [9] J. Bruce and M. Veloso, “Real-time randomized path planning for robot navigation,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, vol. 3, 2002, pp. 2383–2388. DOI: 10.1109/IRDS.2002.1041624.
- [10] K. Naderi, J. Rajamäki, and P. Hämäläinen, “RT-RRT*: A real-time path planning algorithm based on rrt*,” in *Proceedings of the 8th ACM SIGGRAPH Conference on Motion in Games (MIG)*, 2015, pp. 113–118. DOI: 10.1145/2822013.2822036.
- [11] M. Otte and E. Frazzoli, “RRTX: Asymptotically optimal single-query sampling-based motion planning with quick replanning,” *The International Journal of Robotics Research*, vol. 35, no. 7, pp. 797–822, 2016. DOI: 10.1177/0278364915594679.

-
- [12] K. Kant and S. W. Zucker, “Toward efficient trajectory planning: The path-velocity decomposition,” *The International Journal of Robotics Research*, vol. 5, no. 3, pp. 72–89, 1986. DOI: 10.1177/027836498600500304.
 - [13] A. Sintov and A. Shapiro, “Time-based rrt algorithm for rendezvous planning of two dynamic systems,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 6745–6750. DOI: 10.1109/ICRA.2014.6907855.
 - [14] M. Phillips and M. Likhachev, “SIPP: Safe interval path planning for dynamic environments,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2011, pp. 5628–5635. DOI: 10.1109/ICRA.2011.5980306.
 - [15] M. Hüppi, L. Bartolomei, R. Mascaro, and M. Chli, “T-PRM: Temporal probabilistic roadmap for path planning in dynamic environments,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022. DOI: 10.1109/IROS47612.2022.9981739.
 - [16] N. Kerimov, A. Onegin, and K. Yakovlev, “Safe interval randomized path planning for manipulators,” in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, vol. 35, 2025, pp. 213–217. DOI: 10.1609/icaps.v35i1.36120.
 - [17] J. Sim, J. Kim, and C. Nam, *Safe interval rrt* for scalable multi-robot path planning in continuous space*, 2024. arXiv: 2404.01752 [cs.R0].
 - [18] M. Pivtoraiko and A. Kelly, “Kinodynamic motion planning with state lattice motion primitives,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2011, pp. 2172–2179. DOI: 10.1109/IROS.2011.6094900.
 - [19] S. LaValle and J. Kuffner, “Randomized kinodynamic planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, vol. 1, 1999, pp. 473–479. DOI: 10.1109/ROBOT.1999.770022.
 - [20] M. Kleinbort, K. Solovey, Z. Littlefield, K. E. Bekris, and D. Halperin, “Probabilistic completeness of rrt for geometric and kinodynamic planning with forward propagation,” *IEEE Robotics and Automation Letters*, vol. 4, no. 2, 2019. DOI: 10.1109/LRA.2018.2888947.
 - [21] K. Hauser and Y. Zhou, “Asymptotically optimal planning by feasible kinodynamic planning in a state–cost space,” *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1431–1443, 2016. DOI: 10.1109/TR0.2016.2602363.
 - [22] Y. Li, Z. Littlefield, and K. E. Bekris, “Sparse methods for efficient asymptotically optimal kinodynamic planning,” in *Algorithmic Foundations of Robotics XI: Selected Contributions of the Eleventh International Workshop on the Algorithmic Foundations of Robotics*, H. L. Akin, N. M. Amato, V. Isler, and A. F. van der Stappen, Eds. Cham: Springer International Publishing, 2015, pp. 263–282, ISBN: 978-3-319-16595-0. DOI: 10.1007/978-3-319-16595-0_16.
 - [23] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011. DOI: 10.1177/0278364911406761.
 - [24] D. J. Webb and J. van den Berg, “Kinodynamic RRT*: Asymptotically optimal motion planning for robots with linear dynamics,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2013, pp. 5054–5061. DOI: 10.1109/ICRA.2013.6631299.
 - [25] J. Kuffner and S. LaValle, “RRT-Connect: An efficient approach to single-query path planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, vol. 2, 2000, pp. 995–1001. DOI: 10.1109/ROBOT.2000.844730.
-

-
- [26] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, “Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2014, pp. 2997–3004. DOI: 10.1109/IROS.2014.6942976.
 - [27] M. P. Strub and J. D. Gammell, “Adaptively informed trees (AIT*): Fast asymptotically optimal path planning through adaptive heuristics,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 3191–3198. DOI: 10.1109/ICRA40945.2020.9197338.
 - [28] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, “CHOMP: Gradient optimization techniques for efficient motion planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2009, pp. 489–494. DOI: 10.1109/ROBOT.2009.5152817.
 - [29] J. Schulman et al., “Motion planning with sequential convex optimization and convex collision checking,” *The International Journal of Robotics Research*, vol. 33, pp. 1251–1270, 2014. DOI: 10.1177/0278364914528132.
 - [30] A. Byravan, B. Boots, S. S. Srinivasa, and D. Fox, “Space-time functional gradient optimization for motion planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 6499–6506. DOI: 10.1109/ICRA.2014.6907818.
 - [31] D. Hsu, R. Kindel, J.-C. Latombe, and S. Rock, “Randomized kinodynamic motion planning with moving obstacles,” *The International Journal of Robotics Research*, vol. 21, no. 3, pp. 233–255, Mar. 2002. DOI: 10.1177/027836402320556421.
 - [32] C. Chen, M. Rickert, and A. Knoll, “Kinodynamic motion planning with space-time exploration guided heuristic search for car-like robots in dynamic environments,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015, pp. 2666–2671. DOI: 10.1109/IROS.2015.7353741.
 - [33] Z. A. Ali and K. Yakovlev, “Safe interval path planning with kinodynamic constraints,” in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, vol. 37, 2023, pp. 12 330–12 337. DOI: 10.1609/aaai.v37i10.26453.
 - [34] S. Huang, Y. Wu, Y. Tao, and V. Kumar, “Safe interval motion planning for quadrotors in dynamic environments,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 2780–2786. DOI: 10.1109/ICRA55743.2025.11127946.
 - [35] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968. DOI: 10.1109/TSSC.1968.300136.
 - [36] L. Janson, B. Ichter, and M. Pavone, “Deterministic sampling-based motion planning: Optimality, complexity, and performance,” *The International Journal of Robotics Research*, vol. 37, no. 1, pp. 46–61, 2017. DOI: 10.1177/0278364917714338.
 - [37] J. H. Friedman, J. L. Bentley, and R. A. Finkel, “An algorithm for finding best matches in logarithmic expected time,” *ACM Trans. Math. Softw.*, vol. 3, no. 3, pp. 209–226, Sep. 1977, ISSN: 0098-3500. DOI: 10.1145/355744.355745.
 - [38] M. Moll, C. Chamzas, Z. Kingston, and L. E. Kavraki, “Hyperplan: A framework for motion planning algorithm selection and parameter optimization,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 2511–2518. DOI: 10.1109/IROS51168.2021.9636651.
-

Appendices

A. Planner Comparison: Cost - Success Plots

In this appendix, we present all cost-success plots for the benchmarks of Section 5.6.1 that compares Db-RRT with its space-time variant ST-Db-RRT and the benchmark plots of Section 5.6.2 comparing ST-Db-RRT, KIST and ST-GBRRT. Finally we present a selection of solution trajectories for the 2nd Order Unicycle and Planar Rotor systems in dynamic environments.

A.1 Success - Plots: Db - RRT vs. ST - Db - RRT

A.2 Success - Plots: ST - Db - RRT vs. KIST vs. ST - GBRRT

A.3 Trajectory Plots

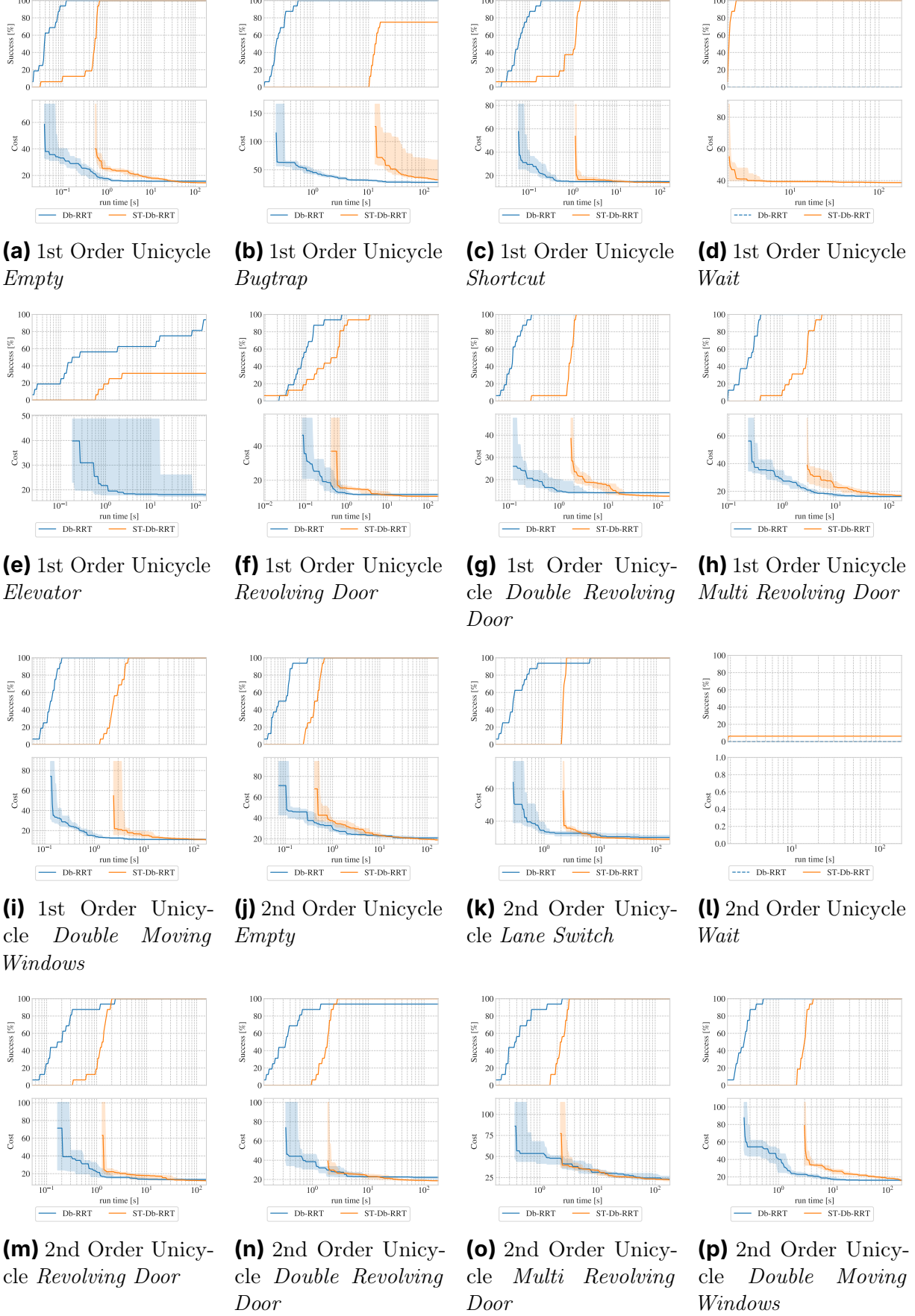


Figure A.1: Cost-success plots of Db-RRT and ST-Db-RRT for the 1st and 2nd Order Unicycle systems.

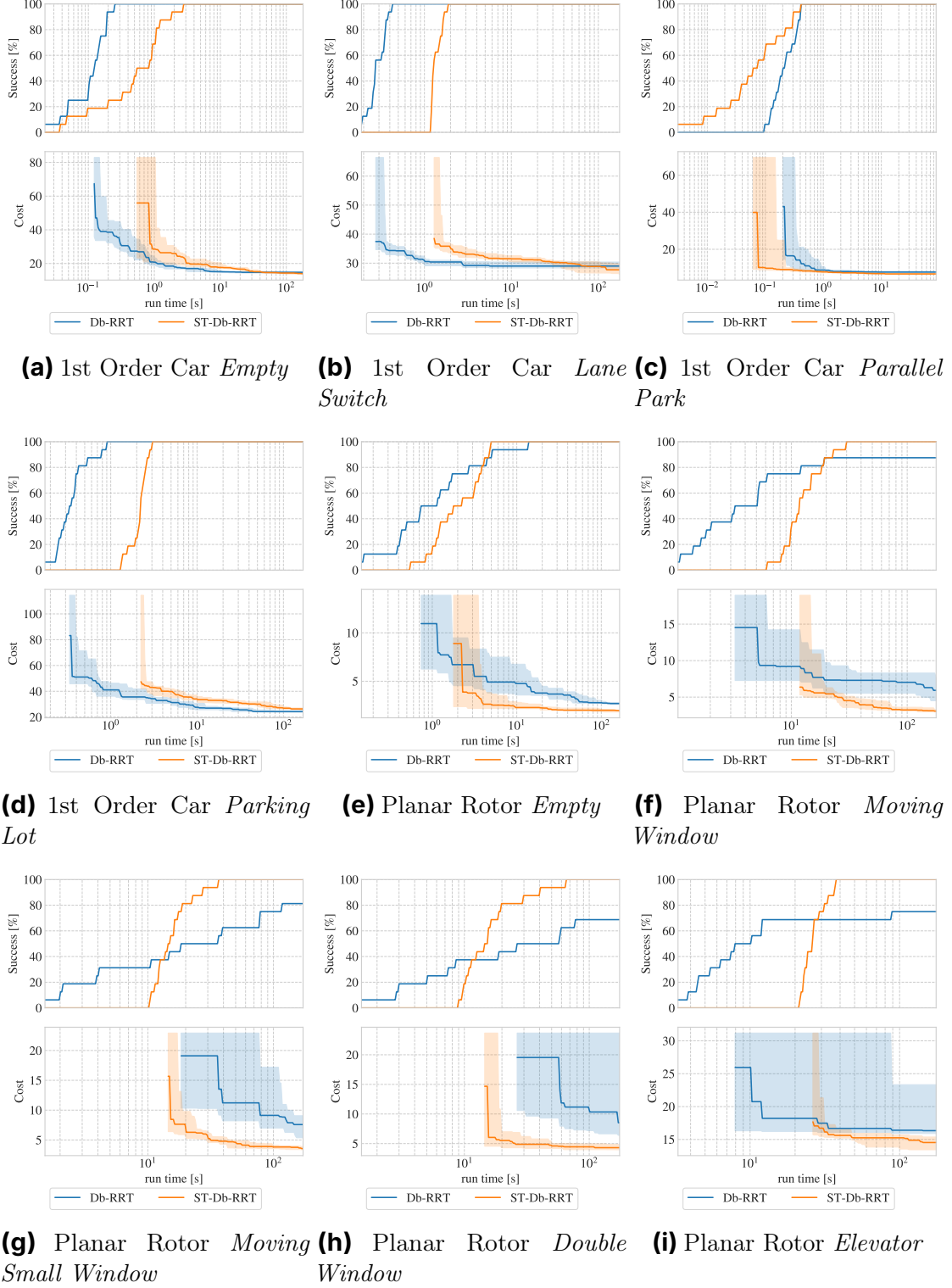


Figure A.2: Cost-success plots of Db-RRT and ST-Db-RRT for the 1st Order Car and Planar Rotor systems.

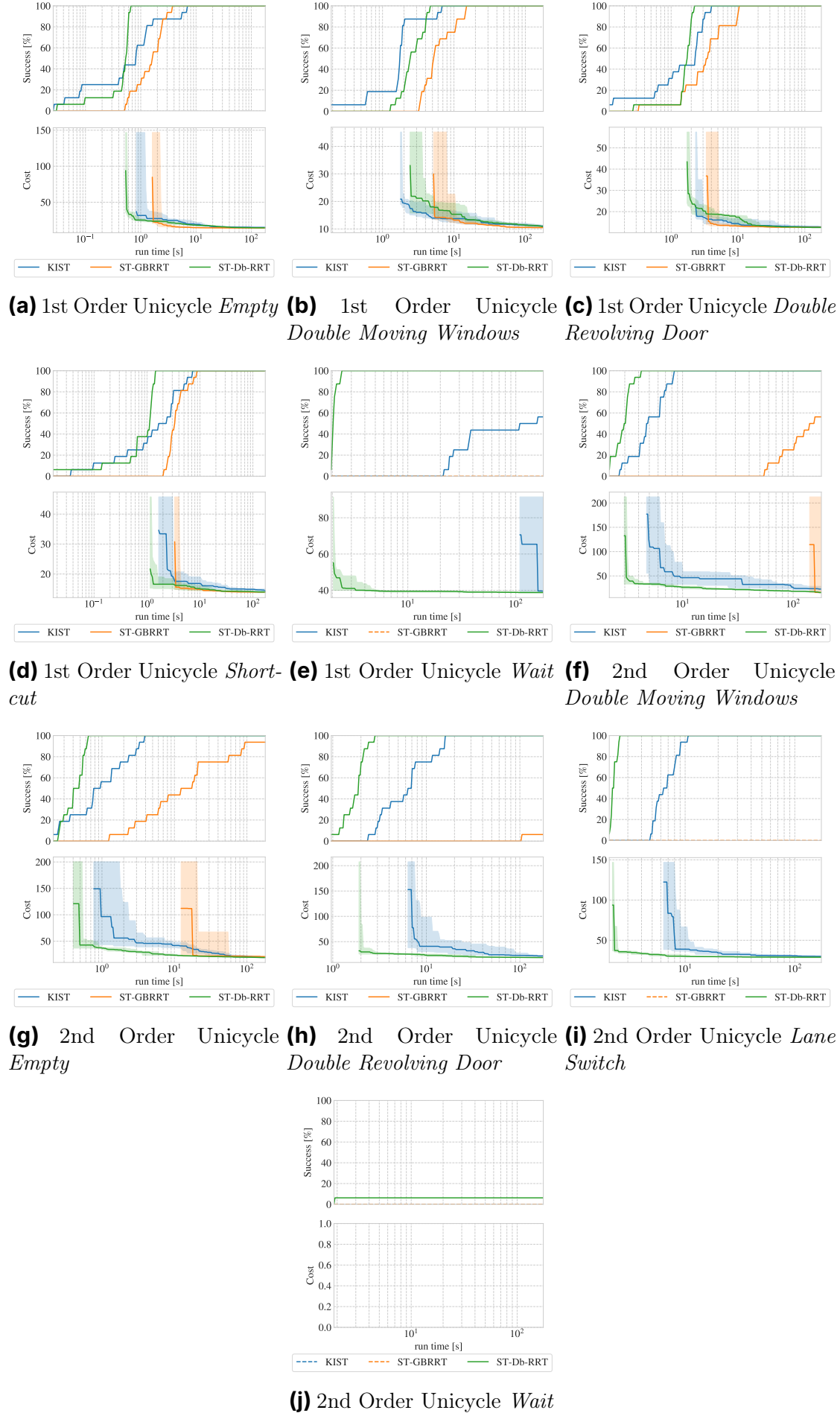


Figure A.3: Cost-success plots for the 1st and 2nd Order Unicycle systems.

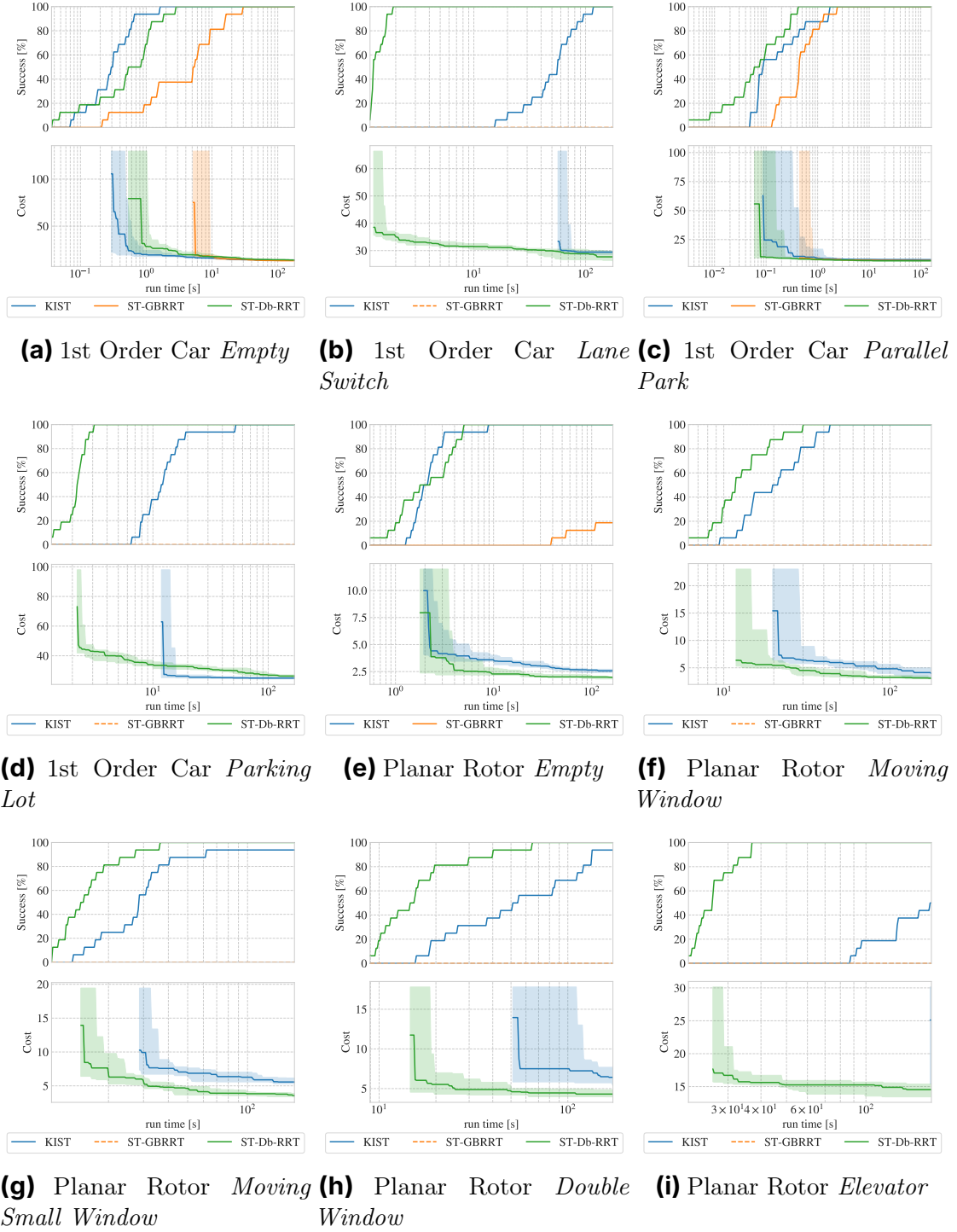


Figure A.4: Cost-success plots for the 1st Order Car and Planar Rotor systems.

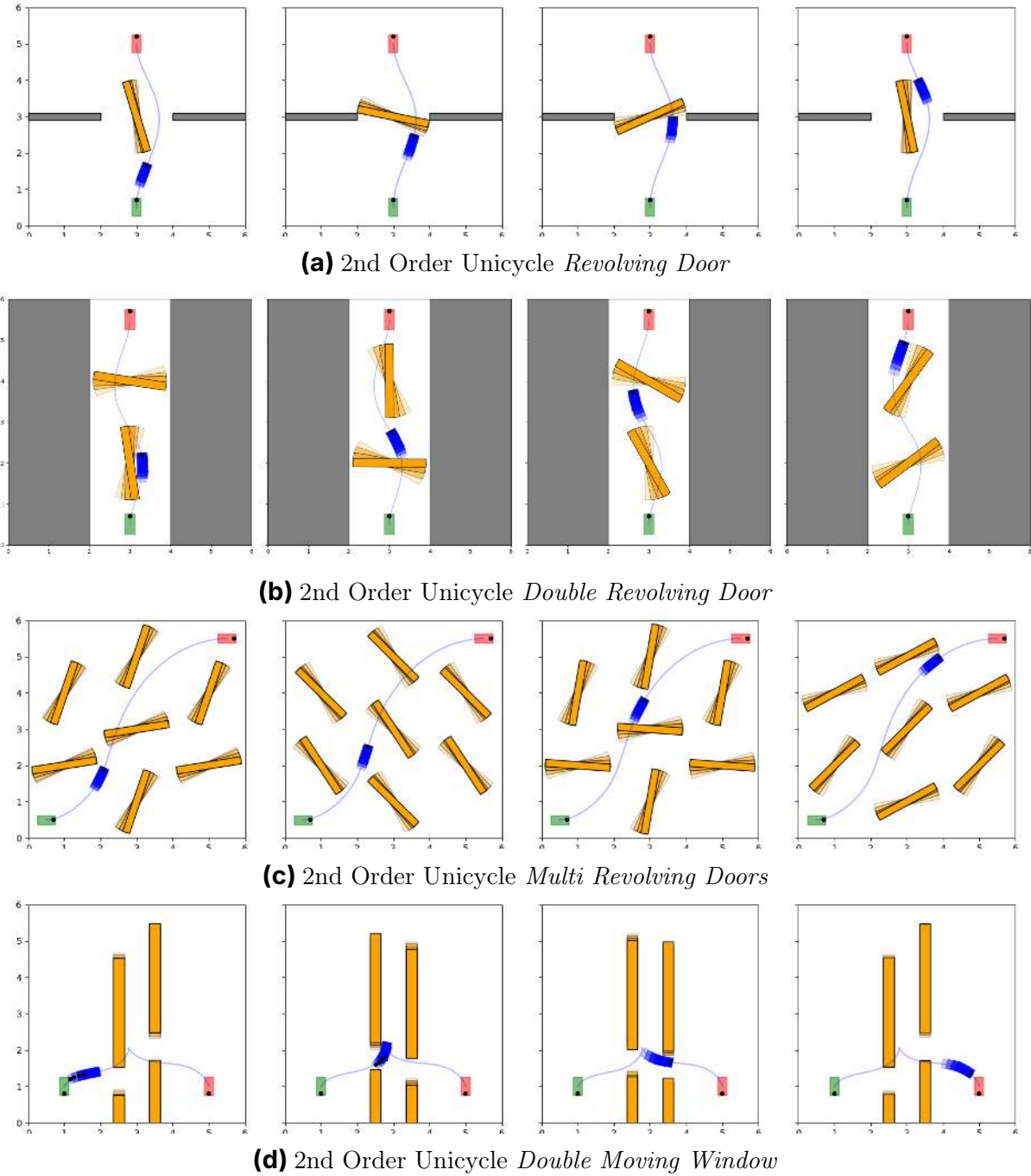
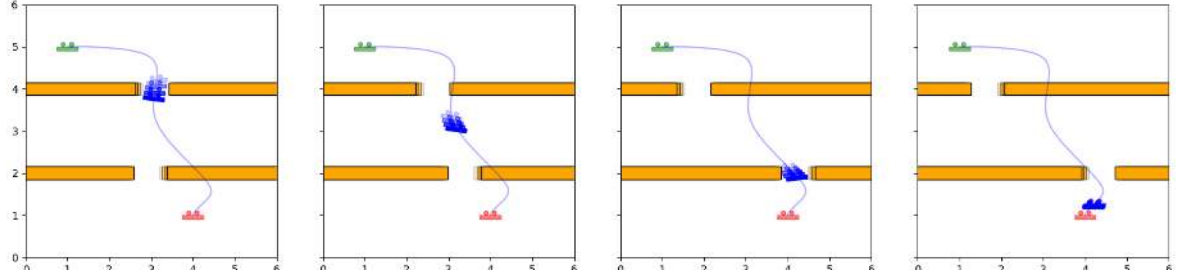
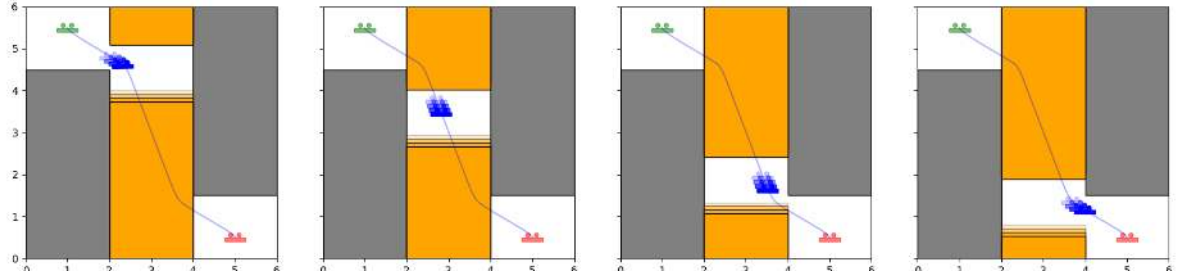


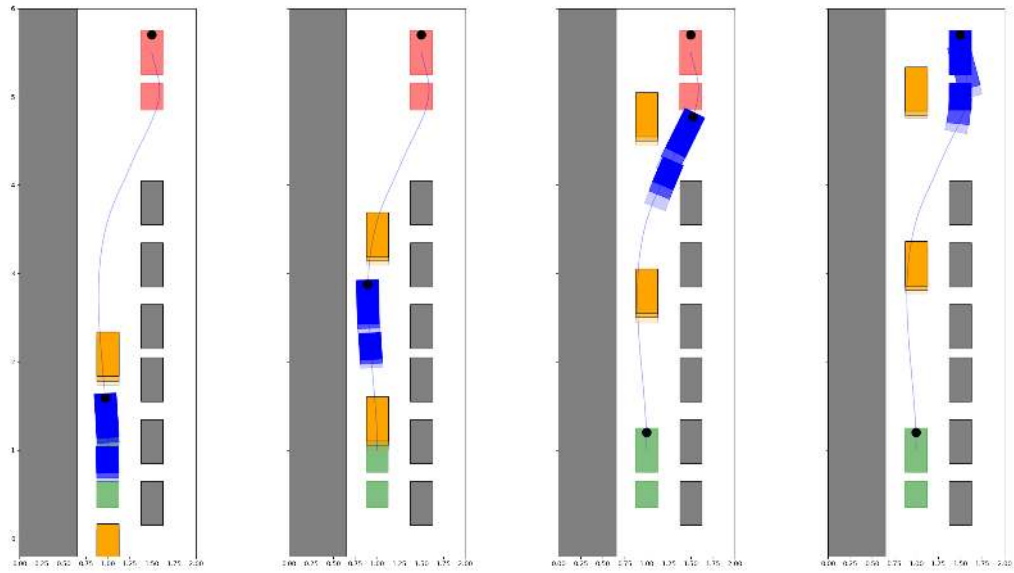
Figure A.5: A selection of solution trajectories for the 2nd Order Unicycle system.



(a) Planar Rotor *Double Window*



(b) Planar Rotor *Elevator*



(c) 1st Order Car *Lanes Switch*